



UNIVERSITÀ
DEGLI STUDI
FIRENZE

Scuola di Scienze Matematiche Fisiche e Naturali

Laurea Magistrale in Data Science,
Calcolo Scientifico & Intelligenza Artificiale

**Apprendimento per Rinforzo Multi-Agente
basato su Reti Neurali a Grafo
per il Controllo Semaforico**

Relatore

Prof. Daniele Castellana

Candidato

Alessio Minati

Anno Accademico 2025/2026

Sommario

Il traffico veicolare nelle aree urbane attraversa incroci regolati da semafori, e il modo in cui le fasi vengono decise incide sui tempi di percorrenza e sulla congestione della rete. Le soluzioni classiche, dai piani a tempi fissi ai sistemi coordinati come SCOOT e SCATS, si basano su euristiche definite manualmente e restano per questo poco flessibili di fronte a condizioni di traffico diverse da quelle previste in fase di progettazione. Il reinforcement learning offre un'alternativa, apprendendo una politica di controllo dall'esperienza simulata; le architetture più recenti coordinano gli agenti di intersezioni vicine tramite reti neurali a grafo, ma tendono a concentrarsi sull'efficienza aggregata, senza trattare l'equità tra le direzioni di traffico come obiettivo esplicito né verificare sistematicamente la generalizzazione a condizioni mai osservate in addestramento.

Questo lavoro usa come punto di partenza MetaSTGAT, un'architettura in cui ogni incrocio è controllato da un agente che apprende e gli agenti si scambiano informazioni con gli incroci vicini tramite una rete neurale a grafo, per condurre uno studio sperimentale del compromesso tra efficienza, equità e generalizzazione. L'ambiente, costruito su CityFlow, riproduce reti a griglia di tre dimensioni con traffico generato parametricamente; il confronto include le baseline Fixed-Time e Max-Pressure ed è organizzato in tre studi indipendenti, sul peso assegnato all'equità nella ricompensa, su un'ablazione sistematica delle modifiche introdotte, e sul meccanismo di comunicazione tra intersezioni. Ogni modello è valutato, senza riaddestramento, su topologie, spazature e profili di traffico mai incontrati in addestramento.

Il modello di riferimento riduce l'attesa massima subita dai veicoli del 34% in addestramento e del 41% in valutazione rispetto a Max-Pressure, a fronte di un tempo di percorrenza medio superiore di pochi punti percentuali. Sulle topologie più grandi mai viste in addestramento, Max-Pressure arriva a bloccare almeno un veicolo per l'intera durata dell'episodio in ogni configurazione testata, mentre il modello di riferimento resta entro limiti nettamente più contenuti. L'ablazione mostra inoltre che questo vantaggio dipende dalla combinazione tra il segnale di attesa nello stato

e il termine dedicato nella ricompensa, non da un singolo dettaglio dell'algoritmo di apprendimento.

Questi risultati indicano che l'equità tra direzioni di traffico può essere perseguita senza una rinuncia sostanziale all'efficienza, e che parte di questo vantaggio si conserva su condizioni diverse da quelle di addestramento.

Indice

Sommario	i
Acronimi e Notazione	xi
1 Introduzione	1
1.1 Contesto e motivazioni	1
1.2 Obiettivi e domande di ricerca	2
1.3 Contributi di questo lavoro	3
1.4 Struttura della tesi	4
2 Background	5
2.1 Reti Neurali	5
2.2 Reinforcement Learning	9
2.2.1 Il paradigma agente-ambiente	9
2.2.2 Processo decisionale di Markov	10
2.2.3 Apprendimento model-based e model-free	11
2.2.4 Apprendimento online e offline	11
2.2.5 Ampiezza e profondità dell'aggiornamento	12
2.2.6 Q-learning e Deep Q-Network	14
2.2.7 Esplorazione ε -greedy	15
2.2.8 Estensioni usate in questo lavoro	16
2.2.9 Apprendimento multi-agente	17
2.3 Reti Neurali Ricorrenti	18
2.4 Grafi e Graph Neural Network	21
2.4.1 Cos'è un grafo	21
2.4.2 Message passing	23
2.4.3 Invarianza ed equivarianza per permutazione	26
2.4.4 Reti spatio-temporali su grafo	26
2.4.5 Pesi generati da meta-learning	27
2.4.6 Propagazione a lungo raggio	27

2.5	Traffic Signal Control come problema	28
2.5.1	Terminologia e metriche	28
2.5.2	Due generazioni di controllo	29
2.5.3	Simulazione	31
2.5.4	Controllo semaforico appreso: stato dell'arte	31
2.6	Sintesi	33
3	Metodologia	35
3.1	Formalizzazione del problema	35
3.2	Architettura MetaSTGAT	39
3.2.1	State Encoder duale	40
3.2.2	Modulo spazio-temporale corrente (CST)	41
3.2.3	Modulo spaziale corrente (CS)	43
3.2.4	Testa Q-value	43
3.2.5	Meta-knowledge learner	43
3.2.6	Meta-GAT	47
3.2.7	Meta-LSTM	48
3.2.8	Meccanismi spaziali alternativi	49
3.3	Procedura di addestramento	53
3.4	Differenze rispetto al riferimento originale	57
3.5	Ambiente di simulazione	58
3.5.1	Struttura della rete e mappatura delle fasi	58
3.5.2	Gestione dell'interverde	60
3.5.3	Dataset e generazione parametrica	60
3.5.4	Assunzioni e limiti del modello di simulazione	65
4	Esperimenti e risultati	67
4.1	Metriche di valutazione	67
4.2	Protocollo sperimentale	69
4.3	Confronti proposti	70
4.3.1	Studio del peso α	70
4.3.2	Studio di ablazione	71
4.3.3	Studio sui meccanismi spaziali	71
4.4	Risultati	72
4.4.1	Fixed-Time, Max-Pressure e Pro: il confronto di base	72
4.4.2	Studio del peso α : risultati	74
4.4.3	Studio di ablazione: risultati	76
4.4.4	Studio sui meccanismi spaziali: risultati	78

4.4.5	Tabelle dei risultati	82
5	Conclusioni	87

Elenco delle figure

2.1	Il percettrone: gli input x_1, x_2, x_3 vengono combinati con i pesi w_1, w_2, w_3 e il bias b , la somma z attraversa la funzione di attivazione, producendo l'output $\hat{y}$	6
2.2	Le tre funzioni di attivazione discusse: sigmoide, tangente iperbolica e ReLU.	6
2.3	Una rete feedforward con un input layer a 4 nodi, un hidden layer a 5 nodi e un output layer a un nodo.	7
2.4	Il ciclo agente-ambiente.	9
2.5	Lo spazio dei metodi di apprendimento e pianificazione, classificati secondo la profondità dell'aggiornamento (asse orizzontale, superficiale a sinistra, profondo a destra) e la sua ampiezza (asse verticale, completo in alto, campionato in basso); le etichette in figura usano i termini inglesi <i>shallow/deep backups</i> per l'asse orizzontale e <i>full/sample backups</i> per il verticale. I quattro angoli corrispondono a dynamic programming, ricerca esaustiva, Temporal Difference e Monte Carlo. Adattata da Sutton and Barto (2018).	13
2.6	La rete ricorrente srotolata lungo il tempo.	19
2.7	La cella di memoria dell'LSTM.	20
2.8	Esempi di grafo diretto (sinistra) e non diretto (destra).	22
2.9	Lo schema generale di message passing impilato su più layer.	24
2.10	Il calcolo dell'attenzione per il nodo 1 rispetto al proprio vicinato.	25
3.1	Schema a blocchi dell'architettura STGAT: dallo stato dell'intersezione s_i^t , attraverso l'encoder duale (MLP) e i moduli CS e CST (GAT e LSTM a pesi condivisi), fino all'azione a_i^t scelta dal Double DQN (il blocco "DQN" della figura, adattata dal riferimento originale, ne indica la testa dei Q-value). Adattato da Wang et al. (2022).	40

3.2	Principio del meta-apprendimento applicato da MetaSTGAT: le meta-feature spaziali e temporali condizionano, tramite un meta-learner dedicato, i pesi dei layer LSTM e GAT usati per elaborare lo stato corrente del traffico. Adattato da Wang et al. (2022).	44
3.3	Il Meta-knowledge Learner: le meta-feature spaziali e temporali grezze vengono elaborate rispettivamente dallo SMK-Learner e dal TMK-Learner, producendo gli embedding $SMK(i)$ e $TMK(i)$ usati da Meta-GAT e Meta-LSTM. Adattato da Wang et al. (2022).	45
3.4	Meta-GAT: il modulo CS (sinistra) genera i pesi (W_j, b_j) della propria GAT a partire da SMK, il modulo CST (destra) genera (W_k, b_k) a partire da TMK; entrambi producono l'output z_i del rispettivo modulo. Adattato da Wang et al. (2022).	48
3.5	Meta-LSTM "srotolata" nel tempo: a ogni istante t il meta-learner genera, da $TMK(i)$, i pesi W_Φ e b_Φ della cella LSTM che produce x_i^t dall'input e_i^t e dallo stato nascosto precedente h_i^{t-1}, c_i^{t-1} . Adattato da Wang et al. (2022).	49
3.6	Schema a blocchi completo di MetaSTGAT: rispetto a STGAT (Figura 3.1), GAT e LSTM sono sostituiti dalle rispettive versioni meta-condizionate, con i pesi generati dagli embedding $SMK(i)$ e $TMK(i)$ del Meta-knowledge Learner. Adattato da Wang et al. (2022).	50
3.7	Rendering del visualizzatore di CITYFLOW per una singola intersezione a quattro vie: le indicazioni verde/rosso ai margini dell'incrocio mostrano i movimenti abilitati dalla fase corrente, i rettangoli colorati sulle corsie i veicoli in transito o in coda.	58
3.8	Rappresentazione grafica delle 8 fasi di Tabella 3.3: le frecce indicano i movimenti abilitati da ciascuna fase.	59
3.9	Rendering nel visualizzatore di CITYFLOW delle quattro combinazioni topologia-spaziatura di Tabella 3.4.	62
4.1	Travel time medio (fascia scura), max wait (fascia intermedia) e travel time massimo (fascia chiara) per lo studio sistematico su α , stessi dati delle Tabelle 4.4-4.5.	75
4.2	Variazione percentuale di travel time medio e massimo rispetto a Max-Pressure (riferimento a 0%) per lo studio su α , calcolata per singola configurazione e poi mediata.	75
4.3	Travel time medio, max wait e travel time massimo sui preset di ablation, stessi dati delle Tabelle 4.6-4.7.	77

4.4	Variazione percentuale di travel time medio e massimo rispetto a Max-Pressure (riferimento a 0%) sui preset di ablation, calcolata per singola configurazione/seed e poi mediata.	78
4.5	Travel time medio, max wait e travel time massimo sui meccanismi spaziali, stessi dati delle Tabelle 4.8-4.9.	81
4.6	Variazione percentuale di travel time medio e massimo rispetto a Max-Pressure (riferimento a 0%) sui meccanismi spaziali, calcolata per singola configurazione/seed e poi mediata.	81

Elenco delle tabelle

3.1	Iperparametri di addestramento e architettura del presente studio. . .	56
3.2	Sintesi delle differenze tra questo lavoro e il riferimento originale. . . .	57
3.3	Le 8 fasi semaforiche e i movimenti che ciascuna abilita.	59
3.4	Utilizzo di ciascuna combinazione topologia-spaziatura. Il dettaglio del confine tra training e validazione sulla griglia 4×4 a 100m è in Tabella 3.6.	61
3.5	Le cinque fasce del profilo di addestramento <i>complete</i> (totale: 4.728 veicoli per train1 ; train2 e train3 seguono la stessa struttura con lievi variazioni tra seed, per un totale rispettivamente di 4.723 e 4.620 veicoli).	64
3.6	Tutte le combinazioni di griglia, spaziatura e profilo di traffico impie- gate in questo lavoro, a meno delle varianti seed di validazione e test, con il relativo utilizzo e il nome del file di configurazione corrispondente.	65
4.1	Preset di ablazione e sottosistema isolato da ciascuno. La reward custom è definita da Eq. (3.4), mentre la reward paper è definita da Eq. (3.3).	71
4.2	Travel time medio, massimo, max wait e trip completion dei tre con- trollori presenti in ogni studio di questo capitolo, mediati sulle 3 configurazioni di training.	72
4.3	Come Tabella 4.2, mediata sulle 7 configurazioni di test.	73
4.4	Travel time medio, massimo e max wait, mediati sulle 3 configurazioni di training.	74
4.5	Come Tabella 4.4, mediata sulle 7 configurazioni di test.	74
4.6	Travel time medio, massimo e max wait sui preset di ablazione, me- diati sulle 3 configurazioni di training.	76
4.7	Come Tabella 4.6, mediata sulle 7 configurazioni di test.	77
4.8	Travel time medio, massimo e max wait sui meccanismi spaziali, me- diati sulle 3 configurazioni di training.	79
4.9	Come Tabella 4.8, mediata sulle 7 configurazioni di test.	79

4.10	Percentuale di trip completion tc per ogni modello e ogni configurazione, corretta escludendo dal denominatore i veicoli strutturalmente impossibilitati ad arrivare in tempo. Nessun modello scende sotto l'80%; a parte Fixed-Time (baseline non adattiva) l'unico caso ricorrente sotto il 90% è MetaSTGCN 1L.	83
4.11	Travel time medio $\overline{tt}$ (s) per ogni modello e ogni configurazione. Sulle configurazioni di test, media $\pm$ deviazione standard su 5 seed diversi.	84
4.12	Travel time massimo $tt_{\max}$ (s) per ogni modello e ogni configurazione. Sulle configurazioni di test, media $\pm$ deviazione standard su 5 seed diversi.	85
4.13	Max wait $wait_{\max}$ (s) per ogni modello e ogni configurazione. Sulle configurazioni di test, media $\pm$ deviazione standard su 5 seed diversi. Sulle config 4x4-200, valori a piena corsia.	86

Acronimi e Notazione

Acronimi

Acronimo	Significato
MDP	Markov Decision Process
RL	Reinforcement Learning
DQN	Deep Q-Network
PER	Prioritized Experience Replay
BPTT	Backpropagation Through Time
GNN	Graph Neural Network
GCN	Graph Convolutional Network
GAT	Graph Attention Network
TSC	Traffic Signal Control
SMK	Spatial Meta-Knowledge (learner)
TMK	Temporal Meta-Knowledge (learner)
CS	Cooperation Spatial (modulo Meta-GAT)
CST	Cooperation Spatial-Temporal (modulo Meta-GAT)

Notazione principale

Simbolo	Significato
$\mathcal{S}, \mathcal{A}, P, R, \gamma$	Stati, azioni, transizione, ricompensa, sconto
π, V^π, Q^π	Politica, funzione valore-stato, funzione valore-azione
θ, θ^-	Pesi della rete online e della target network
τ	Coefficiente di soft update
α (in RL)	Esponente di prioritizzazione del PER
α (in questo lavoro)	Peso della penalità anti-starvation nella ricompensa
ε	Probabilità di esplorazione
s_i^t, a_i^t, r_i^t	Stato, azione, ricompensa dell'intersezione i al tempo t
n_i^t, w_i^t, p_i^t	Veicoli per corsia, attesa massima per corsia, fase corrente
d_0	Dimensione dello stato in ingresso
d_1	Dimensione nascosta del modello
H, d_h	Numero di teste di attenzione e dimensione per testa
e_i^t, e_j^t	Embedding temporale e spaziale del Dual State Encoder
x_i^t, h_i^t, c_i^t	Output, stato nascosto e stato di cella del Meta-LSTM
$\text{SMK}(i), \text{TMK}(i)$	Meta-embedding spaziale e temporale dell'intersezione i
$z_{\text{CS}}(i), z_{\text{CST}}(i)$	Output dei moduli Meta-GAT spaziale e spazio-temporale
$\bar{t}t, tt_{\max}$	Travel time medio e massimo
tc	Trip completion, percentuale di veicoli che completano il percorso
$\text{wait}_{\max}$	Max wait, tempo massimo di un veicolo sulla stessa corsia senza superare l'incrocio

Capitolo 1

Introduzione

1.1 Contesto e motivazioni

Il traffico veicolare nelle aree urbane attraversa la rete stradale passando per una sequenza di incroci regolati da semafori. Il modo in cui le fasi vengono decise incide direttamente sui tempi di percorrenza e sulla congestione complessiva della rete, rendendo il controllo semaforico un problema di interesse pratico oltre che di ricerca.

Un incrocio semaforizzato stabilisce periodicamente quali flussi veicolari autorizzare e quali arrestare. Su una singola intersezione il problema è garantire che i veicoli in attesa continuino a scorrere in modo regolare. Le soluzioni classiche affrontano questo compito in due modi: un piano a tempi fissi che ripete una sequenza di fasi di durata prestabilita, calcolata una tantum su stime di traffico medie e mai più rivista in tempo reale, e un controllo attuato dai sensori che concede il verde a una direzione solo quando un sensore rileva la presenza di un veicolo in coda, adattandosi così, entro certi limiti, alle variazioni del traffico osservato.

Se si considera una rete composta da più incroci, il problema cambia natura. La fase scelta a un'intersezione influenza, con un ritardo proporzionale alla distanza, il volume di veicoli in arrivo alle intersezioni adiacenti, per cui le decisioni prese nei singoli punti della rete non possono più essere indipendenti tra loro. Sistemi come SCOOT e SCATS rispondono a questa esigenza da diversi decenni, aggiustando periodicamente i piani semaforici di un'intera area urbana sulla base del traffico rilevato in tempo reale (Wei et al., 2020). Il controllo semaforico su rete si configura quindi come un problema di coordinamento distribuito, inteso come la necessità per un insieme di processi autonomi di concordare su un'azione comune e coerente senza un'autorità centrale.

Entrambe le famiglie di soluzioni, dai piani a tempi fissi ai sistemi coordinati come SCOOT e SCATS, si basano su euristiche definite manualmente e restano per

questo poco flessibili di fronte a condizioni di traffico diverse da quelle previste in fase di progettazione.

Valutare un controllore semaforico richiede di tenere conto di due proprietà distinte. La prima è l'efficienza, la capacità di ridurre il tempo che i veicoli impiegano ad attraversare la rete. La seconda è l'equità, la garanzia che nessuna direzione di traffico venga fatta attendere sistematicamente più a lungo delle altre. Le due proprietà non sono la stessa cosa e non migliorano necessariamente insieme. Un controllore può massimizzare il flusso complessivo e allo stesso tempo lasciare una minoranza di veicoli bloccata a lungo, se la sua logica di decisione non include alcuna attenzione esplicita al caso peggiore. In questo studio dunque un controllore che fa aspettare pochi veicoli molto a lungo non è preferibile a uno leggermente più lento in media ma che non lascia mai nessuno bloccato per un tempo indefinito.

Le soluzioni proposte per affrontare questo compromesso si dividono in due famiglie. Da un lato regole scritte a mano come MAX-PRESSURE (Varaiya, 2013), che selezionano a ogni istante la fase capace di liberare più veicoli di quanti ne farebbe accumulare: un approccio dimostrabilmente efficiente nel lungo periodo, ma per costruzione miope rispetto all'equità, poiché ottimizza sempre il valore istantaneo più alto senza mai considerare da quanto tempo attende la direzione esclusa. Dall'altro il controllo appreso tramite reinforcement learning, dove una politica viene addestrata a massimizzare una ricompensa invece di seguire una regola scritta a mano. Applicato a una rete di più intersezioni, un'architettura recente affida a ogni incrocio un agente che apprende, mentre gli agenti si scambiano informazioni con gli incroci vicini attraverso una rete neurale a grafo, permettendo a ciascuna intersezione di condizionare la propria decisione sullo stato dei vicini senza una politica centralizzata. Le architetture di questo tipo, tra cui MetaSTGAT (Wang et al., 2022), il modello di riferimento di questo lavoro, raggiungono buoni risultati di efficienza aggregata, ma tipicamente non trattano l'equità come obiettivo esplicito di progettazione né verificano sistematicamente la generalizzazione a topologie mai osservate in addestramento. Questo lavoro si concentra proprio su questa famiglia di approcci, reinforcement learning coordinato tramite reti neurali a grafo, usando MetaSTGAT come architettura di partenza.

1.2 Obiettivi e domande di ricerca

L'obiettivo di questo lavoro è indagare sperimentalmente se, e a quali condizioni, un controllore appreso di questo tipo possa perseguire efficienza ed equità congiun-

tamente, mantenendo le prestazioni osservate anche su condizioni mai incontrate in addestramento.

A questo scopo, questo lavoro usa l'architettura di MetaSTGAT come punto di partenza, introducendo un termine dedicato della ricompensa che penalizza l'attesa massima osservata su una corsia non ancora servita dalla fase corrente, applicato a ogni singolo passo di decisione e per ogni intersezione. Per verificare se questo meccanismo funzioni davvero, la valutazione si affida a una metrica puramente osservazionale, calcolata allo stesso modo per ogni controllore indipendentemente da come è stato addestrato: il tempo massimo che un veicolo qualunque trascorre sulla stessa corsia senza superare l'incrocio, in una qualunque intersezione della rete.

Questo lavoro esplora tre domande di ricerca.

1. Un controllore appreso e coordinato tramite reti neurali a grafo può ridurre l'attesa massima subita dai veicoli rispetto a un controllore puramente reattivo come MAX-PRESSURE? E a quale costo in termini di efficienza media?
2. Tale vantaggio si mantiene su topologie di rete, spaziatore stradali e profili di traffico mai incontrati in addestramento?
3. Quali tra le scelte progettuali e le modifiche studiate in questo lavoro, dal meccanismo di comunicazione spaziale ai dettagli della procedura di addestramento, contribuiscono effettivamente al comportamento osservato?

1.3 Contributi di questo lavoro

Nello specifico, i contributi di questa tesi sono i seguenti.

Un ambiente di simulazione multi-intersezione riproducibile. Questa tesi costruisce da zero un ambiente su CITYFLOW (Zhang et al., 2019), con reti stradali a griglia di dimensione 4×4 , 5×5 e 6×6 , due spaziatore stradali distinte, otto fasi semaforiche per intersezione, insieme a una generazione parametrica dei dati di traffico con profili a intensità costante, a gradino e a giornata lavorativa. In ogni configurazione il traffico su una riga della griglia è potenziato artificialmente per simulare un'arteria stradale, condizione necessaria per poter osservare uno squilibrio direzionale in caso di traffico asimmetrico.

Uno studio sistematico del compromesso tra efficienza ed equità. Variando il peso assegnato alla componente di equità nella ricompensa, questa tesi quantifica direttamente il compromesso che il meccanismo introduce, mostrando in

particolare che rimuoverlo del tutto non produce un semplice guadagno di efficienza, ma un collasso dell'equità osservata verso il comportamento di MAX-PRESSURE.

Uno studio di ablazione sull'origine del comportamento osservato. Questa tesi isola singolarmente ciascuna delle modifiche introdotte rispetto alla procedura di riferimento, dal meccanismo di stato e ricompensa fino ai dettagli dell'algoritmo di apprendimento, invece di presentarle come un insieme indistinto di miglioramenti, per valutare empiricamente il contributo di ciascuna al comportamento osservato.

Un confronto tra meccanismi di comunicazione spaziale a parità di resto dell'architettura. Questa tesi confronta tre meccanismi di comunicazione tra intersezioni, *graph attention*, *graph convolution* e propagazione a onda ispirata a SONAR (Trenta et al., 2025), mantenendo identico il resto dell'architettura, per verificare se il comportamento osservato dipenda specificamente dal meccanismo di attenzione appreso o sia una proprietà più generale della coordinazione locale via rete neurale a grafo.

Un protocollo di valutazione a generalizzazione sistematica. Ogni modello di questa tesi, appreso o classico, è valutato senza riaddestramento su topologie, spazature e profili di traffico mai incontrati in addestramento, riportando media e caso peggiore del tempo di viaggio, la percentuale di veicoli che completano il proprio percorso e la massima attesa registrata da un veicolo allo stesso incrocio.

1.4 Struttura della tesi

Il Capitolo 2 introduce il background necessario a leggere i capitoli successivi senza assumere familiarità pregressa con nessuno dei campi coinvolti, dal reinforcement learning alle reti neurali ricorrenti, dalle reti neurali a grafo al controllo semaforico. Il Capitolo 3 formalizza il problema affrontato, presenta l'architettura di MetaSTGAT e le modifiche studiate in questo lavoro, i meccanismi di comunicazione spaziale alternativi e la procedura di addestramento adottata, e descrive infine l'ambiente di simulazione. Il Capitolo 4 descrive il protocollo sperimentale e riporta i risultati ottenuti sui tre studi, peso dell'equità, ablation, meccanismi spaziali. Il Capitolo 5 discute i risultati alla luce degli obiettivi dichiarati in questa introduzione, indica le direzioni lasciate aperte e dichiara i limiti dello studio.

Capitolo 2

Background

Questo capitolo introduce i concetti necessari per leggere i Capitoli 3 e 4 senza assumere familiarità pregressa con i campi coinvolti. Si parte dalle reti neurali e dal loro addestramento (§2.1), per poi presentare il reinforcement learning, dal paradigma agente-ambiente alle estensioni del DQN usate in questo lavoro (§2.2), le reti neurali ricorrenti (§2.3) e le reti neurali a grafo, incluse le formulazioni confrontate sperimentalmente nel Capitolo 4 (§2.4). Il capitolo si chiude presentando il controllo semaforico come problema applicativo, la letteratura che lo affronta e ciò che resta aperto (§2.5).

2.1 Reti Neurali

Una **rete neurale** è una funzione parametrica che trasforma un input in un output attraverso una successione di funzioni matematiche, i cui parametri, chiamati pesi, vengono appresi dai dati invece di essere codificati manualmente. L'unità base è il *neurone artificiale* o *perceptrone*, mostrato in Figura 2.1: esso riceve un vettore di input $x = (x_1, x_2, \dots, x_n) \in \mathbb{R}^n$, ne calcola una combinazione lineare pesata $z = w^\top x + b = \sum_{i=1}^n w_i x_i + b$ (con $w = (w_1, w_2, \dots, w_n) \in \mathbb{R}^n$ il vettore di pesi e $b \in \mathbb{R}$ un termine costante detto *bias*), e applica a z una funzione non lineare σ detta *funzione di attivazione*, producendo l'output $\hat{y} = \sigma(z)$.

Attivazioni comuni, mostrate in Figura 2.2, sono la sigmoide $\sigma(z) = 1/(1 + e^{-z})$, che comprime z in $(0, 1)$, la tangente iperbolica $\tanh(z) = (e^z - e^{-z})/(e^z + e^{-z})$, che lo comprime invece in $(-1, 1)$, e la ReLU (Rectified Linear Unit) $\max(0, z)$, che si limita ad azzerare i valori negativi. La non linearità è essenziale: senza di essa, comporre più trasformazioni lineari produrrebbe comunque una singola trasformazione lineare, non riuscendo ad apprendere delle relazioni complesse e non lineari presenti nei dati.

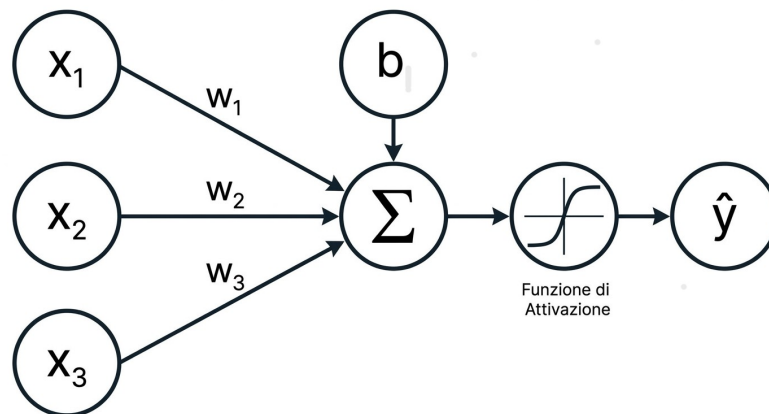


Figura 2.1: Il perceptrone: gli input x_1, x_2, x_3 vengono combinati con i pesi w_1, w_2, w_3 e il bias b , la somma z attraversa la funzione di attivazione, producendo l'output $\hat{y}$.

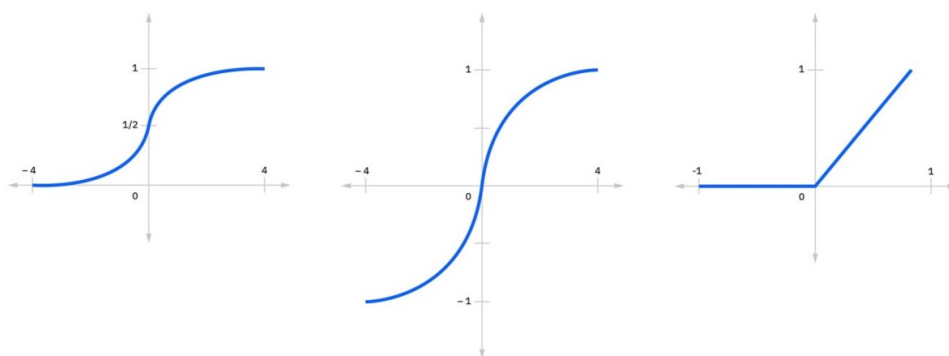


Figura 2.2: Le tre funzioni di attivazione discusse: sigmoide, tangente iperbolica e ReLU.

Una rete neurale *feedforward* o *multi-layer perceptron* (MLP) dispone i neuroni in **strati** (*layer*) successivi: l'output di ogni strato è l'input dello strato seguente, dall'*input layer* (i dati grezzi) attraverso uno o più *hidden layer* fino all'*output layer* finale, come in Figura 2.3. In forma matriciale, un singolo layer calcola

$$h^{(l+1)} = \sigma(W^{(l)}h^{(l)} + b^{(l)})$$

dove $h^{(l)}$ raccoglie gli output di tutti i neuroni dello strato l , $W^{(l)}$ è la matrice che impila i pesi di ciascun neurone di quello strato e $b^{(l)}$ è il vettore dei bias. Impilare più strati permette alla rete di comporre trasformazioni via via più astratte dell'input originale.

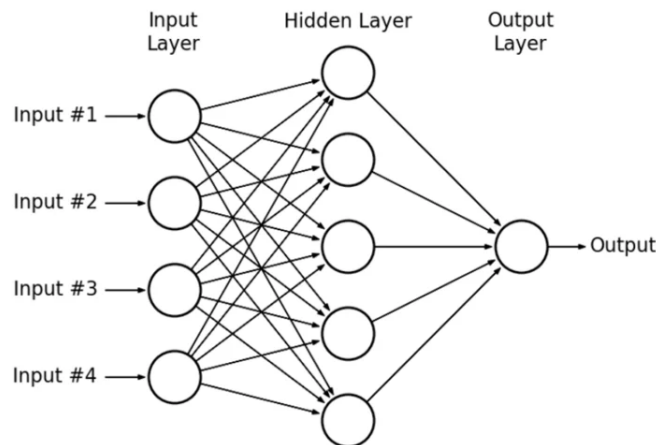


Figura 2.3: Una rete feedforward con un input layer a 4 nodi, un hidden layer a 5 nodi e un output layer a un nodo: ogni connessione porta con sé un peso appreso, omesso per chiarezza. Fonte: Wikimedia Commons.¹

Addestramento: discesa del gradiente e backpropagation. Una rete neurale viene inizializzata con pesi e bias casuali e viene addestrata iterativamente confrontando l'output prodotto con quello desiderato. Una funzione di loss $\mathcal{L}(\hat{y}, y)$ quantifica la distanza tra l'output prodotto $\hat{y}$ e quello desiderato y (che sia esso una etichetta di classificazione o un valore reale nel caso della regressione). L'addestramento consiste nel regolare ogni peso nella direzione che riduce $\mathcal{L}$, un'operazione nota come *discesa del gradiente*:

$$w \leftarrow w - \eta \frac{\partial \mathcal{L}}{\partial w},$$

dove η , il *tasso di apprendimento* o *learning rate*, controlla l'ampiezza di ogni aggiornamento. Calcolare la derivata parziale $\partial \mathcal{L} / \partial w$ per ogni peso della rete, dal

più vicino all'output al più lontano, richiede applicare ripetutamente la *chain rule* del calcolo differenziale attraverso tutti gli strati intermedi: l'algoritmo che esegue questo calcolo in modo efficiente, una sola passata dall'output verso l'input, è la *backpropagation* (Rumelhart et al., 1986). Nella pratica, il gradiente non è esatto ma si stima su un piccolo sottoinsieme casuale di dati (*mini-batch*) alla volta invece che sull'intero insieme di dati disponibile, una variante nota come discesa del gradiente stocastica, molto più economica computazionalmente a fronte di una stima del gradiente più rumorosa ma sufficiente a guidare comunque l'addestramento verso una soluzione utile.

Oltre la discesa del gradiente pura. La regola di aggiornamento vista sopra usa lo stesso tasso di apprendimento η per ogni peso della rete, un limite quando la superficie di loss ha curvatura molto diversa da un peso all'altro (alcune direzioni richiederebbero un passo grande, altre uno minimo): un ottimizzatore più sofisticato adatta l'ampiezza dell'aggiornamento parametro per parametro, sulla base della storia recente dei gradienti osservati. Il *momentum* (Qian, 1999) accumula una media mobile esponenziale dei gradienti passati e aggiorna i pesi in quella direzione media invece che nella sola direzione del gradiente corrente, smorzando le oscillazioni lungo le direzioni in cui il segno del gradiente cambia di continuo. **RMSprop** (Tieleman and Hinton, 2012) adotta un principio complementare: mantiene per ciascun peso una media mobile del quadrato dei gradienti recenti e divide l'aggiornamento per la sua radice, riducendo il passo sui pesi il cui gradiente è stato storicamente grande in valore assoluto e aumentandolo su quelli il cui gradiente è stato piccolo, così da rendere il progresso più uniforme tra pesi con scale molto diverse. **Adam** (Kingma and Ba, 2015), un altro ottimizzatore molto usato, combina invece entrambe le idee, media mobile del gradiente e del suo quadrato. Questo lavoro adotta **RMSprop**, coerentemente con il modello di riferimento (Wang et al., 2022), per l'addestramento delle proprie reti neurali.

Ogni rete usata in questo lavoro, dal DQN (§2.2) alla LSTM (§2.3) alle reti a grafo (§2.4), è in ultima analisi una particolare composizione di questi stessi due ingredienti: strati di neuroni con un'attivazione non lineare, addestrati per discesa del gradiente tramite backpropagation. Ciò che cambia da un'architettura all'altra è come gli strati sono collegati tra loro, non il principio di addestramento sottostante.

2.2 Reinforcement Learning

2.2.1 Il paradigma agente-ambiente

L'apprendimento per rinforzo o *reinforcement learning* (RL) è un paradigma di apprendimento automatico in cui un *agente* impara un certo comportamento interagendo ripetutamente con un *ambiente*, senza che gli siano forniti esempi di comportamento corretto. A ogni passo di tempo t , l'agente osserva lo stato corrente dell'ambiente, sceglie un'*azione* tra quelle disponibili, e riceve in cambio due informazioni: un nuovo stato (l'ambiente è cambiato, anche per effetto dell'azione scelta) e un numero scalare, la *ricompensa*, che indica quanto quell'azione sia stata desiderabile in quel momento. Il problema che il RL affronta è per l'appunto questo, cioè imparare per tentativi ripetuti quale comportamento massimizzi la somma delle ricompense ricevute nel tempo, non solo quella immediata. La Figura 2.4 riassume questo ciclo, utile come riferimento per la terminologia introdotta nel resto della sezione.

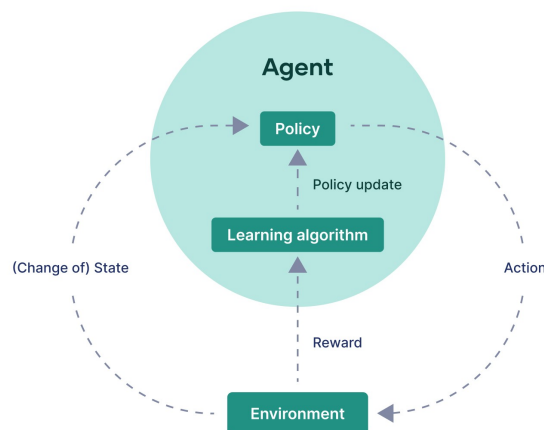


Figura 2.4: Il ciclo agente-ambiente: la policy sceglie un'azione, l'ambiente risponde con un nuovo stato e una ricompensa, e l'algoritmo di apprendimento usa quella ricompensa per aggiornare la policy. Fonte: adattata da un articolo Medium.²

Vale la pena distinguere questo paradigma da quello dell'*apprendimento supervisionato*. In quest'ultimo una rete neurale riceve coppie input-output correttamente associate e impara a riprodurre quella corrispondenza. Nel RL non esiste alcuna etichetta corretta da imitare: l'agente deve scoprire da solo, per tentativi, quale azione sia buona in ciascuno stato, guidato solo dalla ricompensa ricevuta in seguito alla azione che l'ha causata. È un problema strutturalmente più difficile, ma anche particolarmente adatto a situazioni, come il controllo semaforico, in cui nessuno conosce a priori quale sia la "decisione corretta" da etichettare.

2.2.2 Processo decisionale di Markov

La formalizzazione matematica standard di un problema di RL è il *processo decisionale di Markov* (MDP) (Sutton and Barto, 2018), definito dalla tupla

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$$

dove $\mathcal{S}$ è l'insieme di tutti gli stati possibili dell'ambiente, $\mathcal{A}$ l'insieme di tutte le azioni possibili, $P(s' | s, a)$ la probabilità di transitare nello stato s' avendo eseguito l'azione a nello stato s , $R(s, a)$ la ricompensa attesa per quella stessa coppia, e $\gamma \in [0, 1)$ il *fattore di sconto*, che pondera l'importanza delle ricompense future rispetto a quelle immediate: un γ vicino a 0 rende l'agente "miope", interessato quasi solo alla ricompensa immediata, un γ vicino a 1 lo rende attento anche a conseguenze lontane nel tempo. Questa formalizzazione si basa sull'assunzione di Markov, per la quale la probabilità di transizione $P(s' | s, a)$ dipende solo dallo stato corrente s e non dalla sequenza di stati che ha portato allo stesso.

Un agente segue una *politica* $\pi(a | s)$, una distribuzione di probabilità sulle azioni condizionata allo stato corrente; il suo obiettivo è massimizzare il *ritorno* atteso

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

ovvero la somma scontata delle ricompense future a partire dall'istante t . Per poter ragionare su quanto buona sia una politica prima ancora di eseguirla fino in fondo, si definiscono due funzioni valore: la funzione valore-stato $V^\pi(s) = \mathbb{E}_\pi[G_t | s_t = s]$ e la funzione valore-azione $Q^\pi(s, a) = \mathbb{E}_\pi[G_t | s_t = s, a_t = a]$, che misurano rispettivamente quanto ritorno atteso si ottiene seguendo π da uno stato, o da una coppia stato-azione. Queste soddisfano l'*equazione di Bellman*, che esprime il valore di uno stato in funzione del valore atteso degli stati successivi:

$$V^\pi(s) = \sum_a \pi(a | s) \sum_{s'} P(s' | s, a) [R(s, a) + \gamma V^\pi(s')] \quad (2.1)$$

Per una politica ottima π^* che massimizza $V^\pi(s)$ per ogni stato simultaneamente, vale l'analoga equazione di Bellman di ottimalità:

$$Q^*(s, a) = \mathbb{E}_{s' \sim P(\cdot | s, a)} \left[R(s, a) + \gamma \max_{a'} Q^*(s', a') \right] \quad (2.2)$$

Se si conoscesse Q^* , la politica ottima sarebbe immediata da ricavare: scegliere sempre $\operatorname{argmax}_a Q^*(s, a)$. Il problema del RL si riduce quindi, in buona parte, a stimare Q^* senza conoscerlo a priori.

2.2.3 Apprendimento model-based e model-free

Esiste una distinzione fondamentale tra due famiglie di algoritmi RL, a seconda che l'agente costruisca o meno una rappresentazione esplicita della dinamica dell'ambiente. Un algoritmo *model-based* apprende o riceve un modello approssimato di $P(s' | s, a)$ e $R(s, a)$, e lo usa per simulare le conseguenze di più sequenze di azioni prima di sceglierne una, senza eseguirle davvero nell'ambiente. Un algoritmo *model-free*, al contrario, non costruisce alcun modello della dinamica dell'ambiente, bensì stima direttamente Q^* dall'esperienza osservata, senza mai provare a prevedere esplicitamente lo stato successivo. Il vantaggio del model-free è la semplicità e la minore dipendenza dalla qualità di un modello che potrebbe essere difficile da apprendere accuratamente; lo svantaggio è un uso tipicamente meno efficiente dell'esperienza raccolta, poiché non la si riusa per simulare scenari mai osservati direttamente.

Questo lavoro adotta un approccio interamente model-free. Né il modello di riferimento (Wang et al., 2022) né le varianti implementate in questa tesi apprendono o usano un modello esplicito della dinamica del traffico. L'agente stima direttamente una funzione Q dall'esperienza simulata in CityFlow, con una famiglia di tecniche descritte nel resto di questa sezione.

2.2.4 Apprendimento online e offline

Una distinzione ortogonale a quella model-based/model-free appena vista riguarda da dove arriva l'esperienza usata per apprendere.

Un algoritmo *online* interagisce ripetutamente con l'ambiente durante l'addestramento, raccogliendo nuova esperienza con la politica corrente e usandola subito per migliorarsi, in un ciclo continuo di raccolta e apprendimento. Un algoritmo *offline*, al contrario, apprende da un insieme fisso di transizioni raccolto in precedenza, magari con una politica diversa o addirittura sconosciuta, senza alcuna ulteriore interazione con l'ambiente durante il training. Questo risulta essere un approccio necessario quando interagire con l'ambiente reale è costoso o rischioso. Un dettaglio che vale la pena chiarire subito è che un *replay buffer* come quello che il DQN (descritto a breve) introduce, non lo rende di per sé un metodo offline. Il buffer infatti si riempie continuamente di transizioni generate via via da una politica in evoluzione, con raccolta dati e apprendimento che procedono in parallelo, non in due fasi separate come negli algoritmi offline.

Questo lavoro è interamente online: ogni transizione usata per un aggiornamento proviene dalla simulazione in corso in CITYFLOW, mai da un insieme di dati raccolto in anticipo.

2.2.5 Ampiezza e profondità dell'aggiornamento

Una seconda distinzione, anch'essa ortogonale alle precedenti, riguarda il modo in cui si aggiorna una stima di valore combinando una ricompensa con la stima del valore in uno o più stati successivi. Due assi indipendenti classificano ogni aggiornamento, come mostrato in Figura 2.5.

Il primo asse riguarda l'**ampiezza** dell'aggiornamento. Un aggiornamento completo (*full backup* in figura) calcola l'aspettativa matematica considerando tutte le azioni possibili e tutti i possibili stati successivi s' , pesati dalla probabilità di transizione $P(s' | s, a)$ fornita da un modello noto dell'ambiente. Un aggiornamento campionato (*sample backup* in figura) si basa invece su una singola traiettoria o un'esperienza realmente osservata, senza calcolare tutte le diramazioni possibili.

Il secondo asse riguarda la **profondità** dell'aggiornamento. Un aggiornamento superficiale, o a corto raggio (*shallow backup* in figura), guarda solo a pochissimi passi nel futuro, nel caso di TD(0) un solo passo d'azione, e sostituisce il ritorno futuro con la stima già disponibile dell'ultimo passo considerato. Questa è una tecnica nota come *bootstrapping*, per cui si costruisce una stima a partire da un'altra stima invece che dall'esito osservato per intero. Un aggiornamento profondo, o a lungo raggio (*deep backup* in figura), tiene conto delle conseguenze fino alla fine dell'episodio, usando solo ricompense effettivamente realizzate e non stimate.

Le combinazioni dei due assi individuano quattro famiglie di metodi:

- **Completo e superficiale:** la *dynamic programming*, che calcola l'aspettativa esatta su tutti i successori s' ma si ferma dopo un solo passo, facendo bootstrapping sulla stima già disponibile del valore di s' invece di espandere ulteriormente l'albero.
- **Completo e profondo:** la *ricerca esaustiva*, che come la dynamic programming pesa ogni successore con la sua probabilità, ma che espande però l'intero albero delle conseguenze fino alla fine dell'episodio o a un orizzonte fissato. È il caso ideale in cui ci si potesse permettere di enumerare ogni possibile continuazione.
- **Campionato e profondo:** il *Monte Carlo* non usa $P(s' | s, a)$, osserva un'unica traiettoria effettivamente realizzata e ne percorre per intero le conseguenze fino alla fine dell'episodio, calcolando il ritorno effettivo $G_t = \sum_k \gamma^k r_{t+k}$ senza mai usare bootstrapping.
- **Campionato e superficiale:** il *Temporal Difference* (TD), che come Monte Carlo osserva una singola transizione campionata anziché un'aspettativa

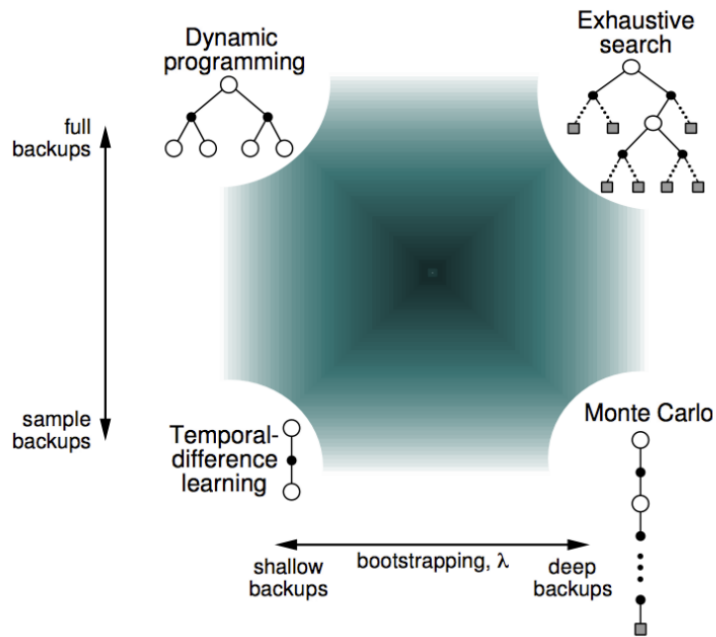


Figura 2.5: Lo spazio dei metodi di apprendimento e pianificazione, classificati secondo la profondità dell'aggiornamento (asse orizzontale, superficiale a sinistra, profondo a destra) e la sua ampiezza (asse verticale, completo in alto, campionato in basso); le etichette in figura usano i termini inglesi *shallow/deep backups* per l'asse orizzontale e *full/sample backups* per il verticale. I quattro angoli corrispondono a dynamic programming, ricerca esaustiva, Temporal Difference e Monte Carlo. Adattata da Sutton and Barto (2018).

sul modello, ma come la dynamic programming si ferma dopo un solo passo, facendo bootstrapping sulla stima corrente del valore dello stato successivo.

Nella sua versione più semplice di Temporal Difference, TD(0), l'aggiornamento della funzione valore-stato è

$$V(s_t) \leftarrow V(s_t) + \eta \left[(r_t + \gamma V(s_{t+1})) - V(s_t) \right],$$

dove il termine tra parentesi quadre è il *Temporal Difference error*, ovvero la discrepanza tra la stima corrente e un target calcolato guardando avanti di un solo passo. Il bootstrapping ha un costo, l'errore della stima usata come target si propaga nell'aggiornamento, ma un beneficio pratico decisivo rispetto a Monte Carlo: l'agente può aggiornarsi a ogni singolo passo, senza attendere la fine dell'episodio, un requisito essenziale per imparare da compiti molto lunghi o addirittura privi di una fine naturale. Il Q-learning descritto di seguito applica esattamente questo principio alla funzione valore-azione Q invece che alla sola funzione valore-stato V .

Questo lavoro si colloca nella casistica dell'aggiornamento campionato e superficiale, quello del Temporal Difference. L'assenza di un modello della dinamica del traffico (§2.2.3) esclude già di per sé la dynamic programming e la ricerca esaustiva, entrambe legate a un aggiornamento completo; inoltre la simulazione in CITY-FLOW procede per episodi lunghi e continui su intersezioni multiple, un contesto in cui attendere la fine dell'episodio per ogni aggiornamento, come farebbe Monte Carlo, sarebbe sia lento sia inutilmente rumoroso rispetto al bootstrap a un passo. Il Q-learning e il DQN che ne deriva, discussi di seguito, adottano infatti un aggiornamento TD, campionato e superficiale.

2.2.6 Q-learning e Deep Q-Network

Q-learning tabellare. Il **Q-learning** (Sutton and Barto, 2018) approssima Q^* applicando iterativamente l'Equazione 2.2 come regola di aggiornamento tramite bootstrapping: a ogni transizione osservata (s, a, r, s') , aggiusta il valore stimato $Q(s, a)$ verso il target $r + \gamma \max_{a'} Q(s', a')$, la stima corrente al passo successivo, invece di attendere il ritorno realizzato per intero a fine episodio come farebbe un metodo Monte Carlo. Il nome "tabellare" indica che questi aggiornamenti possono essere eseguiti considerando una tabella con una riga per ogni stato e una colonna per ogni azione. L'aggiornamento modifica una voce alla volta e ripetendo l'operazione su molte transizioni, ogni voce $Q(s, a)$ converge verso il vero ritorno atteso dell'eseguire a in s . Si ottiene in questo modo una politica ottima, senza il bisogno di indicazioni esplicite sulle azioni da intraprendere.

L'approccio tabellare ha però un limite rappresentato dalle dimensioni della tabella. Se lo spazio degli stati e lo spazio delle azioni sono continui o ad alta dimensionalità il numero di combinazioni possibili esplode, rendendo questo approccio impraticabile. Questo è il caso del controllo semaforico multi-agente, dove la dimensione dello spazio degli stati rende una tabella esplicita irrealizzabile sia da costruire sia da aggiornare.

Deep Q-Network (DQN). Mnih et al. (2015) risolvono il problema sostituendo la tabella con una rete neurale $Q(s, a; \theta)$ allenata ad approssimarne i valori: invece di una voce indipendente per ogni coppia (s, a) , la rete apprende una funzione continua che generalizza a stati mai visti interpolando tra stati simili osservati durante l'addestramento. I pesi θ vengono aggiornati per discesa del gradiente in modo che $Q(s, a; \theta)$ si avvicini allo stesso target di Bellman usato dal Q-learning tabellare, con la differenza che l'aggiornamento modifica l'intera rete invece di una singola voce, propagando l'informazione anche a stati simili mai osservati direttamente. È esattamente questa proprietà a rendere il DQN necessario in questo lavoro: lo spazio di stato del controllo semaforico multi-agente è troppo grande per una tabella, ma una rete neurale può generalizzare a partire da un numero gestibile di transizioni osservate durante la simulazione. Gli autori introducono due accorgimenti per stabilizzare questo schema, entrambi adottati anche in questo lavoro. Il primo è il *replay buffer*: una memoria in cui si accumulano le transizioni (s, a, r, s') osservate, da cui si campionano poi mini-batch in ordine casuale per l'addestramento, rompendo la correlazione temporale tra stati consecutivi di uno stesso episodio che violerebbe altrimenti l'assunzione di campioni indipendenti richiesta dalla discesa del gradiente stocastica. Come anticipato (§2.2.4), il buffer non cambia la natura online del metodo, in quanto si riempie di transizioni raccolte dalla politica corrente mentre l'addestramento procede, non da un insieme fisso raccolto in anticipo. Il secondo è la *target network* $Q(\cdot; \theta^-)$, una copia dei pesi della rete online sincronizzata solo periodicamente: calcolare il target $r + \gamma \max_{a'} Q(s', a'; \theta^-)$ con questa copia invece che con la rete online stessa, che cambia a ogni passo di gradiente, evita che il bersaglio dell'ottimizzazione si sposti insieme ai pesi che lo determinano, un problema noto come *moving target*.

2.2.7 Esplorazione ε -greedy

Un agente completamente *greedy*, che segue cioè sempre e solo l'azione a valore Q stimato più alto, non raccoglierebbe mai esperienza sulle conseguenze di azioni alternative, rischiando di convergere su una politica sub-ottima per via di una mancata

esplorazione. Ad esempio, se nelle prime fasi dell’addestramento un’azione appare leggermente migliore di un’altra, un agente completamente *greedy* la prediligerebbe sistematicamente, impedendo di fatto all’agente di scoprire il potenziale di strategie alternative. La tecnica più comune per bilanciare sfruttamento (*exploitation*, scegliere l’azione ritenuta migliore) ed esplorazione (*exploration*, raccogliere informazione su azioni meno note) è ϵ -greedy (Sutton and Barto, 2018): a ogni passo di decisione, con probabilità ϵ l’agente sceglie un’azione uniformemente a caso, e con probabilità $1 - \epsilon$ sceglie $\arg\max_a Q(s, a; \theta)$. Il valore di ϵ tipicamente decresce nel corso dell’addestramento, da un valore iniziale vicino a 1 (quasi solo esplorazione, quando la stima di Q è ancora inaffidabile) verso un valore finale piccolo ma non nullo (quasi solo sfruttamento, mantenendo comunque un minimo di esplorazione residua).

2.2.8 Estensioni usate in questo lavoro

Il DQN di base ammette diverse estensioni indipendenti, ciascuna delle quali corregge una specifica fonte di instabilità o inefficienza, discusse di seguito:

- Il **Double DQN** (van Hasselt et al., 2016) disaccoppia la selezione dell’azione dalla sua valutazione, usando la rete online per scegliere l’azione migliore e la target network, già introdotta sopra, solo per valutarla: $y = r + \gamma Q(s', \arg\max_{a'} Q(s', a'; \theta^-); \theta^-)$. Poiché la stessa rete che sceglie l’azione con il valore stimato più alto tende anche a sovrastimarne il valore, usare la rete online per scegliere e la target network per valutare riduce questo bias sistematico di sovrastima tipico del DQN standard, che userebbe la stessa rete per entrambi i passi.
- Il **Prioritized Experience Replay** (PER, Schaul et al., 2016) sostituisce il campionamento uniforme dal replay buffer con un campionamento proporzionale al *Temporal Difference error* δ_i . Le transizioni che la rete predice peggio vengono ricampionate più spesso, concentrando la capacità di apprendimento dove serve di più. Assegnando a ciascuna transizione una priorità $p_i = |\delta_i| + \zeta$, con ζ una piccola costante che evita una probabilità nulla, la probabilità di campionamento è $P(i) = p_i^\alpha / \sum_k p_k^\alpha$, con $\alpha \in [0, 1]$ iperparametro che ne controlla la dipendenza dall’errore. Per correggere il bias introdotto da un campionamento non uniforme, la loss viene pesata tramite coefficienti di *importance sampling* $w_i = (N \cdot P(i))^{-\beta}$, dove N è la dimensione del buffer e $\beta \in [0, 1]$ un iperparametro che cresce nel corso dell’addestramento verso 1, aumentando progressivamente il grado di correzione applicato.

- La **Huber loss** (o *smooth L1*) sostituisce la funzione di loss data dall'errore quadratico medio (MSE) con una funzione quadratica vicino allo zero e lineare lontano da esso, risultando meno sensibile agli outlier prodotti da transizioni con errore di stima molto grande (con MSE, un singolo errore enorme domina il gradiente dell'intero batch; con Huber il suo contributo cresce solo linearmente):

$$L_{\kappa}(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & \text{se } |y - \hat{y}| \leq \kappa \\ \kappa \left(|y - \hat{y}| - \frac{1}{2}\kappa \right) & \text{altrimenti} \end{cases} \quad (2.3)$$

Tipicamente $\kappa = 1$.

2.2.9 Apprendimento multi-agente

Quanto descritto finora assume un solo agente che interagisce con un solo ambiente. Quando più agenti operano nello stesso ambiente, ciascuno osservando solo una porzione dello stato globale e influenzando con le proprie azioni anche ciò che gli altri agenti osservano e ricevono come ricompensa, il problema si generalizza da MDP a *processo decisionale di Markov decentralizzato e parzialmente osservabile* (Dec-POMDP). Il termine "decentralizzato" si riferisce al fatto che non esiste un'unica azione scelta da un solo decisore, ma N azioni scelte indipendentemente da N agenti; "parzialmente osservabile" indica invece che ogni agente vede solo il proprio stato locale, non lo stato globale completo dell'ambiente. Nel contesto di questo lavoro, la decentralizzazione si concretizza nel fatto che ogni intersezione decide autonomamente la propria fase, mentre l'osservabilità parziale si concretizza nel campo visivo limitato di ciascuna intersezione (Capitolo 3), che esclude deliberatamente dallo stato osservato ciò che accade agli altri incroci.

Il multi-agente introduce una difficoltà che il caso a singolo agente non ha: la *non stazionarietà*. L'equazione di Bellman e l'intera teoria del Q-learning assumono che l'ambiente risponda in modo coerente nel tempo alla stessa azione nello stesso stato; ma se più agenti apprendono simultaneamente, ciascuno sta anche cambiando il proprio comportamento mentre gli altri lo osservano, cosicché per ogni singolo agente l'ambiente (che include il comportamento degli altri agenti) appare cambiare regole nel tempo, anche se le vere dinamiche fisiche sottostanti restano identiche.

Una seconda difficoltà è l'*assegnazione del merito* (*credit assignment*), per la quale quando più agenti contribuiscono a un solo risultato osservato, non è ovvio quale frazione di quel risultato attribuire a ciascuno. Tale problema però non si pone se ogni agente riceve una ricompensa calcolata esclusivamente sul proprio stato locale,

così come avviene in questo lavoro. Una versione attenuata resta comunque presente: lo stato locale di un agente, e quindi la sua stessa ricompensa, dipende in parte dalle scelte degli altri agenti, per cui l'effetto della propria azione e quello dell'azione altrui rimangono in parte intrecciati anche con una ricompensa puramente locale.

Due schemi di coordinamento ricorrono in letteratura (Li, 2018): l'*addestramento centralizzato con esecuzione decentralizzata* (CTDE), in cui gli agenti condividono informazione globale solo durante il training per poi decidere localmente, e l'*addestramento ed esecuzione entrambi decentralizzati* (DTDE), in cui l'informazione resta sempre e solo locale, anche durante l'addestramento. Una scelta indipendente da questa è se i diversi agenti condividano o meno gli stessi pesi (*parameter sharing*): condividerli riduce il numero di parametri totali e permette a un agente di beneficiare dell'esperienza raccolta da tutti gli altri, a costo di una minore capacità di specializzarsi su condizioni locali peculiari, a meno che il meccanismo non trovi un altro modo per differenziare il comportamento tra agenti pur condividendo i pesi.

Questo lavoro rappresenta un sistema multi-agente in cui ogni intersezione della rete stradale è un agente a sé, con un proprio stato s_i^t , una propria azione a_i^t (la fase semaforica scelta) e una propria ricompensa r_i^t , calcolati indipendentemente per ogni intersezione i (Capitolo 3). L'architettura è di tipo DTDE, con scambio di informazione locale tra intersezioni vicine (§2.4), e adotta il *parameter sharing*: tutte le intersezioni condividono la stessa rete Q , ma un meccanismo di meta-learning genera pesi diversi per nodi con caratteristiche locali diverse, così da attenuare proprio il costo della condivisione appena descritto. Va precisato che questa condivisione, insieme al fatto che l'aggiornamento del gradiente sia calcolato in un solo passo che aggrega la loss di tutte le intersezioni (Capitolo 3), non rende l'addestramento centralizzato nel senso di CTDE: ciò che definisce quello schema è l'uso, durante il solo training, di informazione non disponibile in esecuzione, mentre qui ogni termine della loss usa esclusivamente lo stato e la ricompensa locali della propria intersezione, esattamente gli stessi disponibili in esecuzione.

Quando la rete Q include un modulo ricorrente, condividere i pesi tra le intersezioni pone un'ulteriore difficoltà legata allo stato ricorrente campionato dal replay buffer, discussa in §2.3 dopo aver introdotto le reti ricorrenti.

2.3 Reti Neurali Ricorrenti

Una rete neurale *feedforward* calcola il proprio output come funzione del solo input corrente. Dunque ogni esempio è trattato in modo indipendente dai precedenti, e la rete non conserva alcuna memoria di cosa abbia visto in passato. Questa assunzione

fallisce ogni volta che i dati hanno una struttura sequenziale in cui l'ordine e la storia contano quanto il singolo valore osservato (testo, audio, serie temporali, o lo stato di un'intersezione stradale). Le *reti neurali ricorrenti* (RNN) affrontano il problema introducendo un ciclo: a ogni istante t , un nodo riceve in ingresso non solo il dato corrente x^t ma anche il proprio stato nascosto al passo precedente h^{t-1} ,

$$h^t = \sigma(W_{hx}x^t + W_{hh}h^{t-1} + b_h), \quad (2.4)$$

così che l'informazione rilevante di tutta la sequenza osservata fino a t possa, in linea di principio, persistere in h^t . Lo stesso insieme di pesi W_{hx}, W_{hh}, b_h viene riapplicato a ogni istante di tempo, invece di avere pesi diversi per ogni passo. "Srotolando" la rete lungo l'asse del tempo, cioè disegnando esplicitamente una copia dei nodi per ogni istante $t = 1, 2, \dots$ collegata alla successiva (Figura 2.6), ogni passo diventa uno strato di una rete feedforward molto profonda a pesi condivisi, addestrabile con la stessa backpropagation vista in precedenza applicata lungo la sequenza anziché lungo i layer: questo algoritmo prende il nome di *backpropagation through time* (BPTT).

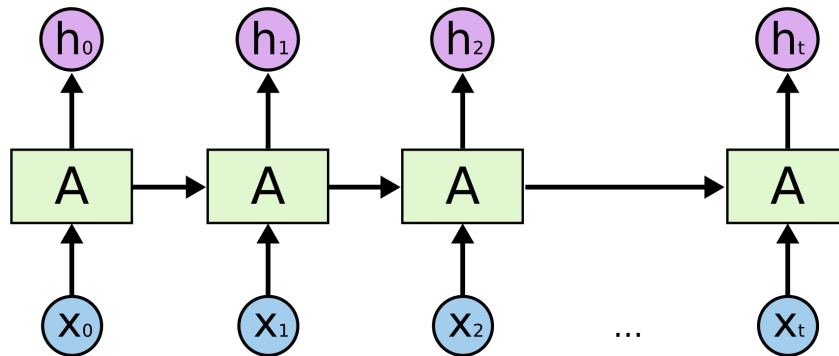


Figura 2.6: La rete ricorrente srotolata lungo il tempo: il modulo A rappresenta la funzione ricorrente, ad esempio l'Equazione 2.4, riapplicata con gli stessi pesi a ogni istante. Fonte: adattata da una discussione su Stack Exchange.³

Il problema del gradiente. Una rete ricorrente addestrata su sequenze lunghe soffre di un problema strutturale: il gradiente propagato all'indietro attraverso molti passi temporali tende ad annullarsi (*vanishing gradient*) o a divergere (*exploding gradient*) a seconda che i pesi ricorrenti abbiano modulo minore o maggiore di uno, poiché il gradiente si ottiene moltiplicando ripetutamente per lo stesso fattore a ogni passo srotolato, esattamente come una potenza c^n tende a zero o esplode al crescere di n a seconda che $|c|$ sia minore o maggiore di uno. Nella pratica, questo rende di

fatto impossibile apprendere dipendenze che coprono più di poche decine di passi con una RNN semplice.

Long Short-Term Memory (LSTM). L'LSTM (Hochreiter and Schmidhuber, 1997; Lipton et al., 2015) risolve il problema sostituendo il semplice nodo ricorrente con una *cella di memoria*: uno stato interno c^t collegato a se stesso da un percorso a peso fisso unitario (il *constant error carousel*), attraverso cui il gradiente può fluire per molti passi senza annullarsi né esplodere. L'accesso a questo stato è regolato da tre *gate*, unità sigmoidali il cui valore in $[0, 1]$ modula quanta informazione passi attraverso di esse (Figura 2.7).

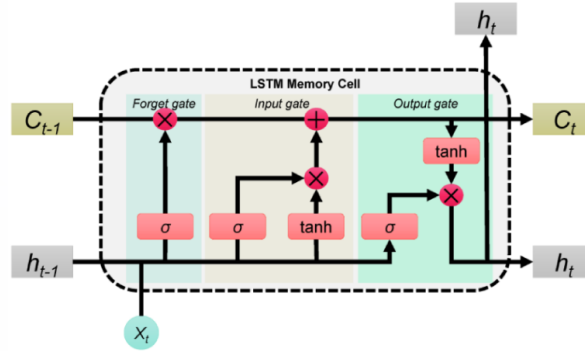


Figura 2.7: La cella di memoria dell'LSTM: il forget gate decide quanto dello stato precedente mantenere, l'input gate quanto del nuovo contenuto candidato incorporare, l'output gate quanto dello stato risultante esporre come output. Fonte: adattata da dida.do.⁴

A livello formale, con pesi $W_g \in \mathbb{R}^{2d_1 \times d_1}$ e bias $b_g \in \mathbb{R}^{d_1}$ condivisi da ogni nodo, i tre gate e il contenuto candidato $g \in \{\text{forget, input, output, cell}\}$ si calcolano come:

$$\begin{bmatrix} f^t \\ \iota^t \\ o^t \\ g^t \end{bmatrix} = \begin{bmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{bmatrix} \left(W_{\{f, \iota, o, g\}}^\top [x^t \parallel h^{t-1}] + b_{\{f, \iota, o, g\}} \right), \quad (2.5)$$

$$c^t = f^t \odot c^{t-1} + \iota^t \odot g^t,$$

$$h^t = o^t \odot \tanh(c^t);$$

il *forget gate* f decide quanto dello stato precedente dimenticare, l'*input gate* ι quanto del nuovo contenuto candidato g incorporare nello stato aggiornato, e l'*output gate* o quanto dello stato risultante esporre come output della cella. Il contenuto

candidato g non è esso stesso un gate, cioè un moltiplicatore in $[0, 1]$, ma il nuovo valore, calcolato con $\tanh$ e quindi in $[-1, 1]$, che i gate regolano. Un valore di forget gate vicino a 1 lascia passare l'informazione pregressa quasi inalterata anche dopo molti passi; uno vicino a 0 la cancella, permettendo alla rete di "dimenticare" deliberatamente informazione non più rilevante.

Gated Recurrent Unit (GRU). Una variante più leggera dell'LSTM, la **GRU** (Cho et al., 2014), fonde il *forget gate* e l'*input gate* in un unico *update gate* e rinuncia allo stato di cella c^t separato, esponendo un solo stato nascosto h^t sia come memoria interna sia come output. Si hanno così meno parametri da addestrare e un grafo computazionale più semplice, a fronte di una capacità di rappresentazione leggermente inferiore rispetto ai quattro gate distinti dell'LSTM. Le due varianti raggiungono nella pratica prestazioni simili su molti compiti, e la scelta tra loro è spesso guidata più dalla convenzione di un'architettura di riferimento che da un vantaggio netto dell'una sull'altra: questo lavoro adotta l'LSTM per coerenza con il modello di riferimento (Wang et al., 2022).

Replay ricorrente e disallineamento dello stato. Quando la rete Q include un modulo ricorrente come l'LSTM appena descritto, campionare dal replay buffer (§2.2) transizioni isolate anziché intere sequenze pone un problema specifico: lo stato ricorrente (h, c) usato durante l'addestramento non coincide necessariamente con quello osservato durante la raccolta dati. Per una transizione campionata isolatamente, l'unica scelta praticabile è inizializzare (h, c) a zero, un disallineamento rispetto allo stato che la rete aveva effettivamente accumulato in quel punto della sequenza originale, un problema noto come *hidden state staleness*. Seguendo Kaptrowski et al. (2019) (R2D2), il campionamento si esegue allora su intere sequenze di lunghezza L , dedicando i primi passi della sequenza (fase di *burn-in*) esclusivamente alla ricostruzione di uno stato ricorrente plausibile, senza calcolare gradiente, per poi eseguire la BPTT sui passi restanti.

2.4 Grafi e Graph Neural Network

2.4.1 Cos'è un grafo

Un *grafo* $G = (V, E)$ è una struttura matematica composta da un insieme di *nodi* V e un insieme di *archi* E , dove ogni arco collega una coppia di nodi. È lo strumento standard per rappresentare entità e le relazioni tra loro, quando quelle relazioni non hanno una struttura regolare o sequenziale. Ne sono esempi una rete di amicizie

tra persone, le pagine web collegate da link, le molecole i cui atomi sono legati da legami chimici, e, in questo lavoro, una rete stradale, dove i nodi sono le intersezioni e gli archi le strade che le collegano direttamente.

Un arco può essere *diretto* (se indica un verso) o *non diretto* (se indica una relazione simmetrica, Figura 2.8); può inoltre avere un *peso* associato, un numero che quantifica l'intensità o il costo della relazione.

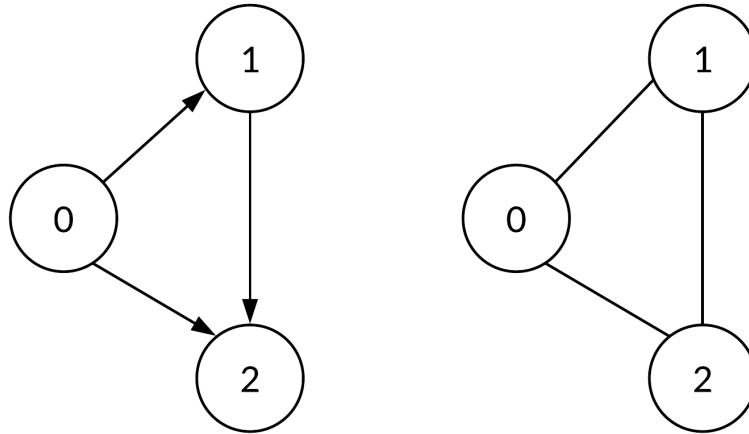


Figura 2.8: Esempi di grafo diretto (sinistra) e non diretto (destra).

L'insieme dei nodi collegati a un nodo i tramite un arco si chiama *vicinato* di i , indicato $\mathcal{N}(i)$; il numero di archi incidenti su i è il suo *grado*. Una sequenza di nodi collegati uno al successivo da un arco è un *cammino*; la lunghezza del cammino più breve tra due nodi qualunque è la loro *distanza* sul grafo, misurata in numero di archi. Un grafo con N nodi si può rappresentare in forma matriciale con la *matrice di adiacenza* $A \in \{0, 1\}^{N \times N}$, dove $A_{iu} = 1$ se e solo se esiste un arco tra i e u (o il peso dell'arco, se il grafo è pesato, invece del semplice 1). È spesso utile aggiungere a ogni nodo un *self-loop*, un arco da un nodo verso se stesso ($A_{ii} = 1$), cosicché un nodo sia incluso nel proprio stesso vicinato: la notazione $\hat{A} = A + I$ indica questa aggiunta e ricorre nelle equazioni seguenti.

Un piccolo esempio: la matrice di adiacenza senza pesi del grafo non diretto in Figura 2.8 è data da

$$A = \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix}.$$

Con l'aggiunta dei self-loop, $\hat{A} = A + I$ ha inoltre un 1 su ogni elemento della diagonale.

Nella rete stradale di questo lavoro, ogni nodo i del grafo porta con sé un vettore di caratteristiche (o *embedding*) che ne descrive lo stato corrente, ad esempio quanti veicoli attendono su ciascuna corsia. Il problema che le *graph neural network* risolvono è come far sì che l’embedding di un nodo tenga conto anche di ciò che accade nei nodi a lui collegati, non solo del proprio stato isolato.

2.4.2 Message passing

Le *Graph Neural Network* (GNN) affrontano questo problema con un principio comune, il *message passing* (Wu et al., 2021; Bacciu et al., 2020). A ogni iterazione (o *layer*) l , ogni nodo i con rappresentazione corrente $h_i^{(l)}$ esegue due operazioni in successione: prima un passo di *aggregazione*, che combina i messaggi ricevuti da ciascun vicino $u \in \mathcal{N}(i)$ in un unico vettore di messaggio m_i ,

$$m_i = \text{AGGREGATE}(\{h_u^{(l)} : u \in \mathcal{N}(i)\}),$$

poi un passo di *aggiornamento*, che combina il messaggio aggregato con la rappresentazione precedente dello stesso nodo per produrre quella nuova,

$$h_i^{(l+1)} = \text{UPDATE}(h_i^{(l)}, m_i).$$

Tutte le GNN discusse in questa tesi, incluse le formulazioni di seguito, sono istanze di questo stesso schema generale, mostrato in Figura 2.9, che differiscono solo nella scelta concreta di AGGREGATE e UPDATE. Nella GCN, AGGREGATE è una media pesata dalla sola topologia; nella GAT, un peso appreso in base al contenuto dei due nodi coinvolti. Impilando più layer, l’informazione può viaggiare oltre il solo vicinato immediato: dopo k layer, la rappresentazione di un nodo dipende (indirettamente) da tutti i nodi raggiungibili entro k archi di distanza, poiché il messaggio ricevuto da un vicino al layer l incorpora già, ricorsivamente, i messaggi che quel vicino aveva ricevuto dai propri stessi vicini al layer $l - 1$.

Graph Convolutional Network (GCN). Kipf and Welling (2017) propongono la formulazione più basilare di message passing tra quelle qui discusse. La rappresentazione di ciascun nodo si aggiorna mediando quelle dei vicini con un peso fisso, determinato unicamente dalla topologia locale (quanti vicini ha un nodo), non dal contenuto delle rappresentazioni stesse. Indicando con $\hat{A} = A + I$ la matrice di adiacenza con self-loop e con $\hat{D}$ la matrice diagonale dei gradi corrispondente ($\hat{D}_{ii} = \sum_u \hat{A}_{iu}$), la regola di propagazione di un singolo layer si scrive, in forma

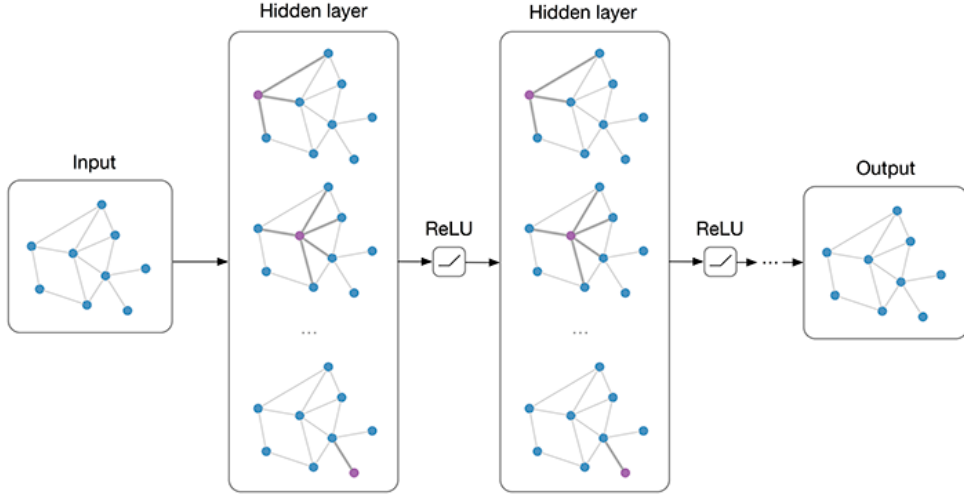


Figura 2.9: Lo schema generale di message passing impilato su più layer: a ogni layer ciascun nodo aggrega le rappresentazioni dei propri vicini e aggiorna la propria, prima di applicare la non linearità σ e passare al layer successivo. Fonte: adattata da Towards AI.⁵

matriciale:

$$H^{(l+1)} = \sigma\left(\hat{D}^{-1/2} \hat{A} \hat{D}^{-1/2} H^{(l)} W^{(l)}\right) \quad (2.6)$$

dove $H^{(l)} \in \mathbb{R}^{N \times d_l}$ raccoglie le rappresentazioni di tutti gli N nodi del grafo al layer l , $W^{(l)}$ è la matrice di pesi condivisa da ogni nodo e σ una non linearità puntuale. Per un singolo nodo i , la normalizzazione simmetrica $\hat{D}^{-1/2} \hat{A} \hat{D}^{-1/2}$ equivale a pesare il contributo di ogni vicino u con $1/\sqrt{\hat{d}_i \hat{d}_u}$:

$$h_i^{(l+1)} = \sigma\left(\sum_{u \in \mathcal{N}(i) \cup \{i\}} \frac{1}{\sqrt{\hat{d}_i \hat{d}_u}} W^{(l)} h_u^{(l)}\right) \quad (2.7)$$

un peso identico per ogni arco del vicinato di i , funzione solo del grado dei due nodi coinvolti e mai del loro contenuto. È, per costruzione, il termine di paragone più naturale rispetto a cui misurare cosa aggiunge un meccanismo di attenzione appreso come quello discusso di seguito.

Graph Attention Network (GAT). Veličković et al. (2018) sostituiscono il peso fisso della GCN con un coefficiente di attenzione appreso, specifico per ciascuna coppia di nodi collegati: il contributo di un vicino non dipende più soltanto dalla topologia, ma anche dal suo contenuto. Dato un nodo i con vicinato $\mathcal{N}(i)$, embedding di input h_i , matrice di proiezione condivisa W e vettore di attenzione $\vec{a}$:

$$e_{iu} = \text{LeakyReLU}\left(\vec{a}^\top [W h_i \parallel W h_u]\right), \quad \alpha_{iu} = \frac{\exp(e_{iu})}{\sum_{u' \in \mathcal{N}(i)} \exp(e_{iu'})} \quad (2.8)$$

$$h'_i = \sigma \left(\sum_{u \in \mathcal{N}(i)} \alpha_{iu} W h_u \right) \quad (2.9)$$

dove $\parallel$ denota la concatenazione: e_{iu} è un punteggio di compatibilità grezzo tra i e u , che il *softmax* sul vicinato trasforma in un coefficiente $\alpha_{iu} \in [0, 1]$ che somma a uno su tutti i vicini di i , esattamente come una distribuzione di probabilità, come illustrato in Figura 2.10.

Graph Attention Networks

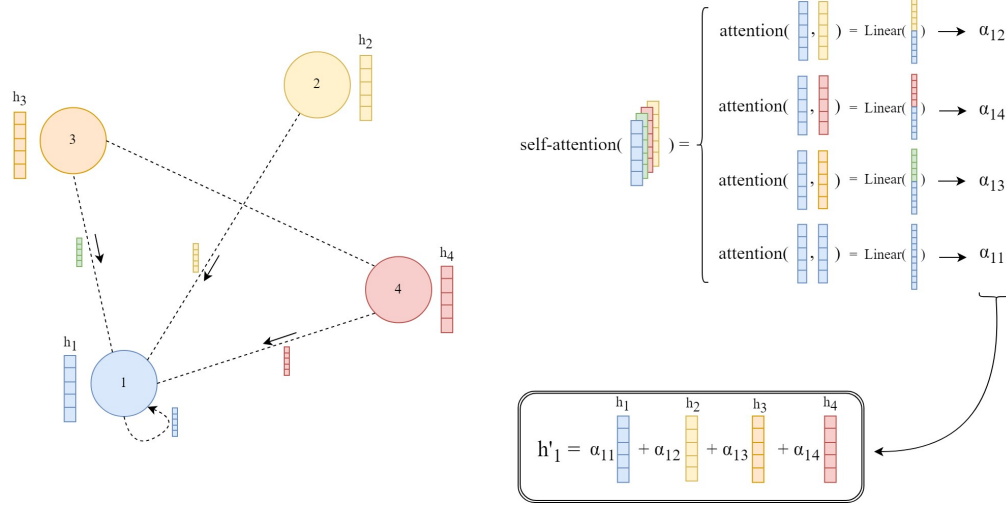


Figura 2.10: Il calcolo dell'attenzione per il nodo 1 rispetto al proprio vicinato $\{1, 2, 3, 4\}$ (con self-loop incluso): ogni coppia (h_1, h_u) produce un coefficiente α_{1u} , usato poi per combinare linearmente gli embedding dei vicini nella nuova rappresentazione h'_1 . La figura, semplificata a scopo illustrativo, accorpa in un unico passo "Linear" il calcolo di compatibilità e la normalizzazione softmax descritti nel testo, e omette la proiezione condivisa W e la non linearità σ . Fonte: adattata da Data Science at Home.⁶

Con K teste di attenzione indipendenti (*multi-head attention*), ciascuna con i propri pesi W e $\vec{a}$, gli output si concatenano nei layer intermedi e si mediano nell'ultimo, secondo lo schema introdotto per l'attenzione da Vaswani et al. (2017). In questo modo teste diverse possono imparare a specializzarsi su aspetti diversi della relazione tra due nodi. A differenza della GCN, il coefficiente α_{iu} consente a un nodo di ponderare diversamente i propri vicini in base al loro contenuto, non solo alla loro posizione nel grafo: una proprietà rilevante per un nodo i cui vicini abbiano importanza marcatamente diversa, come nel caso di un incrocio stradale in cui passano un'arteria principale e una strada secondaria. La funzione di compatibilità additiva appena descritta, per quanto la più nota, non è l'unica possibile: il Capitolo 3 (Meta-GAT) ne impiega una variante strutturalmente diversa, con Query, Key e Value esplicite e distinte al posto della proiezione W condivisa, e un prodotto

scalare al posto della concatenazione.

2.4.3 Invarianza ed equivarianza per permutazione

Sia l'aggregazione della GCN sia quella della GAT condividono due proprietà distinte, che vale la pena separare.

La prima è ciò che permette a queste reti di funzionare su grafi di dimensione diversa senza dover riaddestrare nulla: i pesi W sono condivisi da ogni nodo, e l'operatore AGGREGATE (una somma pesata, sia essa uniforme come nella GCN o per attenzione come nella GAT) accetta un numero qualsiasi di vicini. Il numero di parametri del layer non dipende quindi dal numero di nodi N del grafo, una condizione necessaria perché lo stesso modello, addestrato su un grafo di una certa dimensione, possa essere applicato a un grafo di dimensione diversa.

La seconda è l'*invarianza per permutazione* dell'operatore di aggregazione: una somma non dipende dall'ordine in cui i suoi addendi vengono elencati, quindi il messaggio aggregato m_i di un nodo i non dipende dall'ordine in cui i vicini di i vengono numerati o elencati, ma solo da quali essi siano. È ciò che distingue strutturalmente una GNN da una rete feedforward applicata a un vettore di caratteristiche di lunghezza fissa. Una MLP a cui si desse in input, in un qualche ordine arbitrario, le caratteristiche di ciascun nodo del grafo concatenate assegnerebbe implicitamente un significato specifico a ciascuna posizione del vettore (il primo blocco di ingressi, il secondo, ...) e produrrebbe quindi un output diverso se semplicemente si rinumerassero i nodi, pur descrivendo esattamente la stessa rete stradale. Wei et al. (2019) osservano esattamente questo limite per gli schemi di coordinamento basati su una lista ordinata di vicini.

Applicando l'invarianza per permutazione a ogni nodo del grafo simultaneamente, l'intero layer di message passing risulta *equivariante per permutazione*: rinumerare i nodi in ingresso rinumererà allo stesso modo le rappresentazioni prodotte in uscita, invece di produrre un risultato arbitrariamente diverso (Bronstein et al., 2021). È questa equivarianza, insieme all'indipendenza del numero di parametri da N , a rendere possibile la generalizzazione a topologie di rete mai viste che il Capitolo 4 conduce su MetaSTGAT e sulle sue varianti.

2.4.4 Reti spatio-temporali su grafo

Molti grafi non sono statici: la topologia resta fissa, ma le caratteristiche osservate a ciascun nodo evolvono nel tempo, e le due dimensioni, spaziale e temporale, risultano intrecciate, poiché il valore osservato a un nodo in un dato istante è correlato sia

con i valori passati allo stesso nodo sia con quelli ai nodi limitrofi. La famiglia di modelli che combina un layer di propagazione sul grafo (GCN o GAT, appena visti) con una componente ricorrente (LSTM, §2.3) sull'asse temporale prende il nome di *spatiotemporal graph neural network* (STGNN), un termine ormai consolidato nella letteratura sul forecasting di serie temporali multivariate correlate (Cini et al., 2025). MetaSTGAT, descritto nel dettaglio nel Capitolo 3, costituisce un esempio di questa famiglia applicata al controllo semaforico anziché alla previsione: un layer di *graph attention* cattura la dipendenza spaziale tra intersezioni limitrofe, un layer ricorrente (LSTM) ne cattura l'evoluzione temporale, e i due vengono addestrati congiuntamente end-to-end.

2.4.5 Pesì generati da meta-learning

Le strutture neurali introdotte, GCN e GAT, nella loro formulazione originale, condividono gli stessi pesi su tutti i nodi del grafo: un'assunzione ragionevole quando i nodi sono sufficientemente omogenei, meno quando, come in una rete stradale reale, la topologia locale di ciascun nodo (numero di strade entranti, presenza di un'arteria, posizione periferica o centrale) può differire sensibilmente da nodo a nodo. Un *hypernetwork* (Ha et al., 2016) affronta il problema generando i pesi di un layer non come parametri fissi appresi direttamente, bensì come output di una seconda rete, condizionata sulle caratteristiche locali del nodo: è di fatto una via di mezzo tra pesi completamente condivisi e pesi completamente indipendenti per ogni nodo, che riduce il costo del parameter sharing senza rinunciare del tutto ai suoi benefici. Nel contesto di questo lavoro, un *hypernetwork* condizionato su caratteristiche locali dell'intersezione è esattamente il meccanismo che la letteratura sul controllo semaforico chiama *meta-learning* quando applicato a questo compito: i due termini descrivono lo stesso principio, il secondo è quello più diffuso nei lavori discussi in §2.5, dove si colloca esplicitamente MetaSTGAT.

2.4.6 Propagazione a lungo raggio

Sia la GCN sia la GAT propagano informazione a un solo passo di distanza (un arco nel grafo) per layer: raggiungere un vicino a distanza k richiede impilarne altrettanti, con un costo di parametri crescente e un rischio noto di *over-smoothing* (Li et al., 2018), per il quale le rappresentazioni dei nodi divengono troppo omogenee al crescere della profondità (ogni nodo finisce per "vedere" praticamente tutto il grafo, perdendo la capacità di distinguersi dai propri vicini). Meccanismi più recenti, tra cui SONAR (Trenta et al., 2025), affrontano il problema della propagazione a lungo

raggio secondo un principio diverso, ispirato ai sistemi dinamici oscillatori piuttosto che alla concatenazione sequenziale di layer. Il sistema si comporta come una rete di masse collegate da molle e smorzatori, una massa per nodo del grafo. Raccolto lo stato di tutti gli N nodi al tempo continuo t in $X(t) \in \mathbb{R}^{N \times d}$, la sua evoluzione è governata da un'equazione del second'ordine, un'accelerazione e non una semplice velocità:

$$\ddot{X}(t) = -\mathcal{L}^a X(t) W - D(X(t)) \odot \dot{X}(t) + F(X(t)) \quad (2.10)$$

Tre termini, tre ruoli distinti. $\mathcal{L}^a$ è un Laplaciano di grafo pesato da una conduttanza $a_{ui} \geq 0$ per arco, che tira lo stato di un nodo verso quello dei suoi vicini $u \in \mathcal{N}(i)$, come una molla. $D(\cdot) \geq 0$ è un termine dissipativo che agisce sulla velocità $\dot{X}(t)$, frenando il movimento come un attrito. $F(\cdot)$ è una forzante esterna, che inietta energia dall'esterno del sistema. Riscritta come sistema del prim'ordine, con velocità ausiliaria $V = \dot{X}$, e discretizzata con passo h su L iterazioni, l'aggiornamento a ogni iterazione diventa:

$$\begin{aligned} (\mathcal{L}^a X)_i &= \sum_{u \in \mathcal{N}(i)} a_{ui} (X_i - X_u), \\ V &\leftarrow V - h(\mathcal{L}^a X + D(X) \odot V - F(X)), \\ X &\leftarrow X + h \tanh(V) \end{aligned} \quad (2.11)$$

Le L iterazioni condividono sempre gli stessi pesi a_{ui} , $D(\cdot)$ e $F(\cdot)$, per cui, a differenza di uno stack di L layer indipendenti come GCN o GAT, il costo di parametri di questo meccanismo resta costante al crescere di L , un vantaggio strutturale dichiarato dagli stessi autori.

2.5 Traffic Signal Control come problema

2.5.1 Terminologia e metriche

Il *traffic signal control* (TSC), o controllo semaforico, decide periodicamente quali flussi veicolari autorizzare a un incrocio. Il vocabolario di base, comune a questo capitolo e ai Capitoli 3 e 4, è il seguente.

Ogni via che affluisce a un incrocio è divisa in *corsie*, ciascuna tipicamente dedicata a una o più svolte (diritto, sinistra, destra). Un *movimento* è una coppia ordinata di corsie, una in ingresso e una in uscita, che un veicolo può percorrere attraversando l'incrocio; una *fase* semaforica autorizza simultaneamente un sottoinsieme di movimenti compatibili tra loro, cioè le cui traiettorie non si intersecano. Il

controllore osserva lo stato dell'incrocio e sceglie una fase a ogni *step di decisione*, un intervallo di tempo fissato che si ripete per l'intera durata della simulazione.

Valutare un controllore richiede di misurare cosa succede ai veicoli. Il *tempo di percorrenza* (*travel time*) di un veicolo è il tempo trascorso tra il suo ingresso e la sua uscita dalla rete; l'*attesa* di un veicolo, in un dato istante, è il tempo per cui si trova fermo o comunque non ancora oltre l'incrocio a cui si sta avvicinando. Il *throughput* di un incrocio o di una rete è invece il numero di veicoli che riescono ad attraversarli nell'unità di tempo, una nozione teorica distinta dalla percentuale di veicoli che completano effettivamente il proprio percorso in una simulazione di durata finita, misurata nel Capitolo 4. Le definizioni precise di queste grandezze, e le metriche che questo lavoro ne ricava, sono rimandate ai Capitoli 3 e 4, dove vengono usate per la prima volta in forma quantitativa.

2.5.2 Due generazioni di controllo

Il TSC si articola in due approcci, in ordine crescente di complessità e, storicamente, di introduzione (Shams et al., 2018).

Controllo a tempi fissi. Un piano semaforico ciclico ripete all'infinito la stessa sequenza di fasi, ciascuna di durata prestabilita in base a stime di traffico medie e mai più rivista in tempo reale; nella sua forma più diffusa nei sistemi reali, la ripartizione del verde tra le fasi e la durata complessiva del ciclo possono variare per fascia oraria, ma restano fissate a tavolino e non rispondono al traffico osservato istante per istante. FIXED-TIME, la baseline di questo lavoro (Capitolo 3), ne è la forma più semplice: cicla le fasi in ordine fisso, cambiando fase a ogni step di decisione, senza alcun parametro da configurare oltre all'ordine del ciclo. È estremamente semplice da implementare ed è tuttora la spina dorsale di gran parte dei sistemi semaforici reali: negli Stati Uniti, ad esempio, una larga proporzione delle intersezioni semaforizzate resta priva di sensori di rilevamento ed è tuttora controllata a tempi fissi, proprio per gli alti costi di installazione e manutenzione dei rilevatori (Wang et al., 2024). Non richiede infatti sensori stradali in tempo reale né potenza computazionale per decidere. I suoi svantaggi principali sono la rigidità e l'incapacità di adattarsi alle fluttuazioni improvvise del traffico, caratteristiche che possono risultare critiche in caso di eventi imprevisti.

Controllo reattivo. Una regola locale osserva lo stato corrente dell'intersezione (tipicamente il numero di veicoli in coda per corsia) e sceglie l'azione che ottimizza un criterio istantaneo, senza affidarsi a un piano prestabilito. Questa regola può essere scritta a mano, controllo *euristico*, oppure appresa dall'esperienza tramite reinforcement learning, controllo *appreso*: quest'ultimo è il paradigma adottato da

questo lavoro (§2.2). All'interno del controllo euristico, [Shams et al. \(2018\)](#) distinguono ulteriormente il grado di adattività: il controllo *attuato* più semplice si limita a prolungare o abbreviare le fasi di un piano di base ancora sostanzialmente predefinito (estendere una fase finché un sensore rileva veicoli sulla corsia servita, ad esempio), mentre un controllo pienamente *adattivo* ricalcola la struttura stessa del piano, non solo la sua durata, in risposta al traffico osservato. Piattaforme come SCATS e SCOOT si collocano in questa seconda fascia, coordinando intere aree urbane aggiustando periodicamente i piani semaforici sulla base del traffico osservato, pur restando ancorate a una struttura di piano pre-esistente ([Wei et al., 2020](#)).

MAX-PRESSURE ([Varaiya, 2013](#)) rappresenta invece un sistema di controllo reattivo che, sotto l'ipotesi idealizzata di corsie a capacità infinita, stabilizza le code della rete per qualunque livello di domanda che la rete stessa sia in grado di sostenere, senza alcuna necessità di training. Questo risultato, pur non potendo essere traslato integralmente in scenari reali dove non è rara la possibilità di una completa saturazione di una via, si fonda solamente su dati raccolti a ogni istante decisionale nelle corsie adiacenti all'incrocio e su un principio di base: selezionare a ogni istante la fase a pressione massima. Data una fase semaforica ϕ , la sua pressione è definita come la somma, su tutti i movimenti abilitati da ϕ , della differenza tra il numero di veicoli in coda sulla corsia di ingresso e quelli sulla corsia di uscita, contati sull'intera corsia:

$$P(\phi) = \sum_{(\ell_{\text{in}}, \ell_{\text{out}}) \in \text{movimenti}(\phi)} \left(n(\ell_{\text{in}}) - n(\ell_{\text{out}}) \right), \quad (2.12)$$

dove $n(\ell)$ indica il numero di veicoli sulla corsia ℓ . Questa è la forma semplificata della pressione usata in questo lavoro e in gran parte della letteratura successiva, che assegna un solo movimento a ogni coppia di corsie e non richiede quindi rapporti di svolta (la frazione di veicoli di una corsia diretta verso ciascuna corsia di uscita raggiungibile), necessari solo quando una stessa corsia in ingresso alimenta più movimenti distinti. A ogni step di decisione, l'algoritmo calcola la pressione di tutte le fasi e seleziona quella a pressione massima, $\phi^* = \operatorname{argmax}_{\phi} P(\phi)$, senza alcun piano né ciclo predefinito, né alcun vincolo sulle azioni consecutive: la stessa fase può in linea di principio essere selezionata un numero indefinito di volte. Uno svantaggio è la necessità di sensoristica capace di monitorare le corsie per la loro interezza. Il problema principale, tuttavia, è l'assenza di garanzie di equità: in presenza di un forte afflusso di traffico su un certo insieme di corsie, la pressione maggiore può trovarsi sempre sulle fasi che coinvolgono quelle vie, con conseguenti tempi di attesa eccessivi per i veicoli sulle vie meno trafficate, un aspetto di equità che questo lavoro esamina esplicitamente.

Fixed-Time e Max-Pressure sono le due baseline sperimentali classiche a cui questo lavoro confronta i propri modelli (Capitolo 4), usate qui esattamente nella forma appena descritta. Pur fondandosi su paradigmi opposti, statico l'uno e adattivo l'altro, entrambe non richiedono alcuna fase di addestramento, il che le rende candidati naturali per il confronto con i modelli di deep reinforcement learning discussi nel resto di questo lavoro.

2.5.3 Simulazione

Valutare una politica di controllo semaforico su una rete reale non è praticabile durante lo sviluppo, in quanto cambiare i tempi di un semaforo cittadino migliaia di volte per tentativi, come richiederebbe l'addestramento di un agente RL, non è ovviamente proponibile. Serve quindi un simulatore che riproduca la dinamica del traffico con un dettaglio sufficiente, pur mantenendo un costo computazionale accettabile per l'addestramento di un centinaio di episodi (Capitolo 3).

Un simulatore di traffico microscopico rappresenta ogni veicolo individualmente, con una propria posizione, velocità e traiettoria, aggiornate a ogni istante da un modello che regola la velocità di un veicolo in base al comportamento di quello che lo precede mantenendo una distanza di sicurezza. È il livello di dettaglio presente in questo lavoro, sufficientemente fedele da catturare l'effetto di una singola decisione semaforica su una coda specifica.

CITYFLOW (Zhang et al., 2019) è il simulatore microscopico scelto in questo lavoro. Rispetto ad alternative più diffuse come SUMO (Lopez et al., 2018), CITYFLOW semplifica ulteriormente il modello che regola la dinamica dei veicoli per privilegiare la velocità di simulazione, un compromesso adatto quando l'obiettivo primario è addestrare un agente su un numero elevato di episodi piuttosto che riprodurre con precisione millimetrica la dinamica di un incrocio specifico.

2.5.4 Controllo semaforico appreso: stato dell'arte

Il controllo appreso sostituisce la regola scritta a mano del controllo reattivo con una politica addestrata sull'esperienza, ma non elimina il problema del coordinamento tra intersezioni: lo sposta, dalla progettazione manuale di un piano coordinato alla scelta di come far condividere informazione agli agenti durante l'apprendimento. La letteratura sul reinforcement learning applicato al TSC si può raggruppare, non esaustivamente, per come affronta questa scelta.

RL su singola intersezione. I primi lavori applicano il RL a un solo incrocio alla volta, spesso come banco di prova per verificare che un agente possa competere con le regole reattive prima ancora di affrontare il problema del coordinamento. AttendLight (Oroojlooy et al., 2020) ne è un esempio recente: addestra un singolo modello su un insieme eterogeneo di intersezioni indipendenti, e ne verifica le prestazioni su intersezioni mai viste, ma dichiara esplicitamente che l'estensione a una rete di più intersezioni coordinate resta lavoro futuro.

Multi-agente indipendente. Un secondo filone affronta direttamente una rete di intersezioni, ma senza alcuno scambio esplicito di informazione tra agenti vicini: ciascuna intersezione apprende e decide sulla base del proprio solo stato locale, e un'eventuale coordinazione emerge indirettamente dal fatto che le azioni di un agente alterano il traffico osservato dagli altri. El-Tantawy et al. (2013) propongono un primo schema di questo tipo su scala urbana reale, mentre Chu et al. (2019) confrontano esplicitamente varianti indipendenti e cooperative dello stesso algoritmo su reti di grandi dimensioni. Il vantaggio di questo filone è la scalabilità, ogni agente resta un problema di RL a singolo agente; lo svantaggio è che nessuna azione tiene conto esplicitamente dello stato dei vicini nel momento della decisione.

Coordinato tramite comunicazione o reti a grafo. Un terzo filone fa scambiare esplicitamente informazione tra intersezioni vicine durante la decisione stessa, non solo indirettamente attraverso l'ambiente. Zhang et al. (2025) modellano esplicitamente le dipendenze tra agenti vicini in questo schema. CoLight (Wei et al., 2019) usa un meccanismo di *graph attention* tra intersezioni adiacenti per decidere quanto pesare il contributo di ciascun vicino, con pesi condivisi da tutti i nodi della rete. MetaSTGAT (Wang et al., 2022), il modello di riferimento di questo lavoro, condivide con CoLight il meccanismo di comunicazione via *graph attention*, ma vi combina un modulo di meta-learning (§2.4): invece di condividere gli stessi pesi tra tutte le intersezioni della rete, questo modulo genera dinamicamente i pesi di uno strato di attenzione e di uno strato ricorrente a partire da caratteristiche locali dell'intersezione, così che nodi diversi possano essere serviti da trasformazioni diverse pur restando un unico modello addestrato end-to-end. Gli autori originali riportano una riduzione del travel time medio rispetto a CoLight, un metodo di riferimento tra i più citati per il TSC.

Equità. Indipendentemente dallo schema di coordinamento adottato, alcuni lavori affrontano esplicitamente il problema dell'equità tra direzioni di traffico, penalizzando le fasi responsabili di attese eccessive invece di ottimizzare solo una metrica

aggregata di rete. FELight (Xiao et al., 2025) ne è un esempio: introduce una metrica di equità basata sui tempi di attesa dei veicoli e penalizza le fasi già identificate come responsabili di un ritardo superiore a una soglia prestabilita. Un tratto comune a questa famiglia di approcci è intervenire in modo selettivo, solo quando un’attesa supera una soglia o su una fase individuata in anticipo come problematica, invece che in ogni istante decisionale e per ogni intersezione.

Generalizzazione. La prassi più diffusa nel controllo semaforico appreso è addestrare e valutare un modello sulla stessa rete stradale, al più ripetendo l’intero esperimento, addestramento incluso, su una rete diversa, non trasferendo un singolo modello già addestrato a una rete che non ha mai visto (Wei et al., 2020). CoLight (Wei et al., 2019), per esempio, conduce cinque esperimenti indipendenti, due su reti sintetiche e tre su reti reali, ciascuno con il proprio addestramento; MetaSTGAT (Wang et al., 2022) segue lo stesso schema su una griglia sintetica e due dataset reali. Quando in questi lavori compare il termine *generalization*, indica quasi sempre la tenuta a diverse intensità di traffico sulla stessa rete, non a una topologia strutturalmente diversa (Wang et al., 2026; Sun et al., 2026), come reso esplicito da Zhang et al. nella motivazione di GeneraLight (Zhang et al., 2020). Le eccezioni restano circoscritte a un’unica dimensione di variazione: AttendLight generalizza tra intersezioni singole ma non a una rete coordinata, come già notato sopra; Advanced-XLight (Zhang et al., 2022) misura un rapporto di trasferimento tra un modello addestrato su un dataset reale e valutato direttamente su un altro, senza ulteriore addestramento, ma sempre tra reti di scala paragonabile; lo stesso GeneraLight generalizza al variare del flusso di traffico, non della topologia della rete.

2.6 Sintesi

Il controllo semaforico su rete è un problema di coordinamento distribuito che le regole scritte a mano affrontano con euristiche fisse, poco adattabili a condizioni non previste in fase di progettazione, e che il controllo appreso affronta invece con una politica addestrata sull’esperienza simulata. All’interno del controllo appreso, la letteratura ha esplorato schemi di coordinamento via via più espliciti, dalla singola intersezione al multi-agente indipendente fino alla comunicazione tramite reti neurali a grafo, un asse su cui MetaSTGAT si colloca con un meccanismo di meta-learning che permette a intersezioni di topologia diversa di condividere comunque un unico modello.

Su due assi, però, la letteratura resta parziale. I meccanismi di equità esistenti intervengono in modo selettivo, solo oltre una soglia o su fasi individuate in anticipo, invece che come obiettivo esplicito valutato a ogni istante e per ogni intersezione. La generalizzazione, quando misurata, riguarda quasi sempre l'intensità del traffico sulla rete di addestramento, raramente una topologia, una spaziatura stradale o un profilo di traffico strutturalmente diversi da quelli visti in addestramento, e le eccezioni che esistono restano confinate a un'unica dimensione di variazione alla volta. I capitoli seguenti descrivono come questo lavoro affronta congiuntamente questi due assi, usando MetaSTGAT come architettura di partenza.

Capitolo 3

Metodologia

Questo capitolo definisce direttamente il modello di questo lavoro e la procedura con cui viene addestrato. La Sezione 3.1 formalizza il problema come Dec-POMDP, istanziando stato, azione, transizione, fattore di sconto, ricompensa e topologia di rete. La Sezione 3.2 presenta l'architettura alla base del modello, MetaSTGAT (Wang et al., 2022), chiudendo con i meccanismi spaziali alternativi confrontati nel Capitolo 4. La Sezione 3.3 descrive la procedura di addestramento adottata. La Sezione 3.4 raccoglie in forma tabellare ogni differenza tra questo lavoro e il riferimento originale, definendo le configurazioni MetaSTGAT Pro e quella che ne replica fedelmente la procedura. Il capitolo si chiude con l'ambiente di simulazione costruito su CITYFLOW (Sezione 3.5).

3.1 Formalizzazione del problema

La Sezione 2.2 introduce l'MDP come la tupla $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$ e la sua estensione al caso multi-agente, il Dec-POMDP a osservabilità locale: ogni intersezione i osserva solo il proprio stato, ma le sue azioni influenzano, con un ritardo proporzionale alla distanza spaziale, lo stato delle intersezioni adiacenti. Questa sezione istanzia ciascuna componente di quella tupla per una rete di N intersezioni, secondo la formulazione di problema della Sezione 2.5. Ad ogni istante t , l'intersezione i osserva uno stato $s_i^t \in \mathcal{S}$, sceglie un'azione $a_i^t \in \mathcal{A}$ e riceve una ricompensa r_i^t , mentre l'informazione viene propagata secondo la topologia del grafo stradale. Rispetto alla formulazione del riferimento originale, cambiano stato, azione e ricompensa, discussi nel dettaglio in Sezione 3.4; transizione, fattore di sconto e topologia di rete restano identici.

Stato ($\mathcal{S}$). Ogni intersezione ha 4 strade in ingresso, una per direzione cardinale, con 3 corsie per senso di marcia, per un totale di 12 corsie in ingresso e altrettante in uscita. A questa configurazione si aggiungono 8 fasi semaforiche selezionabili.

Lo stato è la concatenazione di tre componenti:

$$s_i^t = [n_i^t \parallel w_i^t \parallel p_i^t] \in \mathbb{R}^{d_0} \quad (3.1)$$

dove $n_i^t \in [0, 1]^{12}$ è il numero di veicoli per corsia in ingresso entro il raggio di visibilità d_{vis} (normalizzato dividendo per 30 e poi limitato tra 0 e 1), $w_i^t \in [0, 1]^{12}$ è il vettore dei tempi di attesa dei veicoli da più tempo in corsia ($\tanh(\frac{\text{attesa}}{300})$) e $p_i^t \in \{0, 1\}^8$ è la fase corrente in codifica one-hot, per un totale $d_0 = 32$. Il riferimento originale adotta invece uno stato composto solo da n_i^t e p_i^t (dimensione totale $d_0 = 20$), la stessa dimensione usata anche dalla configurazione **paper** di questo lavoro (§3.3), che ne riproduce fedelmente la procedura.

Il campo visivo ristretto e la componente w_i^t sono due modifiche di questo lavoro. Il campo visivo limitato nasce da due ragioni distinte: una pratica e una teorica. La prima mira a ridurre il divario tra simulazione e realtà, per il quale è essenziale limitare l'uso della strumentazione tecnica necessaria per la raccolta dei dati a un'area il più circoscritta possibile. La seconda trova fondamento nella cinematica dei veicoli: ogni veicolo in questa simulazione raggiunge una velocità di percorrenza massima $v_{\text{max}} = 11.1$ m/s (circa 40 km/h) dettata dal limite di velocità imposto su ciascuna strada. Ciò implica che in un intervallo di tempo fissato $\Delta t = \Delta t_{\text{green}} + \Delta t_{\text{yellow}} = 13$ s, un veicolo possa percorrere una distanza non superiore a

$$d_{\text{vis}} = v_{\text{max}} \cdot \Delta t \approx 144.3 \text{ m.}$$

Il calcolo esclude i secondi di "tutto-rosso", in cui il deflusso è interrotto. Il campo visivo è stato quindi ristretto anche per forzare l'agente RL a concentrarsi esclusivamente sui veicoli che una fase semaforica può effettivamente influenzare. Sebbene in questa architettura il cutoff spaziale sia assunto omogeneo per l'intera rete, il codice che lo implementa tiene nativamente in considerazione eventuali casi di limiti di velocità o intervalli temporali variabili; la riduzione del campo visivo, inoltre, non viene applicata se la strada risulta più breve del cutoff stesso. Questa limitazione percettiva è riservata ai soli modelli di reinforcement learning di questo studio e non viene applicata alla baseline reattiva (Max-Pressure).

La componente w_i^t ha la stessa motivazione del termine di ricompensa MaxRedWait_i^t discusso più sotto: dare al modello un segnale diretto e locale sull'attesa, necessario perché possa correlare le proprie decisioni con l'equità tra le direzioni e non solo con

il numero di veicoli in coda.

Da un punto di vista pratico tutte queste feature possono essere calcolate tramite sensori stradali posti ai punti di stop dell'incrocio e a distanze d_{vis} da esso.

Azione ($\mathcal{A}$). $a_i^t \in \{0, \dots, 7\}$ seleziona una delle otto fasi che verranno descritte in §3.5. La transizione di "tutto-rosso" è gestita internamente dal simulatore e non appartiene allo spazio delle azioni. Il riferimento originale lascia all'agente la scelta libera tra tutte le 8 fasi a ogni passo. Questo lavoro introduce invece un vincolo *anti-starvation* (che contrasta cioè l'eccessiva attesa dei veicoli all'incrocio), che invalida la fase corrente come azione se è già stata selezionata due volte consecutive, forzando la rotazione:

$$\mathcal{A}_i^t = \{0, \dots, 7\} \setminus \{a_i^{t-1}\} \quad \text{se } a_i^{t-1} = a_i^{t-2}, \quad \mathcal{A}_i^t = \{0, \dots, 7\} \text{ altrimenti} \quad (3.2)$$

Il numero due non è arbitrario: nelle primissime fasi di training, quando la politica è ancora vicina al caso e non ha alcuna preferenza appresa, un agente senza questo vincolo tende a bloccarsi sulla prima fase che produce un ritorno anche solo leggermente positivo, ripetendola indefinitamente; vietare la ripetizione oltre la seconda volta consecutiva rompe questo ciclo senza impedire a una fase di restare attiva per un tempo ragionevole quando è davvero la scelta migliore per due passi di fila. In termini di tempo simulato, il vincolo limita a due gli step consecutivi sulla stessa fase, cioè 30 s, di cui 26 di verde (il resto è "tutto-rosso", §3.5). La maschera resta attiva anche in fase di valutazione, non solo durante l'addestramento.

Transizione (P). Essendo l'algoritmo di addestramento (Double DQN) *model-free*, le probabilità di transizione $P(s' \mid s, a)$ non vengono modellate esplicitamente, ma campionate empiricamente. L'ambiente CITYFLOW è per natura deterministico: fissati lo stato completo del simulatore e l'azione congiunta di tutte le intersezioni, la transizione dell'intero sistema collassa su un singolo stato successivo con probabilità 1. Questo non vale però per la coppia locale (s_i^t, a_i^t) di una singola intersezione: il suo stato successivo dipende anche dalle azioni scelte dagli altri agenti e dai veicoli fuori dal campo visivo di i , coerentemente con l'osservabilità parziale del Dec-POMDP (§2.2).

Fattore di sconto (γ). In questo lavoro viene fissato $\gamma = 0.85$ per ogni modello, senza alcuna ottimizzazione iperparametrica dedicata.

Ricompensa (R). Il riferimento originale impiega esclusivamente una ricompensa di pura pressione:

$$r_i^t = -\frac{P_i^t}{100}, \quad P_i^t = \text{Incoming}_i^t - \text{Outgoing}_i^t \quad (3.3)$$

dove P_i^t è la *pressione dell'intersezione* i al tempo t , ovvero la differenza tra il numero di veicoli in entrata (Incoming_i^t) e in uscita (Outgoing_i^t) dall'intersezione al tempo t . Il modello di questo lavoro sostituisce questa ricompensa con:

$$r_i^t = \text{clip} \left(\frac{-(\text{Incoming}_i^t - \text{Passed}_i^t) - \alpha \cdot \text{MaxRedWait}_i^t - \text{Penalty}_i^t}{100}, -2, 0 \right) \quad (3.4)$$

dove Incoming_i^t è il numero di veicoli entrati nella finestra visiva dell'intersezione al tempo t , Passed_i^t il numero di veicoli che hanno superato l'incrocio tra t e $t+1$, MaxRedWait_i^t il tempo di attesa massimo, in secondi, tra tutti i veicoli che si trovano nelle corsie con segnale rosso in ingresso all'istante t , $\text{Penalty}_i^t = 50$ se $\text{Passed}_i^t = 0 \wedge \text{Incoming}_i^t > 0$ e 0 altrimenti, e $\alpha \geq 0$ l'iperparametro con cui si modula la penalizzazione per lo *starvation* (l'eccessiva attesa dei veicoli); questo lavoro ne confronta tre valori, $\alpha \in \{0.08, 0.04, 0.00\}$ (Capitolo 4). Il fattore di scala $1/100$ è introdotto per stabilizzare i gradienti. Per il calcolo di tutte queste metriche ciascun modello fa riferimento, se impostata, alla distanza d_{vis} .

Il termine Penalty_i^t scatta quando la fase scelta non fa passare nessun veicolo pur essendocene in coda, il caso del "verde a vuoto": senza una penalizzazione esplicita l'agente può restare a lungo, nelle prime fasi di training, su una scelta degenerare di questo tipo senza ricevere un segnale di correzione chiaro. Il valore 50 (0.5 dopo la scala $1/100$) è scelto per essere comparabile in scala al termine $(\text{Passed}_i^t - \text{Incoming}_i^t)$, non derivato analiticamente. Il termine $\alpha \cdot \text{MaxRedWait}_i^t$ regola invece il concetto cardine di questa tesi: l'iperparametro α modula quanto la ricompensa penalizzi l'attesa massima registrata a un'intersezione, permettendo di studiare il compromesso tra la riduzione del tempo di percorrenza medio e il mantenimento dell'equità tra le direzioni di marcia (Capitolo 4). Infine il *clip* tra -2 e 0 , con il limite inferiore calibrato sperimentalmente nel corso della sperimentazione, evita che i picchi occasionali dei termini precedenti (in particolare MaxRedWait_i^t , potenzialmente illimitato) destabilizzino l'apprendimento con aggiornamenti di gradiente sproporzionati.

Topologia di Rete (G). Le N intersezioni sono rappresentate dai nodi di un grafo $G = (V, E)$. Un arco orientato $(i, j) \in E$ esiste se un segmento stradale connette fisicamente i a j . A ogni nodo è inoltre aggiunto un arco verso se stesso (*self-loop*),

cosicché i layer di *graph attention* possano pesare tanto lo stato del nodo stesso quanto quello dei vicini. Sotto il profilo della generalizzazione spaziale, la presente architettura impone che lo stato $s_i^{(t)}$ e le *meta feature* dipendano unicamente da dati locali, senza alcun accesso all'identità del nodo i all'interno della rete. Il modello non conosce quindi l'identità né la posizione assoluta del nodo, ma ne conosce il grado: tra le meta-feature (§3.2) compare il numero di incroci adiacenti, che distingue un incrocio d'angolo, di bordo o interno, senza però rivelare dove si trovi quell'incrocio nella rete.

3.2 Architettura MetaSTGAT

Il modello di questo lavoro riprende l'architettura di MetaSTGAT (Wang et al., 2022). La scelta di questo modello come punto di partenza è dettata dalla volontà di avere un sistema in grado di coordinare in modo efficace gli incroci di una rete stradale, così da minimizzare i tempi di percorrenza medi dei veicoli e, tramite le modifiche discusse in §3.4, indagare il compromesso con l'equità tra i vari flussi di veicoli. A differenza di Fixed-Time, MetaSTGAT è un modello sensibile allo stato della rete per quanto riguarda la scelta delle fasi; a differenza di Max-Pressure, la ricompensa di questo lavoro (Equazione 3.4) introduce esplicitamente un compromesso tra *throughput* (la capacità di smaltimento dei veicoli) e *starvation* (l'eccessivo tempo di attesa dei veicoli), a costo di una richiesta di informazioni più dettagliata da raccogliere a ogni istante decisionale (§3.1) e di una fase di addestramento.

L'architettura si articola nei seguenti componenti, organizzati in una pipeline che elabora lo stato di ogni intersezione (s_i^t) portandolo ad una rappresentazione compatta, utilizzata per produrre i Q-value (ovvero una stima della ricompensa attesa per ciascuna azione):

- un encoder di stato duale
- una cella ricorrente (LSTM)
- due istanze di un layer di *graph attention* (GAT)
- due moduli di meta-learning, spaziale e temporale
- una layer lineare per calcolare i Q-value.

Questa sezione presenta ciascun componente, soffermandosi sulla ragione che ha condotto Wang et al. (2022) alla sua introduzione. Prima di introdurre la generalizzazione "meta", ci si concentra sulla presentazione dell'architettura STGAT, ovvero

la versione standard del modello priva del meccanismo di meta-apprendimento. La Figura 3.1 anticipa in forma schematica la pipeline appena elencata, utile come riferimento visivo durante la lettura dei paragrafi che seguono.

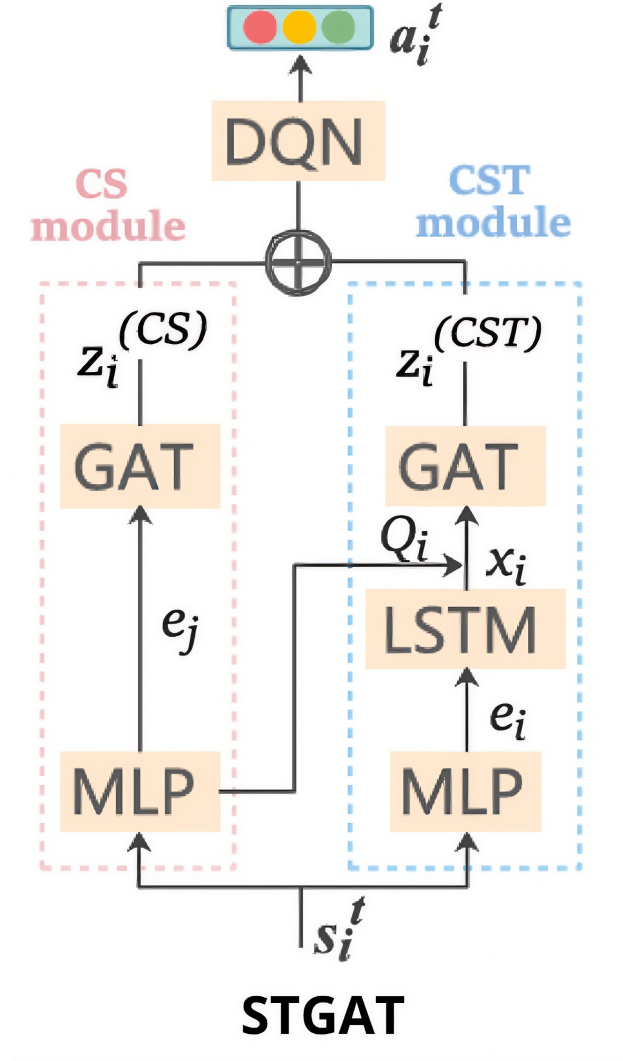


Figura 3.1: Schema a blocchi dell'architettura STGAT: dallo stato dell'intersezione s_i^t , attraverso l'encoder duale (MLP) e i moduli CS e CST (GAT e LSTM a pesi condivisi), fino all'azione a_i^t scelta dal Double DQN (il blocco "DQN" della figura, adattata dal riferimento originale, ne indica la testa dei Q-value). Adattato da Wang et al. (2022).

3.2.1 State Encoder duale

Lo stato $s_i^t \in \mathbb{R}^{d_0}$ ($d_0 = 32$ o 20 , §3.1) di un incrocio i al tempo t attraversa due Multilayer Perceptron (MLP) a due strati con la stessa architettura ma pesi e

inizializzazioni indipendenti:

$$e_i^t = \text{MLP}_t(s_i^t) = \sigma(W_{21} \sigma(W_{11}s_i^t + b_{11}) + b_{21}) \in \mathbb{R}^{d_1} \quad (3.5)$$

$$e_j^t = \text{MLP}_s(s_i^t) = \sigma(W_{22} \sigma(W_{12}s_i^t + b_{12}) + b_{22}) \in \mathbb{R}^{d_1} \quad (3.6)$$

dove d_1 viene fissato a 64, $W_{11}, W_{12} \in \mathbb{R}^{d_1 \times d_0}$ e $W_{21}, W_{22} \in \mathbb{R}^{d_1 \times d_1}$ sono i pesi, $b_{11}, b_{21}, b_{12}, b_{22} \in \mathbb{R}^{d_1}$ i bias e σ la funzione di attivazione ReLU. Il pedice j in e_j^t è un'etichetta del ramo spaziale, coerente con la notazione delle figure di questa sezione, non l'indice di un nodo vicino: anche e_j^t è calcolato a partire dallo stato s_i^t del nodo stesso.

Il vettore $e_i^t = \text{MLP}_t(s_i^t)$ verrà usato come input nella LSTM e veicolerà le caratteristiche temporali dello stato, mentre $e_j^t = \text{MLP}_s(s_i^t)$ verrà usato come input nelle GAT, catturando le caratteristiche spaziali dello stesso.

3.2.2 Modulo spazio-temporale corrente (CST)

Nel momento di una decisione, un conteggio istantaneo dei veicoli, da solo, non distingue una coda che si sta formando da una che si sta esaurendo. Due situazioni che possono richiedere una fase diversa appaiono dunque identiche a un modello privo di memoria. Il primo dei due moduli di STGAT viene introdotto per questo motivo: combina l'informazione temporale accumulata da un'intersezione con l'informazione spaziale corrente di sé stessa e dei suoi vicini, così che la scelta dell'azione possa tenere conto di come il traffico più ravvicinato si sta evolvendo, non solo di come si presenta localmente nell'istante attuale. Wang et al. (2022) motivano questo modulo osservando che altri lavori sulle Spatio-Temporal Graph Neural Networks (STGNN) propongono strutture simili, ma vi combinano le informazioni spaziali e temporali in un unico embedding oppure le usano separatamente; con il modulo CST cercano invece di intrecciare le due informazioni in modo più elaborato.

Il ramo temporale dell'encoder e_i^t (§3.2.1) attraversa anzitutto una cella LSTM con pesi condivisi da ogni nodo della rete:

$$\begin{bmatrix} f_i^t \\ \iota_i^t \\ o_i^t \\ g_i^t \end{bmatrix} = \begin{bmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{bmatrix} \left(W_{\{f, \iota, o, g\}}^\top [e_i^t \parallel h_i^{t-1}] + b_{\{f, \iota, o, g\}} \right), \quad (3.7)$$

$$c_i^t = f_i^t \odot c_i^{t-1} + \iota_i^t \odot g_i^t,$$

$$x_i^t = h_i^t = o_i^t \odot \tanh(c_i^t);$$

in notazione compatta:

$$(h_i^t, c_i^t) = \text{LSTM}(e_i^t; h_i^{t-1}, c_i^{t-1}). \quad (3.8)$$

Il vettore $x_i^t := h_i^t$ prodotto da questa cella rappresenta concettualmente un riassunto temporale dei flussi di traffico locali.

Lo stesso vettore, nel meccanismo di attenzione che segue, diventa l'informazione che i vicini dell'incrocio i "leggono" per decidere quanto pesare il contributo di i nel proprio contesto locale. Questa trasmissione di informazione avviene tramite l'applicazione di una Graph Attention Network (GAT) con una rappresentazione differente rispetto a quella della GAT standard di [Veličković et al. \(2018\)](#) riportata nel Capitolo 2. In tale GAT infatti ciascun nodo viene comparato con gli altri nodi secondo una stessa proiezione dell'embedding. Nella struttura proposta invece, prendendo ispirazione da approcci di machine translation ([Vaswani et al., 2017](#)), i nodi si influenzano vicendevolmente secondo valori distinti: Query, Key e Value. In particolare si scelgono come Query $Q = e_j^t$ (la codifica spaziale corrente del nodo ricevente) e come Key e Value $K = V = x_i^t$ (la memoria temporale di ciascun vicino). L'idea alla base di questa scelta è il tentativo di arricchire le caratteristiche temporali correnti di ciascun nodo con la memoria temporale dei flussi dello stesso e dei nodi vicini. Si precisa che nel caso di $H > 1$ teste di attenzione, come in questo lavoro, i valori Q, K, V non sono proiettati da matrici distinte per ciascuna testa, bensì il vettore a d_1 componenti viene partizionato in H segmenti contigui di dimensione $d_h = d_1/H$, e la testa h opera sul solo blocco $Q_h(i) := Q(i)[hd_h : (h+1)d_h]$ (analogamente per $K_h(u), V_h(u)$). Le teste restano quindi distinte perché leggono porzioni diverse dello stesso embedding, non perché dispongano di proiezioni apprese proprie.

Formalmente, nel caso in esame, per ciascun nodo i con vicinato $\mathcal{N}(i)$ (con $i \in \mathcal{N}(i)$) e h una delle $H = 4$ teste di dimensione $d_h = d_1/H = 16$, questo modulo di GAT viene descritto come segue:

$$\varphi_h(i, u) = \frac{Q_h(i) \cdot K_h(u)}{\sqrt{d_h}}, \quad (3.9)$$

$$\alpha_h(i, u) = \frac{\exp(\varphi_h(i, u))}{\sum_{u' \in \mathcal{N}(i)} \exp(\varphi_h(i, u'))},$$

$$z_h(i) = \sum_{u \in \mathcal{N}(i)} \alpha_h(i, u) V_h(u). \quad (3.10)$$

L'output finale di questo modulo spazio-temporale corrente sarà

$$z_{\text{CST}}(i) = [z_1(i) \parallel \dots \parallel z_H(i)] \in \mathbb{R}^{d_1}. \quad (3.11)$$

Auspiciabilmente, in questo stato è racchiusa una sintesi delle dinamiche spaziali e temporali del traffico locale e degli incroci adiacenti.

3.2.3 Modulo spaziale corrente (CS)

Il modulo spazio-temporale, filtrato dalla memoria della cella ricorrente, può attenuare segnali che sono invece rilevanti nell'istante presente; ad esempio un picco di traffico simultaneo su un intero asse necessita di coordinamento spaziale puro, a prescindere dalla durata del picco stesso. Questo secondo modulo, strutturalmente simile al primo, elabora perciò la sola informazione spaziale corrente e_j^t , senza alcuna componente di memoria. Il vettore e_j^t attraversa una GAT che opera secondo le stesse dinamiche descritte nel paragrafo precedente, ma con l'assegnazione $Q = K = V = e_j^t$. L'output finale del modulo spaziale corrente sarà

$$z_{\text{CS}}(i) = [z_1(i) \parallel \dots \parallel z_H(i)] \in \mathbb{R}^{d_1}, \quad (3.12)$$

Tale rappresentazione del traffico permetterà al modello di focalizzarsi sulla coordinazione dei nodi rispetto alla sola fotografia presente della rete.

3.2.4 Testa Q-value

Le due uscite, $z_{\text{CST}}(i)$ e $z_{\text{CS}}(i)$, catturano quindi aspetti complementari, l'uno informato dalla memoria temporale dei vicini, l'altro dalla sola situazione presente. Tali vettori vengono infine concatenati e attraversano un singolo layer lineare per produrre la stima del valore di ciascuna delle 8 azioni eseguibili all'incrocio i e all'istante t :

$$q(s_i^t, \cdot) = W_q [z_{\text{CST}}(i) \parallel z_{\text{CS}}(i)] + b_q \in \mathbb{R}^8 \quad (3.13)$$

dove $W_q \in \mathbb{R}^{8 \times 2d_1}$ e $b_q \in \mathbb{R}^8$.

3.2.5 Meta-knowledge learner

I due moduli appena descritti, alla base della struttura STGAT, condividono gli stessi pesi per ogni intersezione della rete, indipendentemente dal ruolo che occupa. Ad esempio un incrocio periferico, uno pienamente interno alla rete stradale, o uno attraversato da un'arteria a densità di traffico elevata (§3.5.3) applicano esattamente

la stessa regola di coordinamento. MetaSTGAT introduce un meccanismo di meta-apprendimento che condiziona tale regola su un embedding di caratteristiche locali dell'incrocio, cosicché i layer di GAT e di LSTM possano specializzarsi rispetto alle condizioni osservate da ciascun nodo invece di applicare la stessa funzione ovunque. L'idea alla base di questo meccanismo, condivisa da entrambi i componenti "meta" che seguono, è illustrata in Figura 3.2: le meta-feature spaziali e temporali, elaborate da un meta-learner dedicato, generano i pesi dei layer di LSTM e GAT al posto di parametri fissi condivisi da ogni nodo.

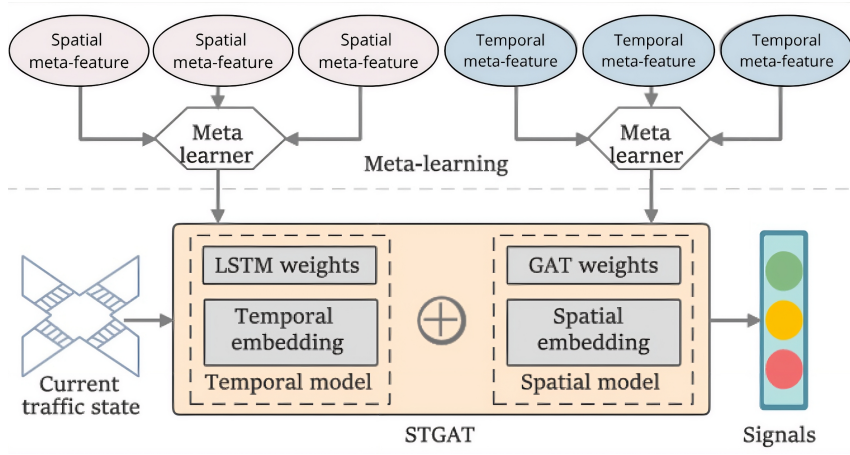


Figura 3.2: Principio del meta-apprendimento applicato da MetaSTGAT: le meta-feature spaziali e temporali condizionano, tramite un meta-learner dedicato, i pesi dei layer LSTM e GAT usati per elaborare lo stato corrente del traffico. Adattato da Wang et al. (2022).

In particolare, due MLP a due strati, uno per l'aspetto spaziale e uno per quello temporale del problema, apprendono un embedding di caratteristiche locali dell'intersezione (Spatial Meta-Knowledge, SMK; Temporal Meta-Knowledge, TMK) a partire dalle meta-feature che descrivono il nodo (x_i^{smk} e x_i^{tmk}):

$$\text{SMK}(i) = \text{MLP}_{\text{smk}}(x_i^{\text{smk}}) \in \mathbb{R}^{64} \quad (3.14)$$

$$\text{TMK}(i) = \text{MLP}_{\text{tmk}}(x_i^{\text{tmk}}) \in \mathbb{R}^{64} \quad (3.15)$$

Come già accennato, tali valori verranno poi usati per generare i pesi di Meta-GAT e Meta-LSTM, tramite dei MLP addizionali specializzati per ciascuno dei moduli, secondo quanto riporteremo in (§3.2.6) e (§3.2.7). Il passaggio dalle meta-feature grezze ai due embedding $\text{SMK}(i)$ e $\text{TMK}(i)$ è riassunto graficamente in Figura 3.3.

Le meta-feature x_i^{smk} e x_i^{tmk} riprendono l'insieme di informazioni del riferimento originale, adattato in questo lavoro; sono pensate per fornire al modello informazio-

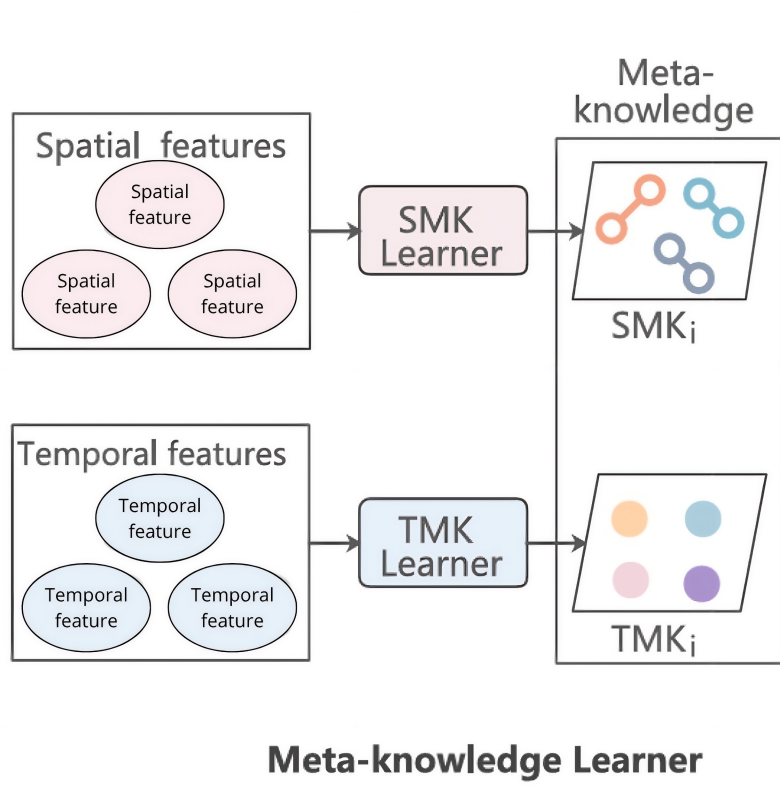


Figura 3.3: Il Meta-knowledge Learner: le meta-feature spaziali e temporali grezze vengono elaborate rispettivamente dallo SMK-Learner e dal TMK-Learner, producendo gli embedding $SMK(i)$ e $TMK(i)$ usati da Meta-GAT e Meta-LSTM. Adattato da Wang et al. (2022).

ni aggiuntive che ne consentano una migliore specializzazione rispetto al contesto specifico di ciascun incrocio.

Il vettore spaziale $x_i^{\text{smk}} \in \mathbb{R}^{26}$ concatena cinque gruppi di feature, ciascuno variabile da nodo a nodo:

- pressione per corsia [12]: per ciascuna corsia, la differenza tra i veicoli in ingresso sulla corsia e la media dei veicoli sulle corsie in uscita raggiungibili da essa, divisa per 30;
- numero di incroci adiacenti [1]: frazione tra il numero di vicini del nodo e il massimo previsto dalla topologia a griglia (4), distingue quindi un incrocio d'angolo, di bordo o interno;
- differenza tra la pressione media propria e quella di ciascun incrocio adiacente [4]: per ciascuno dei vicini, la pressione media delle 12 corsie del nodo meno quella del vicino, con zero-padding negli slot oltre il numero di vicini effettivamente presenti;
- asimmetria di pressione tra le corsie N/S e le corsie E/O dello stesso nodo [1]: media della pressione sulle 6 corsie N/S meno la media sulle 6 corsie E/O;
- pressione per fase [8]: la stessa quantità dell'Equazione 2.12, usata da Max-Pressure (§2.5.2) per decidere la fase da selezionare, una componente per ciascuna delle 8 fasi.

Il costo di raccolta di queste feature si limita a sensori con la stessa capacità visiva già richiesta per lo stato principale e per il reward, sia sulle corsie in ingresso sia su quelle in uscita: le feature di pressione si misurano semplicemente contando i veicoli su queste corsie, mentre il numero di incroci adiacenti è un dato topologico costante per ciascun incrocio.

Il vettore di meta-feature temporali $x_i^{\text{tmk}} \in \mathbb{R}^{25}$ concatena invece tre diversi tipi di informazione, tutte variabili nel tempo all'interno dello stesso nodo:

- variazione della coda per corsia rispetto a 3 passi precedenti [12]: per ciascuna corsia, differenza tra il numero di veicoli in coda all'istante corrente e quello di 3 step prima;
- durata di permanenza della fase corrente [1]: numero di step consecutivi in cui la fase è rimasta attiva, saturato e normalizzato in $[0, 1]$ rispetto a un tetto di 2 step, oltre il quale la maschera anti-starvation di questo lavoro (§3.1) impone comunque un cambio di fase;

- deviazione standard della coda per corsia sulla finestra recente [12]: calcolata sugli ultimi 5 step disponibili.

Anche queste feature vengono calcolate a partire da informazioni fornite dal simulatore e non richiedono strumenti aggiuntivi.

Si noti come nessuna di queste feature entri anche nello stato principale: la scelta è legata alla volontà di non appesantire lo stato del sistema, sfruttando le stesse per agire direttamente sui pesi che regolano la trasmissione dell'informazione.

3.2.6 Meta-GAT

La versione meta delle GAT usate in STGAT riprende la struttura architetturale delle stesse, ma implementa delle modifiche mirate a consentire ai layer di apprendere regole di aggregazione spaziale dipendenti dalle caratteristiche del singolo nodo. I valori assegnati a Q, K, V rimangono invariati, sia per il modulo spaziale ($Q = K = V = e_j^t$) sia per il modulo spazio-temporale ($Q = e_j^t, K = V = x_i^t$). L'unica differenza riguarda l'inserimento, nel punteggio di compatibilità dell'Equazione 3.9, di una trasformazione lineare per nodo che in STGAT non esiste: il modulo CS riceve il proprio meta-embedding da $\text{SMK}(i)$, il modulo CST da $\text{TMK}(i)$, in modo che la regola di attenzione spaziale e quella spazio-temporale si adattino a caratteristiche locali diverse. In particolare, per ciascun nodo i e ciascuna testa h , una rete $\text{MLP}_{\text{GAT}, \text{smk}, h}$ condivisa da tutti i nodi genera, dal solo embedding meta del nodo ricevente, i pesi del modulo spaziale:

$$W_{j,h}^{(i)}, b_{j,h}^{(i)} = \text{MLP}_{\text{GAT}, \text{smk}, h}(\text{SMK}(i)) \in \mathbb{R}^{d_h \times d_h}, \mathbb{R} \quad (3.16)$$

e analogamente, tramite $\text{MLP}_{\text{GAT}, \text{tmk}, h}$, per il modulo spazio-temporale:

$$W_{k,h}^{(i)}, b_{k,h}^{(i)} = \text{MLP}_{\text{GAT}, \text{tmk}, h}(\text{TMK}(i)) \in \mathbb{R}^{d_h \times d_h}, \mathbb{R} \quad (3.17)$$

dove le iper-reti MLP sono composte da due layer, condivise da tutti i nodi della rete: solo il loro output dipende da i , tramite l'input $\text{SMK}(i)$ o $\text{TMK}(i)$. E dunque, per ogni nodo $u \in \mathcal{N}(i)$:

$$\varphi_h(i, u) = \frac{(W_h^{(i)} Q_h(i)) \cdot K_h(u) + b_h^{(i)}}{\sqrt{d_h}}, \quad (3.18)$$

$$\alpha_h(i, u) = \frac{\exp(\varphi_h(i, u))}{\sum_{u' \in \mathcal{N}(i)} \exp(\varphi_h(i, u'))},$$

$$z_h(i) = \sum_{u \in \mathcal{N}(i)} \alpha_h(i, u) V_h(u),$$

$$z(i) = [z_1(i) \parallel \dots \parallel z_H(i)] \in \mathbb{R}^{d_1},$$

con $W_h^{(i)}$ e $b_h^{(i)}$ relativo al proprio modulo di riferimento. La Figura 3.4 mette a confronto i due moduli appena descritti, evidenziando come SMK e TMK alimentino iper-reti distinte per generare i pesi (W, b) specifici di ciascuno.

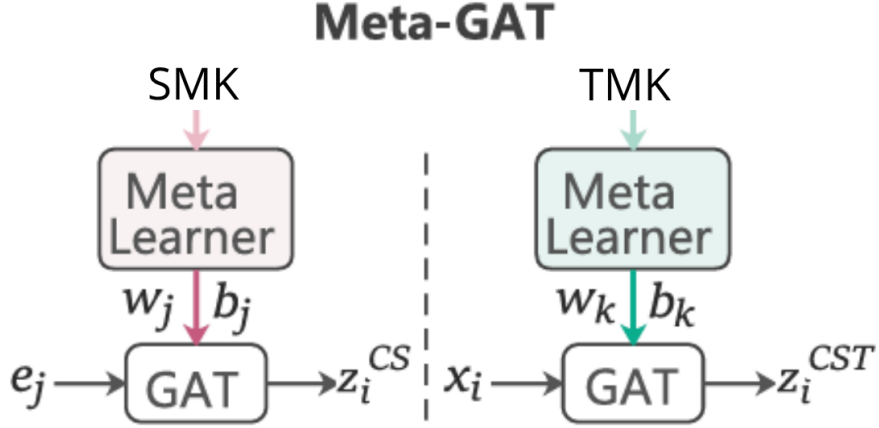


Figura 3.4: Meta-GAT: il modulo CS (sinistra) genera i pesi (W_j, b_j) della propria GAT a partire da SMK, il modulo CST (destra) genera (W_k, b_k) a partire da TMK; entrambi producono l'output z_i del rispettivo modulo. Adattato da Wang et al. (2022).

3.2.7 Meta-LSTM

Meta-LSTM applica alla cella ricorrente di STGAT la stessa generalizzazione che Meta-GAT applica al *graph attention*: i pesi $W_{(f,\ell,o,g)}$ e i bias $b_{(f,\ell,o,g)}$ condivisi in STGAT da ogni nodo della rete, sono qui sostituiti da pesi e bias generati individualmente per ogni nodo.

Un MLP a due layer per ciascun gate genera, dal solo $\text{TMK}(i)$, i pesi $W_{(f,\ell,o,g)}^{(i)} \in \mathbb{R}^{2d_1 \times d_1}$ e i bias $b_{(f,\ell,o,g)}^{(i)} \in \mathbb{R}^{d_1}$, modificando l'Equazione 3.7 come segue:

$$\begin{bmatrix} f_i^t \\ \iota_i^t \\ o_i^t \\ g_i^t \end{bmatrix} = \begin{bmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{bmatrix} \left(\left(W_{\{f,\ell,o,g\}}^{(i)} \right)^\top [e_i^t \parallel h_i^{t-1}] + b_{\{f,\ell,o,g\}}^{(i)} \right) \quad (3.19)$$

$$c_i^t = f_i^t \odot c_i^{t-1} + \iota_i^t \odot g_i^t \quad (3.20)$$

$$x_i^t = h_i^t = o_i^t \odot \tanh(c_i^t) \quad (3.21)$$

In notazione compatta, esplicitando la sola dipendenza aggiuntiva dai pesi generati dal meta-learner:

$$(h_i^t, c_i^t) = \text{LSTM}(e_i^t; h_i^{t-1}, c_i^{t-1} \mid W_{\{f,\ell,o,g\}}^{(i)}, b_{\{f,\ell,o,g\}}^{(i)}), \quad x_i^t := h_i^t. \quad (3.22)$$

L'output x_i^t alimenta il modulo CST di Meta-GAT come K e V (§3.2.6), analogamente a quanto visto con STGAT. L'architettura appena descritta è riassunta graficamente in Figura 3.5, dove W_Φ, b_Φ indicano genericamente i pesi e i bias generati dal meta-learner.

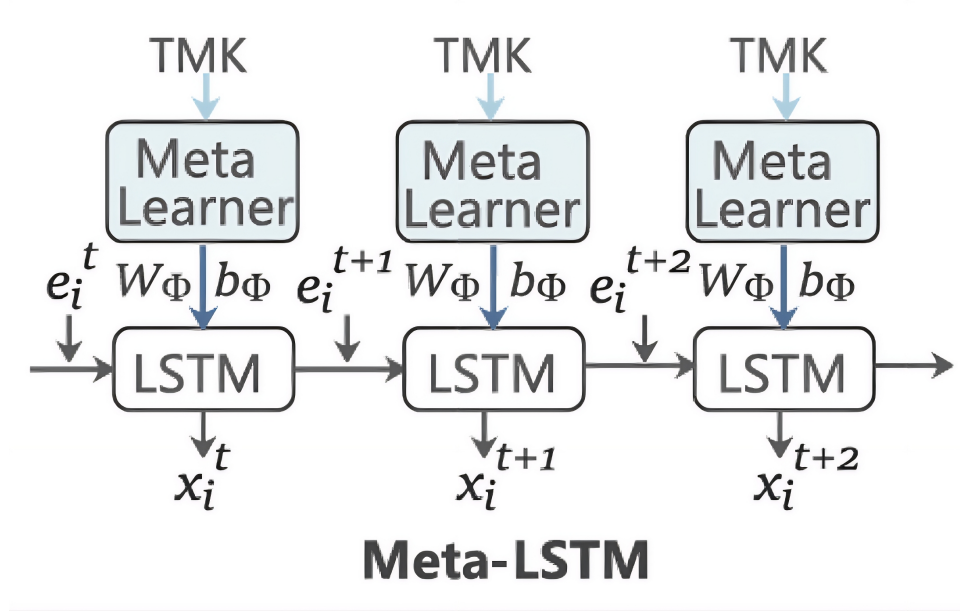


Figura 3.5: Meta-LSTM "srotolata" nel tempo: a ogni istante t il meta-learner genera, da $TMK(i)$, i pesi W_Φ e b_Φ della cella LSTM che produce x_i^t dall'input e_i^t e dallo stato nascosto precedente h_i^{t-1}, c_i^{t-1} . Adattato da Wang et al. (2022).

Messi insieme, i moduli descritti in questa sezione compongono l'architettura di Figura 3.6, che ne riassume il flusso complessivo dallo stato s_i^t all'azione scelta.

3.2.8 Meccanismi spaziali alternativi

Il modulo Meta-GAT decide quanto ciascun vicino debba contribuire all'aggiornamento di un'intersezione tramite un punteggio di compatibilità appreso, specifico per ogni nodo (§3.2.6). Questa formulazione rappresenta tuttavia solo una delle possibili declinazioni del paradigma di *message passing* su grafo. Con l'obiettivo di esplorare soluzioni alternative, in questa sezione vengono introdotte ed estese in chiave meta-learning due diverse architetture di aggregazione spaziale: Meta-GCN

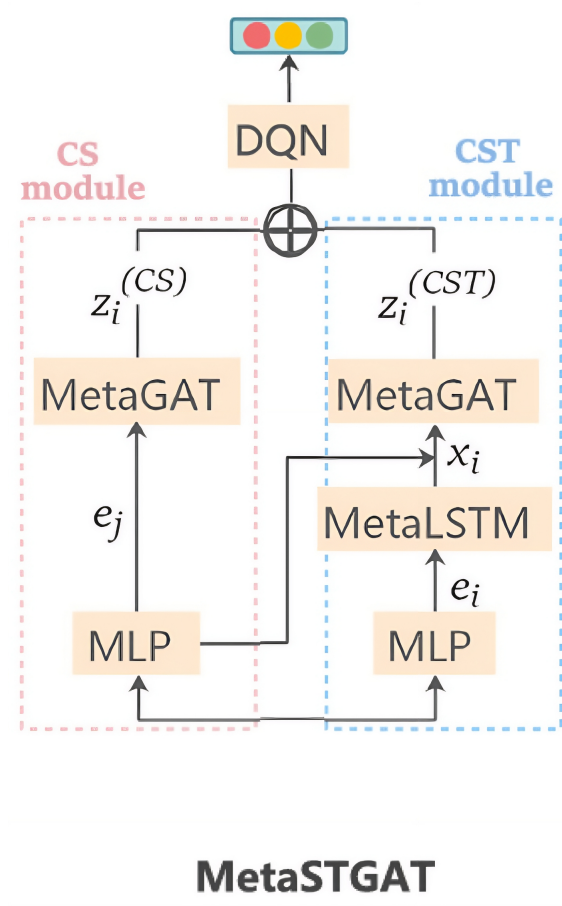


Figura 3.6: Schema a blocchi completo di MetaSTGAT: rispetto a STGAT (Figura 3.1), GAT e LSTM sono sostituiti dalle rispettive versioni meta-condizionate, con i pesi generati dagli embedding $SMK(i)$ e $TMK(i)$ del Meta-knowledge Learner. Adattato da Wang et al. (2022).

e Meta-SONAR, presentate come sostituzione del solo blocco GAT di Figura 3.6, a parità di tutto il resto.

Ogni altro componente dell'architettura (encoder di stato, meta-learner spaziale e temporale, Meta-LSTM, testa Q-value) e la procedura di training (§3.3) rimangono invariati: a parità di tutto il resto, le differenze di prestazione misurate nel Capitolo 4 si possono quindi ricondurre principalmente al solo meccanismo di aggregazione spaziale, entro la variabilità tra seed.

La prima architettura, una *graph convolution network* (GCN), rimuove il punteggio di compatibilità della GAT e lo sostituisce con un peso fisso determinato solo dalla topologia. La seconda, una propagazione ondulatoria a lungo raggio chiamata SONAR (Trenta et al., 2025), sostituisce l'aggregazione a un passo con l'evoluzione discretizzata di un sistema dinamico su più iterazioni interne.

In entrambi i casi il principio "meta" resta identico a Meta-GAT: i pesi dell'aggregazione sono generati da un embedding SMK/TMK per ciascun nodo, non appresi come parametro fisso condiviso.

3.2.8.1 Meta-GCN

La GCN (Kipf and Welling, 2017), introdotta nel Capitolo 2 (Equazione 2.7), pesa il contributo di ogni vicino con la normalizzazione simmetrica del grado: una quantità che dipende solo dalla topologia della rete, mai dal contenuto dello stato dei nodi coinvolti. È dunque il termine di paragone più diretto per isolare cosa aggiunge il punteggio di compatibilità di Meta-GAT: sostituendolo con questa regola fissa, e lasciando invariato tutto il resto dell'architettura, ogni differenza di prestazioni misurata nel Capitolo 4 è imputabile a quel solo meccanismo.

Per un nodo i , con $\mathcal{N}(i)$ il suo vicinato (incluso il self-loop permanente di questo lavoro, §3.1) e $\hat{d}_i = |\mathcal{N}(i)|$ il grado corrispondente, il principio meta genera W e b per ciascun nodo ricevente i , a partire dal suo embedding meta m_i (pari a $\text{SMK}(i)$ per il modulo CS e a $\text{TMK}(i)$ per il modulo CST), tramite un'iper-rete a due strati condivisa da tutti i nodi, in modo analogo a quanto visto in Meta-GAT:

$$W^{(i)}, b^{(i)} = \text{MLP}_{\text{GCN},m}(m_i) \in \mathbb{R}^{d_1 \times d_1}, \mathbb{R}$$

Poi, per ogni nodo i , la regola di aggregazione, che nella versione originale non-meta della GCN userebbe una matrice W condivisa da ogni nodo della rete (Equazione 2.7), diventa:

$$z(i) = \sigma \left(\left(\sum_{u \in \mathcal{N}(i)} \frac{1}{\sqrt{\hat{d}_i \hat{d}_u}} W^{(i)} V(u) \right) + b^{(i)} \right) \quad (3.23)$$

dove σ è la funzione di attivazione ReLU e $V(u)$ il valore del nodo u (h_u nell'Equazione 2.7 del Capitolo 2). Come per Meta-GAT, il modulo CST assegna $V = x_i^t$ (l'uscita della Meta-LSTM, §3.2.7) e $m_i = \text{TMK}(i)$; il modulo CS assegna $V = e_j^t$ e $m_i = \text{SMK}(i)$.

3.2.8.2 Meta-SONAR

Sia Meta-GAT sia Meta-GCN propagano informazione a un solo passo di distanza per ogni applicazione del layer: raggiungere un vicino a k archi di distanza richiede k applicazioni. SONAR (Trenta et al., 2025), introdotto nel Capitolo 2 (§2.4), evita questo costo modellando la propagazione come un sistema dinamico continuo, la cui evoluzione nel tempo, non nello spazio dei layer, porta l'informazione a più hop di distanza in un'unica applicazione. Nella formulazione originale, richiamata in dettaglio nel Capitolo 2, la conduttanza a_{ui} , la dissipazione $D(\cdot)$ e la forzante $F(\cdot)$ dell'Equazione 2.11 sono reti a pesi fissi, condivise da tutti i nodi, senza alcuna nozione di meta-apprendimento. In questo lavoro W nell'Equazione 2.10 è l'identità: il termine si riduce così al solo accoppiamento spaziale pesato dalla conduttanza a_{ui} , un peso scalare per coppia di nodi, analogo al punteggio di compatibilità di Meta-GAT o al peso di normalizzazione del grado di Meta-GCN.

Questo lavoro applica il principio meta alla sola conduttanza a_{ui} , l'analogo diretto del peso appreso per coppia di nodi di Meta-GAT e Meta-GCN: una rete a due strati genera, dall'embedding meta m_i del nodo ricevente i , i parametri $W_a^{(i)} \in \mathbb{R}^{d_1 \times 1}$ e $b_a^{(i)} \in \mathbb{R}$, da cui

$$a_{ui} = \text{ReLU}\left((X_u - X_i)W_a^{(i)} + b_a^{(i)}\right) \quad (3.24)$$

la ReLU garantisce $a_{ui} \geq 0$, coerente con l'interpretazione di a_{ui} come conduttanza. Dissipazione $D(\cdot)$ e forzante $F(\cdot)$ restano reti a pesi fissi applicate allo stato corrente del nodo (un solo layer lineare seguito da ReLU per $D(\cdot)$, due layer lineari con una ReLU intermedia per $F(\cdot)$): sono filtri locali, non meccanismi di comunicazione tra nodi, per cui il legame con le caratteristiche locali codificate da SMK/TMK è più debole che per la conduttanza.

La posizione iniziale X^0 e la velocità iniziale V^0 sono prodotte in modo analogo a Meta-GAT: per il modulo CST $X^0 = x_i^t$ (l'uscita della Meta-LSTM) e $V^0 = W_V e_j^t$ con $m_i = \text{TMK}(i)$, per il modulo CS: $X^0 = e_j^t$ e $V^0 = W_V e_j^t$ con $m_i = \text{SMK}(i)$, dove $W_V \in \mathbb{R}^{d_1 \times d_1}$ è una matrice di proiezione fissa e condivisa da tutti i nodi, analoga in questo a $D(\cdot)$ e $F(\cdot)$.

A differenza di Meta-GAT e Meta-GCN, dove la profondità spaziale si ottiene impilando più applicazioni del layer a pesi indipendenti, in Meta-SONAR le L iterazioni dell'Equazione 2.11 condividono gli stessi pesi $W_a^{(i)}$, la stessa proprietà di

costo costante rispetto a L già discussa per la formulazione originale nel Capitolo 2. Questo lavoro confronta $L = 2$ e $L = 4$ iterazioni, con passo di integrazione $h = 0.1$ in entrambi i casi (Capitolo 4).

3.3 Procedura di addestramento

I modelli di questo lavoro, a meno di ablazioni specifiche, sono addestrati con Double DQN su sequenze campionate da un replay buffer prioritizzato, con burn-in e Back-propagation Through Time (BPTT), Huber loss pesata dagli *importance-sampling weight* di PER, hard update della target network e gradient clipping; l'ottimizzatore è RMSprop per fedeltà al riferimento originale.

Algoritmo di Deep Reinforcement Learning. Il riferimento originale impiega DQN standard. Il modello di questo lavoro adotta invece Double DQN (van Hasselt et al., 2016) (§2.2), che disaccoppia selezione e valutazione dell'azione nel calcolo del target, riducendo il bias di sovrastima del valore Q tipico di DQN.

Funzione di loss. Le componenti di Double DQN, Huber loss e PER sono definite in astratto, per una singola transizione, in §2.2; questo paragrafo formalizza il modo in cui si combinano sulle sequenze ricorrenti e multi-nodo di questo lavoro.

Sia N il numero di intersezioni della rete e ℓ_{seq} la lunghezza di una sequenza campionata k , di cui le prime ℓ_{burn} istanze servono solo al burn-in per inizializzare lo stato ricorrente (h, c) , mentre le successive $\ell_{\text{grad}} = \ell_{\text{seq}} - \ell_{\text{burn}}$ istanze contribuiscono al gradiente. Per ciascun nodo i e ciascun passo t successivo al burn-in, il target Double DQN è:

$$y_i^t = r_i^t + \gamma Q(s_i^{t+1}, \underset{a'}{\operatorname{argmax}} Q(s_i^{t+1}, a'; \theta); \theta^-) \quad (3.25)$$

dove online e target network elaborano lo stesso stato ricorrente propagato dal burn-in. L'errore di differenza temporale è

$$\delta_i^t = Q(s_i^t, a_i^t; \theta) - y_i^t,$$

e la loss della sequenza k è la Huber loss media su tutti i nodi e i passi validi:

$$\mathcal{L}_k = \frac{1}{N \ell_{\text{grad}}} \sum_{i=1}^N \sum_t L_{\kappa}(\delta_i^t) \quad (3.26)$$

con soglia di Huber $\kappa = 1$ (il valore di libreria usato senza ritocchi, §2.2). Il campionamento dal replay buffer avviene per sequenze intere, non per transizioni isolate, di conseguenza anche la priorità del PER è definita a livello di sequenza, come la media di $|\delta_i^t|$ sugli stessi nodi e passi di $\mathcal{L}_k$, e aggiornata dopo ogni batch. La loss ottimizzata a ogni passo di gradiente è la media, sulle B sequenze del batch, delle loss $\mathcal{L}_k$ pesate dai rispettivi coefficienti di importance sampling w_k :

$$\mathcal{L} = \frac{1}{B} \sum_{k=1}^B w_k \mathcal{L}_k. \quad (3.27)$$

Il calcolo di $\mathcal{L}$ è seguito da un solo passo di RMSprop e dal gradient clipping descritti sotto.

Calcolo del gradiente. Il riferimento originale allena la componente ricorrente (LSTM) senza una fase di burn-in, calcolando il gradiente a partire dall’istanza campionata con lo stato ricorrente (h, c) non ricostruito, un disallineamento tra lo stato ricorrente osservato durante la raccolta dati e quello ricostruito in training. Qui si campionano invece sequenze di $\ell_{\text{seq}} = 4$ transizioni dal replay buffer, con un burn-in di $\ell_{\text{burn}} = 2$ passi secondo lo schema R2D2 (Kapturowski et al., 2019): i passi di burn-in aggiornano (h, c) senza contribuire al gradiente, correggendo il disallineamento appena descritto, mentre i successivi $\ell_{\text{grad}} = 2$ passi aggiornano i pesi del modello. Si applica infine un *gradient clipping*, limitando a norma 1 il gradiente, al fine di stabilizzare la convergenza.

Replay buffer. Il riferimento originale utilizza un replay buffer con una capacità di 10.000 transizioni congiunte (l’intero stato della rete a un singolo istante) e con campionamento uniforme. Si adotta qui invece un replay buffer organizzato per sequenze, con Prioritized Experience Replay (PER, Schaul et al., 2016), la relativa correzione tramite pesi di *importance sampling* (§2.2) e una capacità di 4.800 transizioni congiunte, equivalenti a 40 episodi di storia. Il PER concentra l’aggiornamento dei pesi sulle sequenze con errore di stima più alto, anziché trattare queste ultime allo stesso modo delle transizioni già ben predette; limitare la capacità del buffer agli ultimi episodi, invece di conservarne l’intera storia, mantiene l’esperienza di training coerente con una politica che ha già imparato qualcosa, invece di continuare a ricampionare le traiettorie quasi casuali dei primissimi episodi. La configurazione **paper** (§3.4), che riproduce fedelmente la procedura del riferimento originale, torna alla capacità di 10.000 transizioni e al campionamento uniforme.

Funzione di loss (confronto). Il riferimento originale impiega l'errore quadratico medio (MSE) tra Q predetto e target. Si predilige qui la Huber loss (§2.2), più robusta agli outlier della MSE, pesata dai coefficienti di importance sampling introdotti dal PER.

Ottimizzatore e target network. L'ottimizzatore RMSprop è usato con *learning rate* 10^{-3} , mantenendo i restanti iperparametri (fattore di smoothing, ε numerico, momentum) ai valori di libreria, non ritoccati in questo lavoro. L'aggiornamento della target network è una copia periodica integrale dei pesi della rete online, non una media mobile: ogni 100 step di ottimizzazione, il bersaglio dell'Equazione 3.25 viene sostituito per intero. Con 100 aggiornamenti del gradiente per episodio (paragrafo successivo) e una frequenza di sostituzione di 100 step, la target network viene quindi sincronizzata esattamente una volta per episodio.

Gli iperparametri usati nel presente studio sono riassunti in Tabella 3.1.

Ciclo di training. Lo pseudocodice seguente descrive il ciclo esterno di addestramento; le equazioni di questa sezione ne dettagliano il calcolo della loss.

Algorithm 1 Ciclo di addestramento (per modello)

```

1: Inizializza rete online  $\theta$ , rete target  $\theta^- \leftarrow \theta$ , replay buffer  $\mathcal{D}$ 
2: for 10 episodi di warm-up do
3:   Esegui un episodio con  $\varepsilon = 1$  (azioni interamente casuali), popola  $\mathcal{D}$ 
4: end for
5: for episodio = 1, ..., 100 do
6:   if episodio mod 10 = 0 then
7:     Esegui l'episodio con  $\varepsilon = 1$ , senza aggiornare  $\theta$  (esplorazione ciclica)
8:   else
9:     Esegui l'episodio con la politica  $\varepsilon$ -greedy corrente, popola  $\mathcal{D}$ 
10:    for 100 aggiornamenti del gradiente do
11:      Campiona un batch di  $B$  sequenze da  $\mathcal{D}$  (PER)
12:      Calcola  $\mathcal{L}$  (Equazioni 3.26–3.27), aggiorna le priorità
13:      Un passo di RMSprop su  $\theta$ , con gradient clipping
14:      if numero totale di aggiornamenti mod 100 = 0 then
15:         $\theta^- \leftarrow \theta$  (hard update della target network)
16:      end if
17:    end for
18:    Decadi  $\varepsilon$  verso il suo minimo (raggiunto al 60% degli episodi)
19:  end if
20: end for

```

L'idea alla base dell'esplorazione ciclica è quella di continuare a inserire nel buffer esperienza che la politica ormai quasi deterministica non visiterebbe più spon-

Parametro	Valore	Origine	Motivazione
learning rate	10^{-3}	questo lavoro	–
γ	0.85	riferimento originale	–
α (equità)	$\{0.08, 0.04, 0.00\}$	questo lavoro	sweep, Capitolo 4
d_1	64	questo lavoro	–
teste H	4	riferimento originale	–
d_{vis}	144.3 m	valore derivato	$v_{\text{max}}(\Delta t_{\text{green}} + \Delta t_{\text{yellow}})$
Penalty	50 (0.5 dopo /100)	questo lavoro	comparabile in scala a (Passed–Incoming)
clip ricompensa	$[-2, 0]$	questo lavoro	stabilizzare il gradiente da MaxRedWait
ε -greedy	$0.9 \rightarrow 0.01$	questo lavoro	decadimento $\approx 0.928/\text{episodio}$, minimo all’episodio 60
esplorazione ciclica	1 episodio ogni 10 a $\varepsilon = 1$	questo lavoro	–
batch size B	20	riferimento originale	–
capacità buffer	4.800 transizioni congiunte	questo lavoro	= 40 episodi
ℓ_{seq} , burn-in	4, 2	questo lavoro	schema R2D2
PER α	0.6	libreria	–
PER β	$0.4 \rightarrow 1.0$, lineare su 10^5 frame	libreria	–
PER ζ	10^{-5}	libreria	evita priorità nulla
Huber κ	1.0	libreria	–
gradient clipping (norma)	1.0	questo lavoro	–
ottimizzatore	RMSprop, solo lr impostato	libreria (resto)	fedeltà al riferimento originale
target update	hard, ogni 100 step	questo lavoro	1 sync/episodio
dimensioni nascoste	64 ovunque	questo lavoro	encoder, SMK/TMK, iperreti
SONAR L, h	$\{2, 4\}, 0.1$	questo lavoro	–
episodi, warm-up	100, 10	questo lavoro	warm-up interamente casuale
update per episodio	100	riferimento originale	–
seed di training	1 per modello	questo lavoro	nessuno sweep multi-seed in training
artery boost	3.0	questo lavoro	pesa l’instradamento, non aggiunge veicoli
software	Python 3.10, PyTorch 2.13 (CPU), torch_geometric 2.8, CityFlow 0.1	questo lavoro	–

Tabella 3.1: Iperparametri di addestramento e architettura del presente studio.

taneamente, al fine di dare al modello la possibilità di non convergere su soluzioni sub-ottimali.

3.4 Differenze rispetto al riferimento originale

La Tabella 3.2 raccoglie in forma sintetica ogni differenza tra questo lavoro e il riferimento originale, introdotta e argomentata nel dettaglio nelle sezioni precedenti.

Componente	Wang et al. (2022)	Questo lavoro	Sezione
Campo visivo	illimitato	limitato a $d_{\text{vis}} \approx 144.3$ m	§3.1
Stato	n_i^t, p_i^t ($d_0 = 20$)	aggiunge w_i^t ($d_0 = 32$)	§3.1
Ricompensa	pressione pura (Eq. 3.3)	Passed–Incoming, Max-RedWait, Penalty, clip (Eq. 3.4)	§3.1
Maschera azioni	nessuna	anti-starvation, 2 fasi consecutive	§3.1
Meta-feature	definite dagli autori	adattate (es. tetto a 2 step legato alla maschera)	§3.2
Algoritmo	DQN	Double DQN	§3.3
Ricorrenza	nessun burn-in	burn-in 2 + BPTT 2 (R2D2)	§3.3
Replay buffer	10.000 transizioni, uniforme	4.800 transizioni, PER	§3.3
Loss	MSE	Huber, pesata da PER	§3.3
Gradient clipping	assente	norma 1	§3.3
Svolta a destra	libera, senza attendere il verde	consentita solo se autorizzata dalla fase	§3.5

Tabella 3.2: Sintesi delle differenze tra questo lavoro e il riferimento originale.

A partire da queste differenze, questo lavoro definisce due configurazioni di riferimento, usate nel Capitolo 4: **MetaSTGAT Pro** applica tutte le modifiche elencate in Tabella 3.2, ed è la configurazione a cui questo capitolo fa riferimento salvo indicazione contraria; **paper** riproduce invece fedelmente la procedura di Wang et al. (2022), tornando ai valori della colonna centrale della tabella. Il Capitolo 4 valuta inoltre, tramite uno studio di ablazione dedicato, il contributo di ciascun gruppo di modifiche isolatamente, tramite configurazioni intermedie tra queste due.

3.5 Ambiente di simulazione

L'ambiente di simulazione si basa su CITYFLOW (Zhang et al., 2019), un simulatore microscopico open source con motore in C++ e binding in Python, progettato specificamente per il controllo semaforico su larga scala tramite reinforcement learning, per le ragioni già discusse nella Sezione 2.5.3. La Figura 3.7 ne mostra il visualizzatore su una singola intersezione.

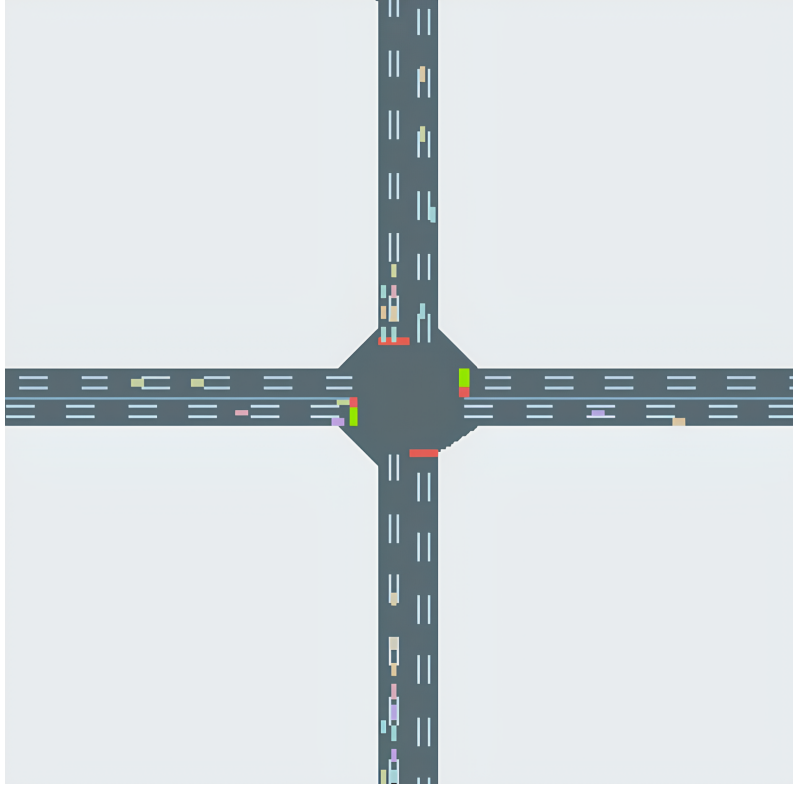


Figura 3.7: Rendering del visualizzatore di CITYFLOW per una singola intersezione a quattro vie: le indicazioni verde/rosso ai margini dell'incrocio mostrano i movimenti abilitati dalla fase corrente, i rettangoli colorati sulle corsie i veicoli in transito o in coda.

3.5.1 Struttura della rete e mappatura delle fasi

La topologia della rete è codificata in un file di configurazione (*roadnet*), che definisce l'insieme delle intersezioni (nodi) e dei segmenti stradali (archi) con le relative molteplicità di corsia. Per ogni intersezione il *roadnet* elenca le fasi semaforiche disponibili, ciascuna definita come l'insieme dei movimenti (coppie corsia di origine e di destinazione) abilitati in quella fase. L'azione dell'agente si riduce pertanto alla selezione dell'indice corrispondente alla fase da attivare. Ogni intersezione di questo lavoro modella 12 corsie in ingresso (quattro direzioni cardinali, tre corsie

ciascuna: sinistra, diritto, destra) e 8 fasi semaforiche selezionabili. La Tabella 3.3 riporta la mappatura di tali fasi (Figura 3.8), adottando la nomenclatura N/S/E/W (*North/South/East/West*) e T/L (*Through*, diritto, e *Left*, sinistra).

Fase	Nome	Movimenti abilitati
0	NTST	Nord diritto+destra, Sud diritto+destra
1	NLSL	Nord sinistra, Sud sinistra
2	NTNL	Nord sinistra+diritto+destra
3	STSL	Sud sinistra+diritto+destra
4	WTET	Ovest diritto+destra, Est diritto+destra
5	WLEL	Ovest sinistra, Est sinistra
6	ETEL	Est sinistra+diritto+destra
7	WTWL	Ovest sinistra+diritto+destra

Tabella 3.3: Le 8 fasi semaforiche e i movimenti che ciascuna abilita.

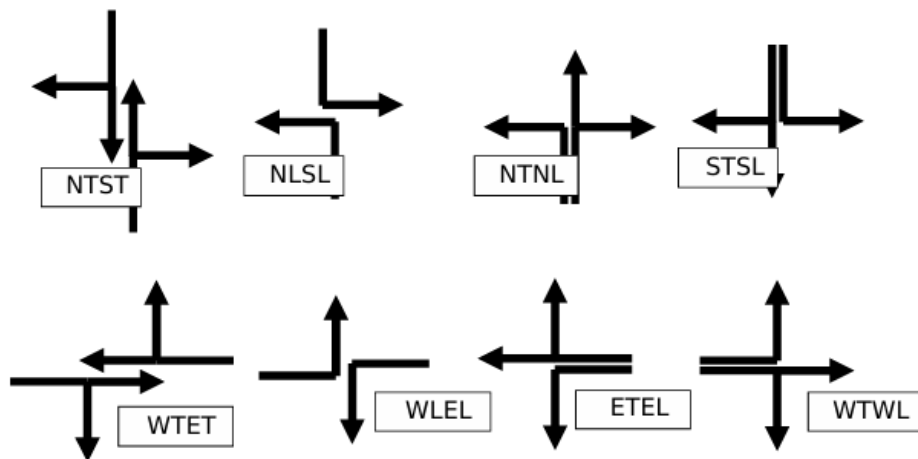


Figura 3.8: Rappresentazione grafica delle 8 fasi di Tabella 3.3: le frecce indicano i movimenti abilitati da ciascuna fase.

Il riferimento originale permette ai veicoli che svoltano a destra di muoversi liberamente, senza attendere il semaforo verde, a patto che non vi siano veicoli provenienti da altre direzioni. Al fine di prevenire una qualunque possibile interferenza tra diversi flussi e garantire un controllo più rigoroso, si è deciso di non adottare questo comportamento, permettendo la svolta a destra solo quando esplicitamente autorizzata dalla fase semaforica. L'insieme delle fasi sopra riportate rappresenta una configurazione tale da sfruttare al massimo lo spazio delle direzioni possibili, evitando al contempo conflitti tra le traiettorie dei veicoli.

3.5.2 Gestione dell'interverde

L'interverde è il periodo di tempo che intercorre tra la fine del verde di una fase semaforica e l'inizio del verde di quella successiva. Il suo scopo è quello di garantire la sicurezza stradale, permettendo ai veicoli che si trovano nell'intersezione di sgombrarla prima che venga attivata la fase successiva. Il motore di simulazione CITYFLOW non modella nativamente tale fase transitoria, nello specifico, non gestisce i semafori gialli e il conseguente passaggio sicuro tra due fasi consecutive; dunque l'interverde deve essere costruito esplicitamente. In questo lavoro, l'intervallo decisionale dell'agente (lo *step* di simulazione) ha una durata complessiva $\Delta t_{\text{step}} = 15$ s simulati, idealmente ripartita in tre sotto-fasi, con le stesse durate del riferimento originale: verde ($\Delta t_{\text{green}} = 10$ s), giallo ($\Delta t_{\text{yellow}} = 3$ s) e "tutto-rosso" ($\Delta t_{\text{red}} = 2$ s). Per il limite di CITYFLOW sopra descritto e in questo studio accettato come compromesso, l'ambiente assimila il giallo al verde senza imporre modelli di decelerazione specifici, mantenendo pertanto i movimenti autorizzati dalla policy per una durata totale di $\Delta t_{\text{green}} + \Delta t_{\text{yellow}} = 13$ s consecutivi. Al termine di questa finestra temporale, viene eseguita per $\Delta t_{\text{red}} = 2$ s una fase che non è presente tra le azioni selezionabili dal modello, ovvero il semaforo rosso in contemporanea su tutte le corsie. Questo intervallo di tutto-rosso viene eseguito a ogni step, indipendentemente dal fatto che la fase successiva coincida o meno con quella corrente: anche selezionare due volte di fila la stessa fase comporta i 2 s di tutto-rosso. Questo intervallo di guardia è necessario per garantire lo svuotamento dell'intersezione prima del cambio di fase, così da prevenire collisioni. A questo punto lo step giunge al termine e il modello formula una nuova decisione, scegliendo quale delle 8 fasi implementare nel ciclo successivo.

È importante sottolineare che questa accortezza riduce di fatto il tempo effettivo utile di decisione del singolo agente di controllo semaforico, in quanto due dei quindici secondi di ogni step sono dedicati alla fase "tutto-rosso" e quindi da considerarsi tempo morto (rappresentando circa il 13% del tempo totale).

3.5.3 Dataset e generazione parametrica

Ogni roadnet e ogni flusso di traffico usato in questo lavoro è generato parametricamente e in modo interamente deterministico da uno script dedicato, senza alcun campionamento empirico. Questo approccio consente di avere un pieno controllo sulle variabili operative come la struttura della rete, la densità veicolare e presenza di arterie principali, oltre a garantire l'esatta riproducibilità degli scenari di simula-

zione, un requisito metodologico fondamentale per un equo confronto tra modelli di controllo semaforico.

Topologie e spaziature. L'ambiente di simulazione implementa tre topologie a griglia regolare di dimensioni 4×4 (16 intersezioni), 5×5 (25 intersezioni) e 6×6 (36 intersezioni), impiegate per l'addestramento e per valutare la capacità di generalizzazione dei modelli rispetto alla scala spaziale.

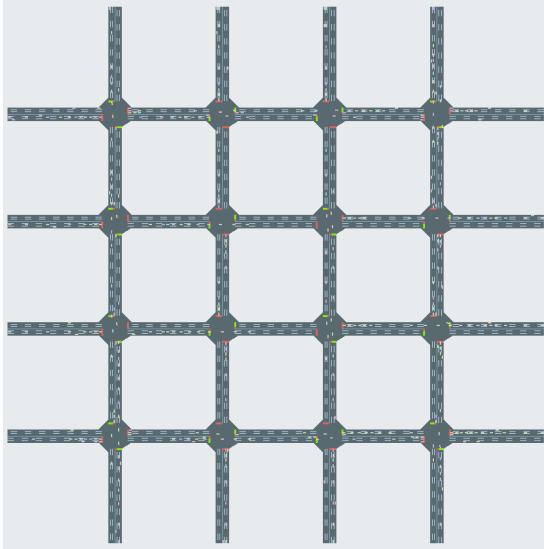
Un aspetto rilevante nella scelta della spaziatura tra incroci è la comparsa spontanea della cosiddetta onda verde, uno standard strutturale nell'ingegneria del traffico contemporanea: un flusso di veicoli che si propaga attraverso la rete senza fermarsi ai semafori. La spaziatura tra incroci è pertanto di due tipi: calibrata per l'onda verde e non calibrata. In assenza di offset temporali, in quanto le transizioni di fase sono sincrone per tutti i nodi della rete, l'unico modo per ottenere un'onda verde è calibrare la distanza fisica tra gli incroci sulla velocità di marcia e sulla durata del ciclo: alla velocità di riferimento $v_{\max} = 11.1$ m/s, in un ciclo di $\Delta t_{\text{step}} = 15$ s viene percorsa una distanza pari a 166.5 m. Dunque la prima spaziatura proposta è proprio di questa misura, ma ne faremo riferimento con l'etichetta "100m". La spaziatura non calibrata invece è 100m più estesa e viene indicata con "200m".

Le roadnet non condividono tutte lo stesso ruolo. La griglia 4×4 con spaziatura "100m" compare sia in training sia in test, mentre le altre tre combinazioni topologia-spaziatura restano riservate esclusivamente al test e non vengono mai incontrate durante l'addestramento (Tabella 3.4). Questa separazione serve a isolare due assi di generalizzazione: le griglie 5×5 e 6×6 misurano la tenuta della policy su reti più estese di quella di addestramento, con più intersezioni da coordinare, mentre la spaziatura "200m" applicata alla griglia 4×4 misura la tenuta su una topologia mai vista con incroci più distanziati, priva della calibrazione per l'onda verde descritta sopra.

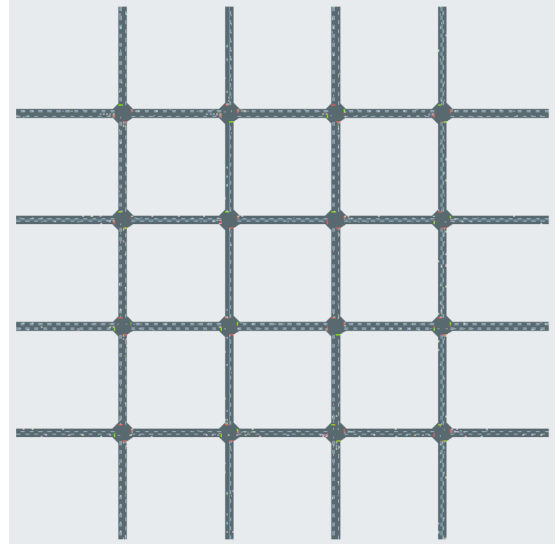
Griglia	Spaziatura	Utilizzo
4×4	100m	Train, Validation, Test
4×4	200m	Test
5×5	100m	Test
6×6	100m	Test

Tabella 3.4: Utilizzo di ciascuna combinazione topologia-spaziatura. Il dettaglio del confine tra training e validazione sulla griglia 4×4 a 100m è in Tabella 3.6.

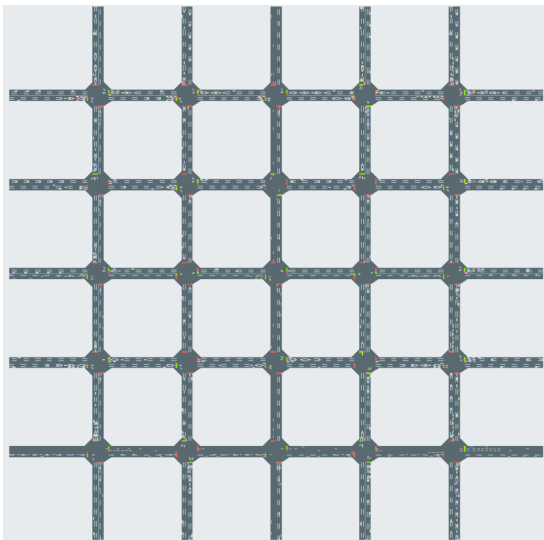
Percorsi veicolari. Il percorso di ciascun veicolo attraverso la rete non è fissato a priori, ma generato casualmente per ciascun punto di ingresso sul perimetro della



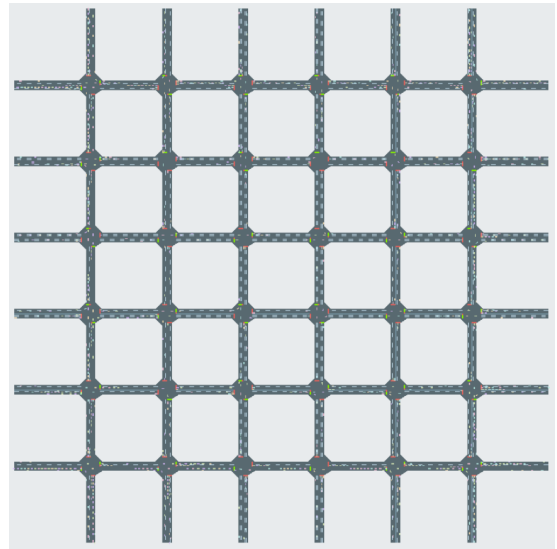
(a) 4×4 , 100m



(b) 4×4 , 200m



(c) 5×5 , 100m



(d) 6×6 , 100m

Figura 3.9: Rendering nel visualizzatore di CITYFLOW delle quattro combinazioni topologia-spaziatura di Tabella 3.4.

griglia: a ogni incrocio attraversato, il percorso prosegue dritto con probabilità 0.60, svolta a destra con probabilità 0.30 o svolta a sinistra con probabilità 0.10, finché non esce dai confini della rete o non ripercorrerebbe un incrocio già attraversato (evitando così percorsi ciclici). La probabilità complessiva di un percorso è il prodotto delle probabilità delle singole scelte che lo compongono; il numero di veicoli assegnato a ciascun percorso è proporzionale al suo peso relativo sul totale dei percorsi generati da quel punto di ingresso, calibrato per raggiungere il volume di traffico target di ciascuna fascia (Tabella 3.5).

Profili di traffico. Per ciascuna topologia sono stati generati tre profili di traffico: un flusso a intensità costante (*flat*), un profilo a gradino (*peak*, diviso in tre fasi temporali con un raddoppio del carico nel terzo centrale) e un profilo dinamico (*complete*) che simula il traffico di una giornata lavorativa (dettagliato nel seguito), impiegato esclusivamente in fase di addestramento. Ogni episodio ha una durata di $T_{\max} = 1800$ s simulati, corrispondenti a $1800 \text{ s} / \Delta t_{\text{step}} = 120$ cicli decisionali per agente.

I volumi dei veicoli immessi nella rete per ciascuna configurazione topologica sono stati determinati tramite una calibrazione empirica, così da garantire un livello di saturazione comparabile tra reti di dimensioni diverse e quindi una difficoltà del task di controllo invariata rispetto la scala topologica. In particolare la densità di ogni topologia è stata bilanciata affinché il *trip completion* ottenuto dalla baseline MAX-PRESSURE risultasse omogeneo (90%–94%).

Profilo di addestramento *complete*. A differenza degli scenari di validazione e test, caratterizzati da densità costanti o a singolo picco, il profilo di addestramento sintetizza l'evoluzione della domanda veicolare di un'intera giornata solare (24 ore) all'interno di un singolo episodio di 1800 s. Come dettagliato nella Tabella 3.5, il flusso è diviso in cinque fasce a intensità variabile: due fasce a basso carico (assimilabili agli orari notturni), due picchi di massima domanda (flussi di traffico di inizio e fine turno lavorativo) e un intervallo di raccordo centrale a regime moderatamente sostenuto. Questa marcata variazione di domanda veicolare all'interno di un singolo episodio è funzionale poiché costringe il modello ad adattare dinamicamente il piano di controllo, senza convergere verso una policy specializzata soltanto su un unico flusso di traffico.

Stocasticità e sbilanciamento direzionale. Per mitigare il rischio di *overfitting* e garantire la robustezza del modello, la generazione del profilo di addestramento integra due accorgimenti.

Fascia	Intervallo simulato	Step di decisione	Veicoli/secondo
Flat basso	0-525s	0-35	1.36
Peak basso	525-675s	35-45	3.67
Flat alto	675-1200s	45-80	2.93
Peak alto	1200-1425s	80-95	4.78
Flat basso	1425-1800s	95-120	1.27

Tabella 3.5: Le cinque fasce del profilo di addestramento *complete* (totale: 4.728 veicoli per **train1**; **train2** e **train3** seguono la stessa struttura con lievi variazioni tra seed, per un totale rispettivamente di 4.723 e 4.620 veicoli).

Innanzitutto, considerata la natura deterministica di CITYFLOW, addestrare un modello su una singola configurazione esporrebbe l'agente all'apprendimento mnemonico di un'esatta sequenza di veicoli. Per prevenire questa eventualità, si generano con diversi *seed* tre profili di traffico differenti, in modo da perturbare la distribuzione dei percorsi e gli istanti di immissione, pur mantenendo la medesima struttura generale descritta in Tabella 3.5; tali varianti vengono quindi alternate ciclicamente durante l'addestramento. Anche le configurazioni di validazione e test, su cui i modelli vengono valutati metodicamente, vengono generate con diversi *seed* (5 per configurazione), così da ottenere statistiche più robuste e informative sui risultati conseguiti.

In secondo luogo, per mettere alla prova la capacità del modello di adattarsi a situazioni locali di traffico differenti, è stata indotta un'asimmetria direzionale, designando una riga orizzontale della griglia come arteria principale (Est-Ovest), con un fattore di potenziamento di 3 che ne aumenta il peso nella distribuzione dei percorsi generati (senza alterare il volume totale di veicoli della configurazione). Tale arteria viene posta su una riga diversa per ogni variante di addestramento, al fine di scongiurare l'ipotesi che le policy sviluppate siano dipendenti dalla posizione dell'incrocio, mentre viene mantenuta fissa sulla stessa riga nelle configurazioni di validazione e di test, in modo da garantire la comparabilità dei risultati.

Training, validation e test. La Tabella 3.6 raccoglie in un unico prospetto tutte le combinazioni di griglia, spaziatura e profilo di traffico introdotte in questo lavoro, con il relativo utilizzo nella pipeline sperimentale; le varianti seed di validazione e test sono omesse, poiché non introducono combinazioni distinte da quelle elencate. Le tre configurazioni di training compaiono anche nella batteria di test, per un confronto diretto con la prassi, diffusa in letteratura, di valutare un modello sulla stessa rete su cui è stato addestrato (Capitolo 4). In tutto possiamo contare 11 configurazioni: 3 di training, 1 di validation e 7 di test.

Griglia	Spaziatura	Profilo di traffico	Utilizzo	Configurazione
4×4	100m	Complete (arteria riga 0)	Train, Test	config_4x4_100m_train1
4×4	100m	Complete (arteria riga 1)	Train, Test	config_4x4_100m_train2
4×4	100m	Complete (arteria riga 2)	Train, Test	config_4x4_100m_train3
4×4	100m	Flat	Validation	config_4x4_100m_6k_flat
4×4	100m	Peak	Test	config_4x4_100m_6k_peak
4×4	200m	Flat	Test	config_4x4_200m_6k_flat
4×4	200m	Peak	Test	config_4x4_200m_6k_peak
5×5	100m	Flat	Test	config_5x5_100m_9.4k_flat
5×5	100m	Peak	Test	config_5x5_100m_9.4k_peak
6×6	100m	Flat	Test	config_6x6_100m_11.5k_flat
6×6	100m	Peak	Test	config_6x6_100m_11.5k_peak

Tabella 3.6: Tutte le combinazioni di griglia, spaziatura e profilo di traffico impiegate in questo lavoro, a meno delle varianti seed di validazione e test, con il relativo utilizzo e il nome del file di configurazione corrispondente.

3.5.4 Assunzioni e limiti del modello di simulazione

Si esplicitano di seguito le semplificazioni strutturali adottate in fase di simulazione, finalizzate a ridurre la complessità computazionale e a rendere il problema trattabile:

- **Fasi transitorie:** Come precedentemente illustrato, l'intervallo semaforico di giallo è assimilato cinematicamente a un prolungamento del verde. La prevenzione dei conflitti è affidata unicamente alla fase di "tutto-rosso".
- **Omogeneità topologica:** L'analisi è confinata a topologie a griglia regolare, con incroci ortogonali e segmenti stradali di lunghezza uniforme. Tale astrazione permette di isolare l'impatto della scala dimensionale e della spaziatura sulle prestazioni di generalizzazione, rimuovendo le distorsioni causate dalle irregolarità geometriche tipiche delle reti urbane reali.
- **Flusso unimodale e omogeneo:** La simulazione modella esclusivamente il traffico veicolare privato. Non sono previsti pedoni, ciclisti, trasporti pubblici o mezzi di soccorso. Inoltre, il flusso è perfettamente omogeneo: ogni veicolo condivide i medesimi parametri cinematici (dimensione, accelerazione massima, velocità massima).
- **Limiti di velocità uniformi:** Il limite di velocità $v_{\max} = 11.1$ m/s è una costante globale, identica per ogni corsia della rete e per ogni veicolo generato, dunque sono assenti gerarchie stradali diverse.
- **Omogeneità degli incroci:** Ogni intersezione è vincolata a una geometria a quattro vie provvista di 8 fasi semaforiche, escludendo di fatto incroci con un numero diverso di vie.

- **Assenza di *lane-changing*:** Le corsie in approccio sono dedicate in modo esclusivo ai singoli movimenti (svolta a sinistra, dritto, svolta a destra). I veicoli vengono generati e si immettono automaticamente nella corsia compatibile con il loro percorso e il cambio di corsia (*lane-changing*) è una dinamica che non è stata implementata.
- **Viabilità bidirezionale:** L'infrastruttura di rete è composta esclusivamente da strade a doppio senso di marcia. La modellazione di strade a senso unico, per quanto frequente nei contesti urbani, non rientra nell'insieme delle configurazioni generate.

Capitolo 4

Esperimenti e risultati

Questo capitolo descrive il protocollo sperimentale di questa tesi e ne riporta i risultati. Definisce anzitutto le metriche di valutazione (§4.1) e il protocollo sperimentale (§4.2); presenta poi il confronto di base tra le baseline classiche e MetaSTGAT Pro insieme ai tre studi sperimentali di questa tesi, sul peso α della componente di equità, sull'ablation della procedura di addestramento e sui meccanismi di comunicazione spaziale alternativi (§4.3), e ne riporta i risultati (§4.4).

4.1 Metriche di valutazione

Nella letteratura sul Traffic Signal Control le metriche di valutazione proposte sono numerose. Questo capitolo introduce le 4 metriche principali utilizzate, coerentemente con il duplice obiettivo di questo lavoro, ovvero efficienza ed equità (come motivato nel Capitolo 1): il *travel time* (tempo di viaggio, medio e massimo) e il *trip completion* (veicoli che completano il percorso) per l'efficienza, il *max wait* (l'attesa massima registrata da un veicolo ad un incrocio) per l'equità. C'è da sottolineare che né *travel time* né *trip completion* sono metriche che appaiono esplicitamente nella ricompensa dei modelli. Pertanto qualunque risultato su di esse sarà l'indiretta conseguenza di come la reward ha indirizzato l'apprendimento della policy.

Travel time. Per ogni veicolo v , il tempo di viaggio è

$$tt_v = \begin{cases} t_{\text{arrivo}}(v) - t_{\text{spawn}}(v) & \text{se } v \text{ è arrivato entro la fine dell'episodio} \\ t_{\text{fine episodio}} - t_{\text{spawn}}(v) & \text{altrimenti (un lower bound del vero valore)} \end{cases} \quad (4.1)$$

Una delle due metriche cardine in questo studio è il travel time medio $\overline{tt}$, ovvero la media di tt_v calcolata sull'intera popolazione di veicoli generati nell'episodio. Tale

metrica non considera soltanto i veicoli arrivati, ma include anche i veicoli che non hanno completato il percorso entro la fine della simulazione. La ragione è evitare un bias di sopravvivenza: un modello che congestiona la rete e lascia arrivare solo una minoranza di veicoli su percorsi ottimizzati otterrebbe, calcolando la media solo sui veicoli arrivati, un risultato positivo ma non rappresentativo del comportamento reale della rete.

La seconda vera metrica cardine in questo lavoro di tesi è il travel time massimo $\overline{tt}_{\max}$, ovvero il massimo tra tutti i travel time dei veicoli generati, anche in questo caso indipendentemente dal completamento del percorso, per motivi analoghi a quelli descritti per il travel time medio.

Trip completion. Questa metrica misura la percentuale di veicoli che hanno completato il percorso entro la fine dell’episodio, escludendo dal totale i veicoli strutturalmente impossibilitati a farlo: quelli il cui tragitto, percorso a velocità massima costante e senza alcun arresto ai semafori, richiederebbe già da solo più dei secondi restanti alla fine dell’episodio a partire dal proprio istante di generazione. Questo sottoinsieme dipende solo dallo scenario di traffico (percorso e istante di generazione di ciascun veicolo), non dal controllore, e viene quindi calcolato una sola volta per configurazione e riutato per ogni modello. In questo studio questa metrica viene indicata con tc . Il dettaglio di tc per ogni singolo modello e ogni singola configurazione è in Tabella 4.10, a fine capitolo.

Max wait. Questa metrica riguarda il tempo più lungo che un veicolo trascorre su una singola corsia del proprio percorso. Sia w_v questo tempo per il veicolo v : l’intervallo tra l’istante in cui v entra in una corsia del suo tragitto e quello in cui supera l’incrocio a cui essa conduce, calcolato come il massimo su tutte le corsie attraversate lungo il percorso. Il conteggio su una corsia non si azzera se il veicolo avanza al suo interno, e prosegue anche se il veicolo si è fermato e ripartito una o più volte prima di superare l’incrocio; si interrompe soltanto quando la vettura lo supera. Il max wait dell’episodio è

$$\text{wait}_{\max} = \max_v w_v, \quad (4.2)$$

il massimo su tutti i veicoli generati nell’episodio.

Come per il travel time, include anche gli stalli ancora in corso al termine dell’episodio. Sulle configurazioni con spaziatura di 200 metri (§3.5.3), il conteggio è esteso all’intera corsia invece di limitarsi al campo visivo d_{vis} dei modelli di reinforcement learning (§3.1): un veicolo fermo oltre d_{vis} dal semaforo sarebbe altrimenti escluso

dal computo, sottostimando l’attesa peggiore effettivamente osservata sulla corsia. In questo studio $\max \text{wait}$ viene indicato con $\text{wait}_{\max}$.

La presente metrica non va confusa con MaxRedWait_i^t , il termine usato nella ricompensa (Capitolo 3) per penalizzare l’attesa durante il training. MaxRedWait_i^t è calcolato a ogni singolo passo di decisione, su una sola intersezione alla volta, ma soprattutto è limitata al campo visivo dell’incrocio. $\text{wait}_{\max}$ è invece calcolato una sola volta a fine episodio, come massimo su tutta la rete, e fa riferimento all’intera corsia.

4.2 Protocollo sperimentale

Ogni modello RL è addestrato sulle tre configurazioni di training presentate nella Tabella 3.6.

Seguendo un ciclo fisso, le tre configurazioni si alternano episodio per episodio. A fine training, l’ultimo checkpoint e quello che ha ottenuto il valore di loss più basso durante l’intero addestramento sono confrontati sulla configurazione di validazione `config_4x4_100m_6k_flat`, mediando $\overline{tt}$ su 5 seed diversi. Questa fase serve a evitare due problemi che si concatenano tra loro. Il primo è che in un ciclo di apprendimento online la politica non converge necessariamente in modo monotono, dunque gli ultimi episodi addestrati potrebbero allontanare la policy da un punto di minimo quantomeno locale. Il secondo è che la scelta del modello che ha registrato la loss minima non garantisce le capacità di generalizzazione del modello, in quanto una loss bassa potrebbe anche essere sintomo di una policy che ha soltanto memorizzato delle condizioni particolari delle configurazioni di training, senza però aver realmente appreso una politica in grado di generalizzare.

La selezione si basa sul solo $\overline{tt}$ e non sulle metriche di caso peggiore che questo lavoro tiene in primo piano altrove: mediare il travel time medio sui 5 seed di validazione dà già una stima ragionevolmente stabile del checkpoint migliore, mentre fare lo stesso con un massimo come $tt_{\max}$ o $\text{wait}_{\max}$ resterebbe una stima molto più rumorosa, perché un massimo risente della coda della distribuzione assai più di una media. È un limite pratico del protocollo, non una scelta di principio.

Il modello che risulta essere migliore in questa fase di selezione, viene poi utilizzato nella fase di test. Ogni valutazione, sia in validation sia nella batteria di test che descriveremo ora, avviene con $\varepsilon = 0$: nessuna esplorazione residua, solo la politica appresa allo stato puro. Inoltre i risultati ottenuti in ciascuna configurazione testata, sono mediati su 5 seed diversi, analogamente a quanto fatto per $\overline{tt}$ nella fase di validazione.

Misurare la capacità di generalizzazione dei modelli è uno degli obiettivi dichiarati di questo lavoro (Capitolo 1), un asse che gran parte della letteratura sul controllo semaforico appreso non copre sistematicamente, in quanto la prassi più diffusa, seguita anche dall’articolo originale di MetaSTGAT (Wang et al., 2022), è addestrare e valutare un modello sulla stessa rete stradale, senza mai trasferirlo a una topologia diversa.

Per questo motivo la batteria di test di questo lavoro include sia le 3 configurazioni di training sia 7 configurazioni aggiuntive mai viste durante l’addestramento. Le prime permettono un confronto diretto con la prassi di letteratura appena descritta, riportando un risultato ottenuto nelle stesse condizioni con cui la maggior parte degli altri lavori riporta il proprio; le seconde, assenti nella maggior parte degli studi citati, misurano invece se le prestazioni osservate si confermano anche con il cambio di topologia e di condizioni di traffico.

Ogni modello, RL o baseline, è dunque valutato sistematicamente sulle stesse 10 configurazioni: le 3 varianti di training stesse (per misurare la prestazione in-distribuzione) più 7 configurazioni mai viste in training (4x4 a 100m con profilo *peak*, 4x4 a 200m invece di 100m con densità sia *flat* sia *peak*, 5×5 e 6×6 a densità sia *flat* sia *peak*), usate per misurare la generalizzazione a profili di traffico, spaziatore e topologie mai incontrati in training.

4.3 Confronti proposti

Il confronto sperimentale di questo capitolo è organizzato in tre studi indipendenti, ciascuno dei quali varia una sola dimensione (il peso α , un gruppo di modifiche alla procedura, il meccanismo spaziale) rispetto al modello di riferimento ($\alpha = 0.08$, MetaSTGAT a un layer) a cui ci riferiremo con il termine *MetaSTGAT Pro*, lasciando invariato tutto il resto. Verranno analizzati nel seguito i tre studi proposti. Ciascun confronto conterrà al suo interno, come riferimento, le due baseline classiche, Fixed-Time e Max-Pressure, e il modello *MetaSTGAT Pro*.

4.3.1 Studio del peso α

Il primo studio (§4.4.2) pone a confronto tre varianti del modello MetaSTGAT Pro che differiscono solo per il valore del peso α riguardante la penalità anti-starvation nella ricompensa (Eq. 3.4). In particolare vengono valutati i valori di $\alpha \in \{0.08, 0.04, 0.00\}$. È il confronto più diretto per quantificare il compromesso tra efficienza ed equità dichiarato come obiettivo di questo lavoro. Variando solo questo iperparametro della ricompensa, senza toccare stato o architettura, osserveremo quanto costa in termini

di travel time medio ottenere una data riduzione del travel time massimo e del max wait.

4.3.2 Studio di ablazione

Il secondo confronto (§4.4.3) isola sistematicamente ogni blocco di differenze tra l'implementazione di questo lavoro e la procedura descritta nell'articolo originale di MetaSTGAT, tramite 6 preset che vanno dalla replica fedele del paper al modello MetaSTGAT Pro. È un confronto necessario perché né l'uno né l'altro estremo, da soli, spiegano il divario di prestazioni osservato tra i due: isolare un sottosistema alla volta permette di valutare il contributo di ciascuna modifica.

Ogni preset applica un gruppo coerente di scelte configurative, isolando un sottosistema alla volta rispetto al modello MetaSTGAT Pro (§3.4):

Preset	Reward	Cosa isola rispetto a Pro
pro	custom	—
temporal	custom	burn-in + BPTT sulla Meta-LSTM
single_dqn	custom	Double DQN
vanilla_buffer	custom	PER, Huber, grad clipping, warm-up, esplorazione ciclica
environment	paper	campo visivo, feature dello stato w_i^t , maschera fasi consecutive
paper	paper	Replica fedele di Wang et al. (2022)

Tabella 4.1: Preset di ablazione e sottosistema isolato da ciascuno. La reward **custom** è definita da Eq. (3.4), mentre la reward **paper** è definita da Eq. (3.3).

A differenza degli altri quattro preset, che modificano un solo sottosistema, **environment** cambia contemporaneamente osservazione (campo visivo, w_i^t , maschera) e ricompensa (torna a quella di Wang et al., 2022): i suoi risultati misurano quindi l'effetto congiunto delle due cose, non isolano un singolo meccanismo.

4.3.3 Studio sui meccanismi spaziali

Il terzo studio (§4.4.4) sostituisce il solo meccanismo di comunicazione spaziale tra intersezioni di MetaSTGAT Pro. Vengono testate le seguenti varianti già introdotte nel Capitolo 3.2.8:

- MetaSTGAT (il riferimento, a un layer come nella formulazione standard e a due layer)
- MetaSTGCN (a uno e due layer)

- MetaSTSONAR (con $L = 2$ e $L = 4$ iterazioni)

È il confronto che verifica se il beneficio osservato dipenda specificamente dal meccanismo di attenzione appreso, o se una qualunque forma di condivisione spaziale dell'informazione tra nodi vicini produca risultati equivalenti.

4.4 Risultati

I tre studi del precedente capitolo (§4.3) sono presentati secondo uno schema comune: una tabella mediata sulle 3 configurazioni di training, una corrispondente sulle 7 di test, un grafico a bande sovrapposte che le riassume visivamente, un plot della variazione percentuale delle metriche principali rispetto a Max-Pressure, gli eventuali casi particolari osservabili solo scendendo alla singola configurazione, chiusi da una discussione che include, dove il risultato non era quello atteso, una possibile spiegazione. In queste tabelle, dallo studio sul peso α in poi, sono evidenziati in grassetto i due risultati migliori di ogni colonna; nelle due tabelle del confronto di base (§4.4.1), che hanno solo tre righe, resta invece in grassetto il solo valore migliore.

Fixed-Time, Max-Pressure e MetaSTGAT Pro ($\alpha = 0.08$, il modello di riferimento di questo lavoro) compaiono in ciascuno dei tre studi. Dunque, per non ripetere la stessa discussione su questi tre riferimenti di base, la sottosezione seguente li confronta da soli, mentre le sottosezioni successive approfondiranno i rispettivi confronti (peso α , ablazione, meccanismi spaziali).

4.4.1 Fixed-Time, Max-Pressure e Pro: il confronto di base

Modello	$\bar{t}t$ (s)	$tt_{\max}$ (s)	$wait_{\max}$ (s)	tc (%)
Fixed-Time	339.3	1030.0	935.0	86.2
Max-Pressure	161.9	1070.0	1040.0	97.9
MetaSTGAT Pro ($\alpha = 0.08$)	172.8	630.0	425.0	97.4

Tabella 4.2: Travel time medio, massimo, max wait e trip completion dei tre controllori presenti in ogni studio di questo capitolo, mediati sulle 3 configurazioni di training.

Fixed-Time è il controllore peggiore su quasi ogni metrica di riferimento, $wait_{\max}$ compreso: la rotazione fissa delle fasi non produce alcun vantaggio nemmeno in training, dove Max-Pressure e MetaSTGAT Pro lo battono nettamente su ogni metrica. Non osservando mai lo stato della rete, resta comunque il riferimento più semplice rispetto a cui ogni strategia reattiva o appresa può essere confrontata.

Modello	$\bar{t}t$ (s)	$tt_{\max}$ (s)	$wait_{\max}$ (s)	tc (%)
Fixed-Time	460.7	1496.1	1311.0	84.0
Max-Pressure	238.3	1647.4	1636.7	91.9
MetaSTGAT Pro ($\alpha = 0.08$)	241.0	1017.4	792.0	93.8

Tabella 4.3: Come Tabella 4.2, mediata sulle 7 configurazioni di test.

Tra Max-Pressure e MetaSTGAT Pro il confronto è più sfumato. In training e in test Max-Pressure resta il modello con il travel time medio più basso in assoluto (rispettivamente 161.9s e 238.3s), coerente con la sua ottimalità di portata a livello di rete; MetaSTGAT Pro paga per questo un travel time leggermente peggiore sia in training (172.8s, +6.7%) sia in test (241.0s, +1.1%). Un’eccezione a questo quadro emerge scendendo alle singole configurazioni di test: sulle due griglie 6×6 , le più estese testate e mai viste in training, MetaSTGAT Pro registra un travel time medio inferiore a quello di Max-Pressure (238.1s contro 244.7s in *flat*, 275.1s contro 283.3s in *peak*), l’unico caso tra le 10 configurazioni in cui questo avviene. Lo stesso pattern, sulle stesse due configurazioni, si osserva anche per $\alpha = 0.04$ e per `vanilla_buffer` (§4.4.3), il che rende meno probabile si tratti di rumore. Una spiegazione plausibile è che, sulla rete più estesa e mai vista in training, il coordinamento appreso tra intersezioni inizi a incidere anche sull’efficienza media, mentre Max-Pressure, privo di qualunque nozione della rete oltre l’incrocio corrente, perda parte del proprio vantaggio quando la rete cresce oltre la scala di training; questo lavoro non arriva a verificare se il divario continui a crescere su reti ancora più estese. Per quanto riguarda il $wait_{\max}$ il rapporto si inverte nettamente: MetaSTGAT Pro registra un valore il 59% più basso di Max-Pressure in training e il 52% più basso in test (Tabelle 4.2-4.3). Questi risultati introducono il compromesso al centro di questo lavoro, discusso nella sua forma completa al variare di α nella prossima sottosezione. Il vantaggio relativo di MetaSTGAT Pro sul $wait_{\max}$ di Max-Pressure è più ampio in training (59%) che in test (52%); una lettura cauta è che parte di questo vantaggio si eroda, come atteso, su condizioni mai incontrate in addestramento, senza che questo permetta di per sé conclusioni più forti sulle capacità di generalizzazione del modello.

Sul completamento (tc) la separazione più netta è quella tra Fixed-Time e tutti gli altri modelli di questo capitolo: 86.2%/84.0% (training/test) contro un intervallo compreso tra il 94.4% e il 97.9% in training e tra l’89.0% e il 95.8% in test per ogni altra variante (Tabella 4.10, a fine capitolo); l’unica eccezione parziale è MetaSTGCN, il cui completamento più basso è discusso a parte in §4.4.4. Con questo unico scarto rilevante già osservato qui, le sottosezioni seguenti non ripetono la discussione di

tc configurazione per configurazione: si limitano a segnalarne la tendenza generale, rimandando alla stessa tabella per il dettaglio completo.

4.4.2 Studio del peso α : risultati

Questa sottosezione confronta i due controllori di riferimento (§4.4.1) con le tre varianti del modello Pro che differiscono solo per il peso α della penalità anti-starvation, con $\alpha \in \{0.08, 0.04, 0.00\}$.

Modello	$\bar{tt}$ (s)	$tt_{\max}$ (s)	$\text{wait}_{\max}$ (s)
Fixed-Time	339.3	1030.0	935.0
Max-Pressure	161.9	1070.0	1040.0
MetaSTGAT Pro ($\alpha = 0.08$)	172.8	630.0	425.0
MetaSTGAT Pro ($\alpha = 0.04$)	170.7	685.0	560.0
MetaSTGAT Pro ($\alpha = 0.00$)	173.5	740.0	555.0

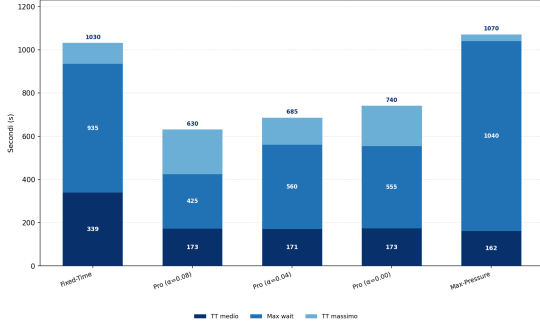
Tabella 4.4: Travel time medio, massimo e max wait, mediati sulle 3 configurazioni di training.

Modello	$\bar{tt}$ (s)	$tt_{\max}$ (s)	$\text{wait}_{\max}$ (s)
Fixed-Time	460.7	1496.1	1311.0
Max-Pressure	238.3	1647.4	1636.7
MetaSTGAT Pro ($\alpha = 0.08$)	241.0	1017.4	792.0
MetaSTGAT Pro ($\alpha = 0.04$)	241.1	1054.7	930.9
MetaSTGAT Pro ($\alpha = 0.00$)	247.5	1164.4	1012.3

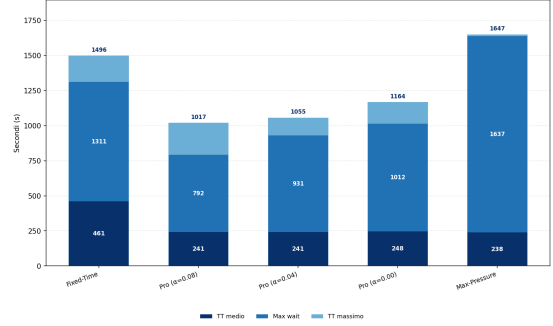
Tabella 4.5: Come Tabella 4.4, mediata sulle 7 configurazioni di test.

Sul travel time medio ciò che verrebbe naturale attendersi è che al crescere del valore di α la metrica in questione peggiori, con le due configurazioni prive di penalità anti-starvation, Max-Pressure e $\alpha = 0.00$, che risulterebbero essere le migliori, in quanto nessuna delle due paga un compromesso con l’equità, quindi nessuna dovrebbe peggiorare travel time medio in cambio di un vantaggio che non persegue. I dati non confermano questa attesa in modo netto: $\alpha = 0.00$ non è la variante Pro con il travel time medio più basso, né in training né in test (173.5s in training contro 172.8s di $\alpha = 0.08$ e 170.7s di $\alpha = 0.04$; 247.5s in test contro 241.0s e 241.1s), ma il distacco dalle altre due varianti non è sistematico come atteso. Una causa plausibile: con $\alpha = 0$ il termine di ricompensa legato all’attesa scompare, ma il vettore w_i^t resta nello stato osservato dalla rete (§3.1); la rete continua quindi a vedere un segnale che la ricompensa non penalizza più, un disallineamento tra stato e obiettivo che può disturbare l’addestramento.

Tra le tre varianti Pro, $\alpha = 0.04$ resta quella con il travel time medio più basso in training (170.7s), pur restando dietro a Max-Pressure in entrambi i regimi.

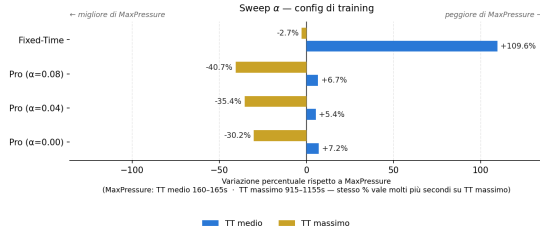


(a) Configurazioni di training

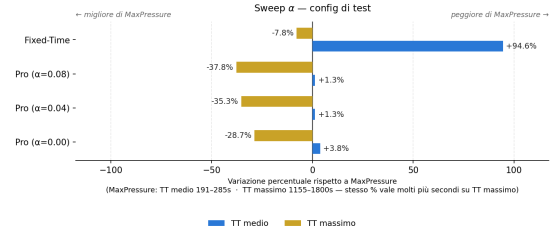


(b) Configurazioni di test

Figura 4.1: Travel time medio (fascia scura), max wait (fascia intermedia) e travel time massimo (fascia chiara) per lo studio sistematico su α , stessi dati delle Tabelle 4.4-4.5.



(a) Configurazioni di training



(b) Configurazioni di test

Figura 4.2: Variazione percentuale di travel time medio e massimo rispetto a Max-Pressure (riferimento a 0%) per lo studio su α , calcolata per singola configurazione e poi mediata.

Su $tt_{\max}$ il quadro è quello atteso: la metrica peggiora in modo strettamente monotono al diminuire di α , cioè al ridursi del peso dato al caso di starvation più estremo, in entrambi i regimi. Su $wait_{\max}$ l'andamento è quasi monotono: in test $\alpha = 0.00$ (1012.3s) resta peggiore di $\alpha = 0.04$ (930.9s) come atteso, ma in training i due valori sono sostanzialmente pari (555.0s contro 560.0s, una differenza compatibile con la variabilità tra seed). Il riferimento $\alpha = 0.08$ ottiene comunque sia il $tt_{\max}$ sia il $wait_{\max}$ più bassi in entrambi i regimi, il 59% in meno di Max-Pressure in training e il 52% in meno in test. In test sia $\alpha = 0.04$ sia $\alpha = 0.00$ restano nettamente sotto Fixed-Time su entrambe le metriche ($wait_{\max}$ rispettivamente 930.9s e 1012.3s, $tt_{\max}$ 1054.7s e 1164.4s, contro 1311.0s e 1496.1s di Fixed-Time): rimuovere del tutto la penalità anti-starvation non avvicina le prestazioni di caso peggiore a quelle di un controllo non adattivo.

Nonostante il lieve vantaggio di $\alpha = 0.04$ sul travel time medio in training, il resto di questo lavoro adotta $\alpha = 0.08$ come riferimento standard per gli studi di ablazione e dei meccanismi spaziali (§4.4.3, §4.4.4). La ragione è la priorità dichiarata tra le metriche di questo lavoro: $\alpha = 0.04$ è sistematicamente meno incisivo di $\alpha = 0.08$ su $wait_{\max}$ e $tt_{\max}$, le due metriche di caso peggiore che questa tesi tiene in primo piano, a fronte di un guadagno sul solo travel time medio, la meno critica delle tre e comunque assente in test. $\alpha = 0.08$ resta quindi la scelta più coerente con l’obiettivo del lavoro, anche al costo di qualche secondo in più di travel time medio.

4.4.3 Studio di ablazione: risultati

Questa sottosezione confronta il modello Pro ($\alpha = 0.08$, riferimento) con i cinque preset di ablation presenti nella Tabella 4.1: **paper** (replica fedele del riferimento originale), **environment** (campo di visibilità, w_i^t e maschera azioni rimossi), **temporal** (burn-in e BPTT disattivati), **single_dqn** (Double DQN sostituita da DQN) e **vanilla_buffer** (PER, Huber, soft update, gradient clipping, warm-up ed esplorazione ciclica disattivati), tutti allo stesso $\alpha = 0.08$ del riferimento.

Modello	$\bar{tt}$ (s)	$tt_{\max}$ (s)	$wait_{\max}$ (s)
Fixed-Time	339.3	1030.0	935.0
Max-Pressure	161.9	1070.0	1040.0
Pro ($\alpha = 0.08$, riferimento)	172.8	630.0	425.0
paper	187.8	950.0	915.0
environment	188.3	1335.0	1290.0
temporal	176.0	715.0	560.0
single_dqn	171.9	600.0	485.0
vanilla_buffer	171.5	695.0	560.0

Tabella 4.6: Travel time medio, massimo e max wait sui preset di ablazione, mediati sulle 3 configurazioni di training.

Sul travel time medio in training **vanilla_buffer**, **single_dqn** e il modello Pro restano in una fascia stretta (171.5-172.8s), con **temporal** poco distante (176.0s); **environment** se ne stacca già in questo regime (188.3s), non solo in test. In test **vanilla_buffer** e Pro restano quasi coincidenti (241.0-241.1s), seguiti da **single_dqn** (243.2s) e **temporal** (245.9s), mentre **environment** e **paper** restano nettamente distanti (268.6s e 270.4s). Il distacco di **environment** non è concentrato sulle due configurazioni a 200m come si potrebbe attendere dalla sola disattivazione del cutoff di visibilità: la variazione percentuale rispetto al riferimento Pro, calcolata per singola configurazione, è di circa il 10% sulle due config a 200m e di circa il 12%

Modello	$\bar{tt}$ (s)	$tt_{\max}$ (s)	$wait_{\max}$ (s)
Fixed-Time	460.7	1496.1	1311.0
Max-Pressure	238.3	1647.4	1636.7
Pro ($\alpha = 0.08$, riferimento)	241.0	1017.4	792.0
paper	270.4	1553.6	1513.3
environment	268.6	1495.3	1463.1
temporal	245.9	1129.3	940.7
single_dqn	243.2	1051.7	917.6
vanilla_buffer	241.1	1044.9	804.0

Tabella 4.7: Come Tabella 4.6, mediata sulle 7 configurazioni di test.

sulle restanti cinque config di test (4×4 a 100m, 5×5 , 6×6), un divario distribuito in modo pressoché uniforme su tutte le configurazioni fuori distribuzione, non specifico della spaziatura. Questo rende meno convincente l'ipotesi che il solo cutoff di visibilità sia responsabile del calo: **environment** disattiva contemporaneamente anche w_i^t nello stato, la maschera anti-starvation e il termine di attesa nella ricompensa (Tabella 3.2), e l'effetto osservato è più probabilmente il risultato congiunto di queste rimozioni che di una sola.

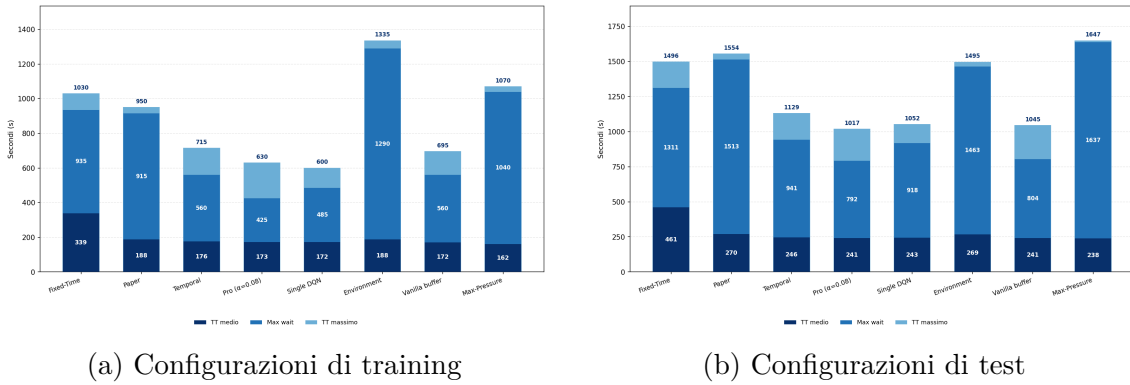
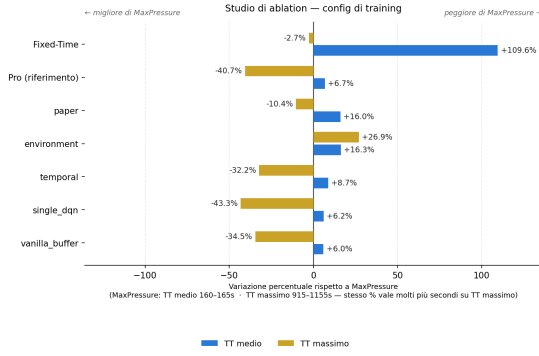


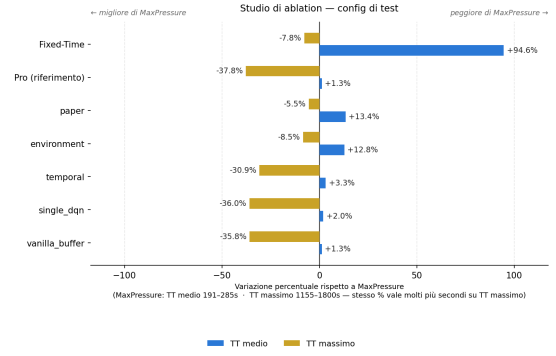
Figura 4.3: Travel time medio, max wait e travel time massimo sui preset di ablation, stessi dati delle Tabelle 4.6-4.7.

Su $tt_{\max}$ e $wait_{\max}$, **paper** ed **environment** restano nettamente indietro in entrambi i regimi, coerentemente con l'unico elemento che condividono e che differisce da ogni altro preset: l'assenza del termine legato all'attesa nella ricompensa.

Sempre su queste due metriche, in training **single_dqn** ottiene il miglior $tt_{\max}$ in assoluto (600.0s), con il modello Pro secondo (630.0s); sul $wait_{\max}$ è invece Pro a ottenere il miglior risultato del confronto (425.0s), davanti a **single_dqn** (485.0s). **temporal**, che disattiva solo burn-in e BPTT, è qui il preset con il $tt_{\max}$ peggiore tra questi quattro (715.0s) e pareggia **vanilla_buffer** sul $wait_{\max}$ (560.0s).



(a) Configurazioni di training



(b) Configurazioni di test

Figura 4.4: Variazione percentuale di travel time medio e massimo rispetto a MaxPressure (riferimento a 0%) sui preset di ablation, calcolata per singola configurazione/seed e poi mediata.

In test **temporal** resta peggiore di Pro su entrambe le metriche (1129.3s contro 1017.4s di $tt_{\max}$, 940.7s contro 792.0s di $wait_{\max}$). Il divario cresce gradualmente con la scala della griglia, calcolato per singola configurazione rispetto al riferimento Pro: circa il 10% sulle tre configurazioni 4×4 , l'11% sulla 5×5 , il 12% sulla 6×6 . Una causa plausibile resta l'assenza di burn-in e BPTT: senza propagazione del gradiente lungo la sequenza né un periodo di adattamento dello stato ricorrente all'inizio di ogni episodio, il modulo temporale della Meta-LSTM ha meno modo di generalizzare il proprio stato interno a condizioni diverse da quelle di training; questo lavoro non dispone però di un'evidenza indipendente, oltre all'andamento per configurazione appena descritto, che permetta di isolare questa causa da altre possibili.

In test Pro ottiene il $tt_{\max}$ e il $wait_{\max}$ migliori anche tra **single_dqn** e **vanilla_buffer** (1017.4s e 792.0s, contro 1051.7s e 917.6s di **single_dqn**, 1044.9s e 804.0s di **vanilla_buffer**): questi due preset non mantengono alcun vantaggio sistematico sul riferimento.

4.4.4 Studio sui meccanismi spaziali: risultati

Questa sottosezione confronta il modello Pro (MetaSTGAT, un layer) con le sue varianti a meccanismo spaziale alternativo definite in §3.2.8: MetaSTGAT a due layer, MetaSTGCN a uno e due layer, e MetaSTSONAR con $L = 2$ e $L = 4$ ricorrenze. Tutti i modelli condividono il resto dell'architettura, gli iperparametri e la procedura di training.

Il divario più netto di questo confronto separa MetaSTGCN dagli altri due meccanismi. In test, entrambe le varianti MetaSTGCN registrano valori nettamente peggiori su ogni colonna (travel time medio 281.1-348.3s contro 241.0-252.7s di Me-

Modello	$\bar{tt}$ (s)	$tt_{\max}$ (s)	$\text{wait}_{\max}$ (s)
Fixed-Time	339.3	1030.0	935.0
Max-Pressure	161.9	1070.0	1040.0
MetaSTGAT, 1L (Pro, riferimento)	172.8	630.0	425.0
MetaSTGAT, 2L	181.5	685.0	525.0
MetaSTGCN, 1L	260.6	1655.0	1625.0
MetaSTGCN, 2L	201.6	865.0	835.0
MetaSTSONAR, $L=2$	179.3	710.0	575.0
MetaSTSONAR, $L=4$	172.1	660.0	475.0

Tabella 4.8: Travel time medio, massimo e max wait sui meccanismi spaziali, mediati sulle 3 configurazioni di training.

Modello	$\bar{tt}$ (s)	$tt_{\max}$ (s)	$\text{wait}_{\max}$ (s)
Fixed-Time	460.7	1496.1	1311.0
Max-Pressure	238.3	1647.4	1636.7
MetaSTGAT, 1L (Pro, riferimento)	241.0	1017.4	792.0
MetaSTGAT, 2L	252.7	1037.6	890.6
MetaSTGCN, 1L	348.3	1749.0	1738.7
MetaSTGCN, 2L	281.1	1233.9	1170.9
MetaSTSONAR, $L=2$	249.8	1134.0	956.6
MetaSTSONAR, $L=4$	246.8	1092.9	932.6

Tabella 4.9: Come Tabella 4.8, mediata sulle 7 configurazioni di test.

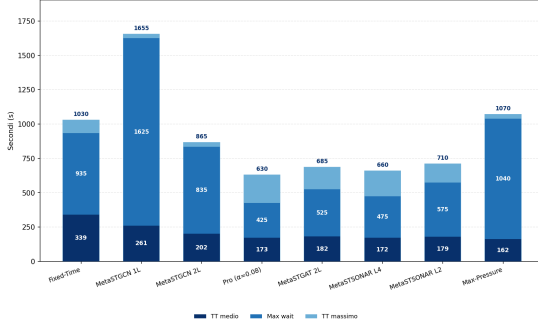
taSTGAT/MetaSTSONAR); MetaSTGCN a un layer supera persino Max-Pressure in negativo su $tt_{\max}$ e $wait_{\max}$ (1749.0s e 1738.7s, contro 1647.4s e 1636.7s di Max-Pressure), pur trattandosi di un modello addestrato con lo stesso obiettivo di equità del riferimento. Anche sul completamento MetaSTGCN a un layer resta la sola eccezione oltre a Fixed-Time già anticipata in §4.4.1 (88.5% in training, 85.4% in test, contro il 94.4-97.9%/89.0-95.8% di tutte le altre varianti). La differenza concettuale descritta in §3.2.8.1, un peso di aggregazione fissato dalla sola topologia contro un punteggio di compatibilità appreso per coppia di nodi, si traduce quindi in un divario di prestazioni ampio su questo compito. Un secondo layer in MetaSTGCN migliora la variante a singolo layer su tutte le metriche riportate in entrambi i regimi ($\overline{tt}$ di training 260.6→201.6s, $tt_{\max}$ di test 1749.0→1233.9s, $wait_{\max}$ di test 1738.7→1170.9s): la profondità aggiuntiva compensa qui, almeno in parte, l'assenza di un meccanismo di compatibilità appreso, pur restando lontana dalle prestazioni di MetaSTGAT e MetaSTSONAR.

MetaSTGAT e MetaSTSONAR hanno prestazioni pressoché pari tra loro su travel time medio; sul $wait_{\max}$ di test, però, è il modello Pro di riferimento a ottenere il risultato migliore dell'intero confronto (792.0s), davanti sia alla propria variante a due layer (890.6s) sia a entrambe le profondità di MetaSTSONAR (956.6s e 932.6s).

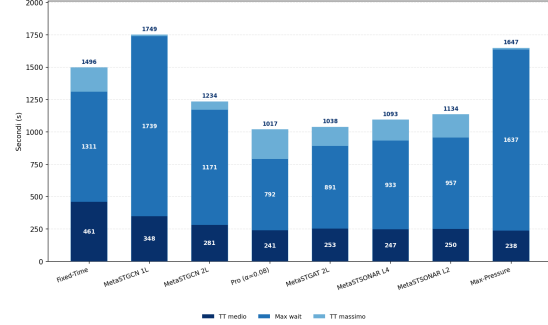
Tra le due profondità di MetaSTSONAR, in questo studio è $L = 4$ a risultare leggermente migliore di $L = 2$ sul $tt_{\max}$ in entrambi i regimi (660.0s contro 710.0s in training, 1092.9s contro 1134.0s in test), coerente con il vantaggio teorico di campo recettivo più ampio atteso da una profondità doppia.

Aggiungere un secondo layer a MetaSTGAT, a differenza di MetaSTGCN, peggiora qui le prestazioni su ogni metrica riportata: travel time medio (172.8s → 181.5s in training, 241.0s → 252.7s in test), $tt_{\max}$ (630.0s → 685.0s in training, 1017.4s → 1037.6s in test) e $wait_{\max}$ (425.0s → 525.0s in training, 792.0s → 890.6s in test, +12%).

Un caso particolare nelle configurazioni di training (Tabella 4.13): MetaSTGCN a un layer ottiene un $wait_{\max}$ medio di 1625s, peggiore di Max-Pressure stesso (1040s), pur condividendo con il riferimento la stessa ricompensa orientata all'equità. È il solo modello di tutto questo lavoro, tra quelli addestrati con il termine anti-starvation pieno, a non riuscire a battere la baseline reattiva su questa metrica nel proprio stesso regime di addestramento, coerentemente con la spiegazione architetture data sopra: un peso di aggregazione fissato dalla topologia non ha modo di imparare a dare priorità al segnale del vicino più in sofferenza, per quanto la ricompensa lo incentivi a farlo.

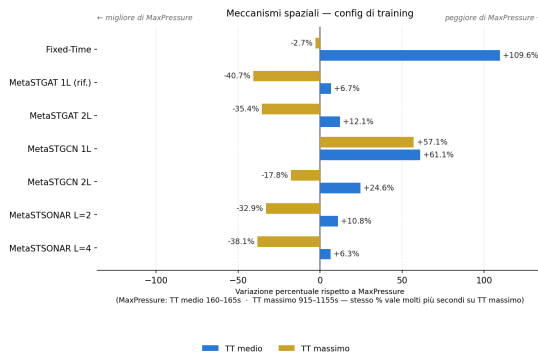


(a) Configurazioni di training

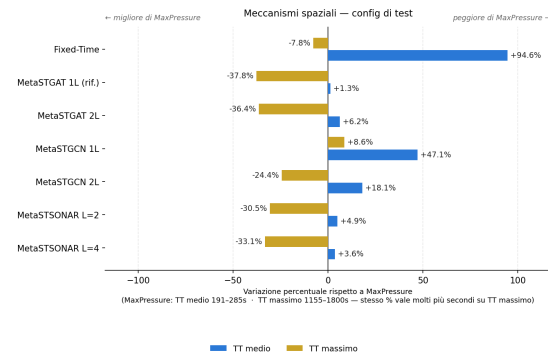


(b) Configurazioni di test

Figura 4.5: Travel time medio, max wait e travel time massimo sui meccanismi spaziali, stessi dati delle Tabelle 4.8-4.9.



(a) Configurazioni di training



(b) Configurazioni di test

Figura 4.6: Variazione percentuale di travel time medio e massimo rispetto a Max-Pressure (riferimento a 0%) sui meccanismi spaziali, calcolata per singola configurazione/seed e poi mediata.

4.4.5 Tabelle dei risultati

Sono qui di seguito riportati i risultati completi di tutti i modelli rispetto a ciascuna configurazione testata.

Modello	Train 1	Train 2	Train 3	4x4 Pk	4x4-200 F	4x4-200 Pk	5x5 F	5x5 Pk	6x6 F	6x6 Pk
Fixed-Time	84.8	85.7	88.1	85.5	85.1	82.9	86.1	83.8	83.3	81.6
Max-Pressure	98.0	98.3	97.2	92.9	93.5	92.2	92.4	91.4	91.2	89.6
Pro ($\alpha=0.08$)	97.3	97.8	97.1	94.0	95.4	93.1	94.6	92.6	94.0	93.0
Pro ($\alpha=0.04$)	97.7	97.7	97.1	94.0	95.8	93.2	94.4	93.5	93.7	93.2
Pro ($\alpha=0.00$)	96.6	97.2	97.2	93.9	95.4	93.2	94.2	92.7	93.3	91.8
MetaSTGCN 2L	96.5	97.3	96.9	93.2	95.3	92.1	93.7	92.6	93.2	92.5
Ablation: paper	95.0	96.6	96.3	91.8	94.4	90.8	91.9	91.2	91.4	89.4
Ablation: temporal	96.8	97.4	97.3	93.5	95.7	93.4	93.6	91.8	92.9	92.0
Ablation: single_dqn	97.3	97.7	97.8	93.9	95.5	92.8	94.5	92.6	93.9	90.4
Ablation: environment	95.4	96.1	96.5	92.1	93.4	91.2	92.0	89.5	91.3	89.0
Ablation: vanilla_buffer	96.1	97.7	97.5	93.9	95.7	93.3	94.2	93.1	93.4	92.4
MetaSTGCN 1L	89.1	88.5	87.9	85.7	84.2	82.8	87.4	86.4	87.6	83.9
MetaSTGCN 2L	95.0	94.4	96.3	91.4	93.7	91.1	92.2	90.6	91.7	90.3
MetaSTSONAR L=2	96.9	97.8	96.8	93.9	95.7	93.5	93.8	92.8	93.5	92.3
MetaSTSONAR L=4	97.3	97.7	97.9	93.9	95.4	93.4	93.9	92.3	93.0	92.0

Tabella 4.10: Percentuale di trip completion t_c per ogni modello e ogni configurazione, corretta escludendo dal denominatore i veicoli strutturalmente impossibilitati ad arrivare in tempo. Nessun modello scende sotto l'80%; a parte Fixed-Time (baseline non adattiva) l'unico caso ricorrente sotto il 90% è MetaSTGCN 1L.

Modello	Train 1	Train 2	Train 3	4x4 Pk	4x4-200 F	4x4-200 Pk	5x5 F	5x5 Pk	6x6 F	6x6 Pk
Fixed-Time	351.6	342.9	323.3	437.3 ± 6.4	415.5 ± 6.7	438.8 ± 4.9	459.7 ± 4.7	485.0 ± 5.1	484.8 ± 6.2	503.7 ± 4.6
Max-Pressure	165.4	160.2	160.0	218.8 ± 2.3	197.3 ± 4.2	247.8 ± 2.6	217.8 ± 3.3	258.6 ± 4.9	244.7 ± 5.5	283.3 ± 1.4
Pro ($\alpha=0.08$)	178.0	174.8	165.5	229.4 ± 4.1	202.8 ± 3.2	253.3 ± 3.4	220.7 ± 5.0	268.0 ± 7.5	238.1 ± 4.2	275.1 ± 3.4
Pro ($\alpha=0.04$)	175.1	171.7	165.2	229.3 ± 2.8	202.4 ± 2.6	251.7 ± 2.5	221.4 ± 4.8	264.1 ± 3.0	242.6 ± 2.6	276.2 ± 3.2
Pro ($\alpha=0.00$)	177.8	169.5	173.1	230.8 ± 2.7	201.3 ± 6.3	250.5 ± 3.5	226.5 ± 6.5	275.0 ± 6.2	253.4 ± 4.0	295.0 ± 7.8
MetaSTGAT 2L	188.8	177.6	178.1	244.8 ± 3.2	208.3 ± 3.6	262.6 ± 3.4	234.6 ± 4.9	279.1 ± 1.7	247.4 ± 5.8	291.9 ± 6.4
Ablation: paper	194.1	182.9	186.5	253.9 ± 4.6	217.7 ± 3.6	287.3 ± 4.8	245.3 ± 11.4	294.5 ± 5.2	275.8 ± 2.6	318.6 ± 3.5
Ablation: temporal	181.7	173.0	173.4	234.4 ± 3.9	203.7 ± 4.9	253.0 ± 3.4	224.8 ± 7.1	272.9 ± 4.5	246.3 ± 5.0	285.8 ± 5.6
Ablation: single_dqn	179.0	172.9	163.8	228.1 ± 3.6	201.1 ± 4.4	253.3 ± 2.8	219.9 ± 3.2	270.2 ± 8.1	237.7 ± 4.9	291.9 ± 7.3
Ablation: environment	195.8	186.8	182.4	253.5 ± 6.9	222.9 ± 4.5	279.1 ± 3.9	242.2 ± 7.6	291.9 ± 2.6	274.1 ± 8.4	316.5 ± 7.8
Ablation: vanilla_buffer	177.9	170.2	166.5	230.3 ± 2.9	202.2 ± 4.1	252.4 ± 4.2	220.1 ± 4.5	264.8 ± 3.9	238.8 ± 3.2	278.9 ± 6.4
MetaSTGCN 1L	257.4	262.4	261.9	335.7 ± 9.4	328.4 ± 15.3	360.2 ± 5.5	323.4 ± 22.7	361.6 ± 3.7	331.9 ± 12.2	397.2 ± 13.9
MetaSTGCN 2L	206.7	205.9	192.2	277.0 ± 6.4	228.6 ± 6.1	280.6 ± 3.9	259.9 ± 4.3	317.8 ± 2.0	279.6 ± 4.5	324.3 ± 5.9
MetaSTSONAR L=2	181.3	174.4	182.2	234.6 ± 3.4	206.1 ± 5.4	258.3 ± 2.4	229.0 ± 3.9	276.9 ± 2.8	251.1 ± 4.6	292.9 ± 7.5
MetaSTSONAR L=4	176.5	170.5	169.3	232.2 ± 3.9	204.4 ± 7.6	252.0 ± 3.9	227.1 ± 5.1	273.0 ± 3.6	248.9 ± 3.5	290.2 ± 1.6

Tabella 4.11: Travel time medio $\overline{t\bar{t}}$ (s) per ogni modello e ogni configurazione. Sulle configurazioni di test, media $\pm$ deviazione standard su 5 seed diversi.

Modello	Train 1	Train 2	Train 3	4x4 Pk	4x4-200 F	4x4-200 Pk	5x5 F	5x5 Pk	6x6 F	6x6 Pk
Fixed-Time	1020.0	975.0	1095.0	1410.0 $\pm$ 49.3	1431.0 $\pm$ 30.9	1290.0 $\pm$ 19.0	1620.0 $\pm$ 32.9	1482.0 $\pm$ 17.5	1653.0 $\pm$ 32.0	1587.0 $\pm$ 37.2
Max-Pressure	915.0	1155.0	1140.0	1194.0 $\pm$ 20.3	1692.0 $\pm$ 133.6	1446.0 $\pm$ 105.0	1800.0 $\pm$ 0.0	1800.0 $\pm$ 0.0	1800.0 $\pm$ 0.0	1800.0 $\pm$ 0.0
Pro ($\alpha=0.08$)	600.0	600.0	690.0	861.0 $\pm$ 45.1	774.0 $\pm$ 55.8	918.0 $\pm$ 51.4	1074.0 $\pm$ 13.7	1083.0 $\pm$ 66.7	1269.0 $\pm$ 87.3	1143.0 $\pm$ 36.0
Pro ($\alpha=0.04$)	660.0	705.0	690.0	924.0 $\pm$ 48.0	819.0 $\pm$ 33.7	1002.0 $\pm$ 112.8	1092.0 $\pm$ 53.2	1092.0 $\pm$ 49.7	1260.0 $\pm$ 28.5	1194.0 $\pm$ 30.3
Pro ($\alpha=0.00$)	720.0	645.0	855.0	1044.0 $\pm$ 30.9	840.0 $\pm$ 47.4	954.0 $\pm$ 12.0	1269.0 $\pm$ 94.2	1149.0 $\pm$ 56.6	1482.0 $\pm$ 163.1	1413.0 $\pm$ 207.9
MetaSTGAT 2L	660.0	600.0	795.0	915.0 $\pm$ 44.5	786.0 $\pm$ 48.9	930.0 $\pm$ 26.8	1161.0 $\pm$ 51.6	1101.0 $\pm$ 51.6	1224.0 $\pm$ 68.8	1146.0 $\pm$ 35.0
Ablation: paper	930.0	975.0	945.0	1212.0 $\pm$ 142.7	1257.0 $\pm$ 93.1	1326.0 $\pm$ 145.3	1800.0 $\pm$ 0.0	1794.0 $\pm$ 12.0	1800.0 $\pm$ 0.0	1686.0 $\pm$ 152.0
Ablation: temporal	735.0	585.0	825.0	987.0 $\pm$ 48.7	861.0 $\pm$ 72.6	963.0 $\pm$ 44.9	1191.0 $\pm$ 95.6	1203.0 $\pm$ 63.9	1470.0 $\pm$ 112.2	1230.0 $\pm$ 37.9
Ablation: single_dqn	600.0	585.0	615.0	825.0 $\pm$ 34.2	759.0 $\pm$ 47.1	948.0 $\pm$ 133.0	1092.0 $\pm$ 128.5	1122.0 $\pm$ 65.3	1251.0 $\pm$ 45.1	1365.0 $\pm$ 148.2
Ablation: environment	1425.0	1080.0	1500.0	1239.0 $\pm$ 173.0	1440.0 $\pm$ 153.6	1296.0 $\pm$ 54.2	1638.0 $\pm$ 161.4	1605.0 $\pm$ 103.5	1599.0 $\pm$ 185.1	1650.0 $\pm$ 195.3
Ablation: vanilla_buffer	660.0	675.0	750.0	954.0 $\pm$ 38.7	795.0 $\pm$ 42.4	936.0 $\pm$ 43.1	1161.0 $\pm$ 102.4	1071.0 $\pm$ 33.7	1215.0 $\pm$ 26.8	1182.0 $\pm$ 22.0
MetaSTGCN 1L	1740.0	1425.0	1800.0	1776.0 $\pm$ 48.0	1791.0 $\pm$ 18.0	1626.0 $\pm$ 179.4	1800.0 $\pm$ 0.0	1743.0 $\pm$ 114.0	1791.0 $\pm$ 18.0	1716.0 $\pm$ 104.6
MetaSTGCN 2L	930.0	855.0	810.0	1125.0 $\pm$ 82.7	1011.0 $\pm$ 144.4	978.0 $\pm$ 29.1	1512.0 $\pm$ 204.0	1275.0 $\pm$ 124.1	1458.0 $\pm$ 49.7	1278.0 $\pm$ 74.3
MetaSTSONAR L=2	705.0	600.0	825.0	957.0 $\pm$ 25.8	873.0 $\pm$ 77.3	1047.0 $\pm$ 29.1	1392.0 $\pm$ 99.5	1128.0 $\pm$ 42.8	1314.0 $\pm$ 38.7	1227.0 $\pm$ 69.3
MetaSTSONAR L=4	585.0	570.0	825.0	933.0 $\pm$ 43.9	846.0 $\pm$ 123.9	927.0 $\pm$ 34.7	1287.0 $\pm$ 99.2	1134.0 $\pm$ 54.2	1320.0 $\pm$ 51.1	1203.0 $\pm$ 6.0

Tabella 4.12: Travel time massimo $t_{\max}$ (s) per ogni modello e ogni configurazione. Sulle configurazioni di test, media $\pm$ deviazione standard su 5 seed diversi.

Modello	Train 1	Train 2	Train 3	4x4 Pk	4x4-200 F	4x4-200 Pk	5x5 F	5x5 Pk	6x6 F	6x6 Pk
Fixed-Time	930.0	900.0	975.0	1047.0 $\pm$ 100.6	1053.0 $\pm$ 75.5	1014.0 $\pm$ 64.8	1602.0 $\pm$ 36.0	1335.0 $\pm$ 74.7	1629.0 $\pm$ 48.9	1497.0 $\pm$ 66.7
Max-Pressure	885.0	1125.0	1110.0	1164.0 $\pm$ 37.5	1680.0 $\pm$ 138.1	1419.0 $\pm$ 98.4	1800.0 $\pm$ 0.0	1794.0 $\pm$ 12.0	1800.0 $\pm$ 0.0	1800.0 $\pm$ 0.0
Pro ($\alpha=0.08$)	465.0	360.0	450.0	591.0 $\pm$ 39.8	549.0 $\pm$ 86.2	756.0 $\pm$ 105.0	726.0 $\pm$ 17.2	933.0 $\pm$ 149.8	948.0 $\pm$ 64.2	1041.0 $\pm$ 77.4
Pro ($\alpha=0.04$)	510.0	630.0	540.0	846.0 $\pm$ 45.1	630.0 $\pm$ 93.7	885.0 $\pm$ 157.9	765.0 $\pm$ 83.2	1026.0 $\pm$ 97.5	1182.0 $\pm$ 102.4	1182.0 $\pm$ 36.0
Pro ($\alpha=0.00$)	525.0	435.0	705.0	888.0 $\pm$ 95.1	690.0 $\pm$ 45.5	813.0 $\pm$ 50.5	951.0 $\pm$ 125.0	1062.0 $\pm$ 71.9	1320.0 $\pm$ 212.8	1362.0 $\pm$ 202.0
MetaSTGAT 2L	480.0	495.0	600.0	714.0 $\pm$ 61.2	564.0 $\pm$ 43.1	792.0 $\pm$ 92.2	975.0 $\pm$ 209.4	1017.0 $\pm$ 121.3	1077.0 $\pm$ 179.6	1095.0 $\pm$ 74.1
Ablation: paper	885.0	945.0	915.0	1071.0 $\pm$ 109.7	1215.0 $\pm$ 91.5	1278.0 $\pm$ 135.3	1800.0 $\pm$ 0.0	1788.0 $\pm$ 24.0	1800.0 $\pm$ 0.0	1641.0 $\pm$ 171.4
Ablation: temporal	555.0	510.0	615.0	786.0 $\pm$ 98.4	609.0 $\pm$ 110.9	783.0 $\pm$ 85.6	846.0 $\pm$ 73.9	1089.0 $\pm$ 121.7	1326.0 $\pm$ 155.5	1146.0 $\pm$ 51.6
Ablation: single_dqn	525.0	435.0	495.0	696.0 $\pm$ 56.6	606.0 $\pm$ 43.1	807.0 $\pm$ 185.0	897.0 $\pm$ 51.9	1047.0 $\pm$ 78.5	1020.0 $\pm$ 100.8	1350.0 $\pm$ 159.9
Ablation: environment	1395.0	1050.0	1425.0	1179.0 $\pm$ 190.8	1413.0 $\pm$ 163.1	1260.0 $\pm$ 50.2	1614.0 $\pm$ 166.7	1581.0 $\pm$ 118.7	1557.0 $\pm$ 219.5	1638.0 $\pm$ 201.3
Ablation: vanilla_buffer	585.0	570.0	525.0	753.0 $\pm$ 85.1	594.0 $\pm$ 50.7	777.0 $\pm$ 59.5	768.0 $\pm$ 235.3	837.0 $\pm$ 17.5	834.0 $\pm$ 99.8	1065.0 $\pm$ 70.4
MetaSTGCN 1L	1710.0	1395.0	1770.0	1752.0 $\pm$ 59.5	1776.0 $\pm$ 40.9	1602.0 $\pm$ 171.9	1800.0 $\pm$ 0.0	1743.0 $\pm$ 114.0	1785.0 $\pm$ 30.0	1713.0 $\pm$ 109.2
MetaSTGCN 2L	900.0	825.0	780.0	1086.0 $\pm$ 122.4	861.0 $\pm$ 192.9	897.0 $\pm$ 77.9	1443.0 $\pm$ 244.9	1233.0 $\pm$ 127.4	1419.0 $\pm$ 72.6	1257.0 $\pm$ 74.3
MetaSTSONAR L=2	600.0	420.0	705.0	771.0 $\pm$ 78.6	645.0 $\pm$ 93.4	948.0 $\pm$ 32.0	1098.0 $\pm$ 328.8	975.0 $\pm$ 99.9	1110.0 $\pm$ 152.1	1149.0 $\pm$ 60.4
MetaSTSONAR L=4	480.0	435.0	510.0	786.0 $\pm$ 118.7	609.0 $\pm$ 123.9	807.0 $\pm$ 91.2	1101.0 $\pm$ 240.9	1116.0 $\pm$ 63.4	930.0 $\pm$ 164.9	1179.0 $\pm$ 49.8

Tabella 4.13: Max wait $\text{wait}_{\max}$ (s) per ogni modello e ogni configurazione. Sulle configurazioni di test, media $\pm$ deviazione standard su 5 seed diversi. Sulle config 4x4-200, valori a piena corsia.

Capitolo 5

Conclusioni

Il Capitolo 1 ha dichiarato tre obiettivi per questo lavoro: efficienza, equità e generalizzazione. Questo capitolo li riprende uno a uno alla luce dei risultati del Capitolo 4, indica le direzioni lasciate aperte e discute infine i limiti dello studio.

Il confronto di base (§4.4.1) conferma che sia MaxPressure sia MetaSTGAT Pro battono nettamente Fixed-Time su ogni metrica, con l'unica eccezione del max wait in training, dove la rotazione fissa delle fasi di Fixed-Time impone per costruzione un limite all'attesa che le configurazioni di training, meno dense, non arrivano a violare, limite che però sparisce non appena il traffico si fa più intenso in test. Tra MaxPressure e Pro il compromesso al centro di questo lavoro emerge già a questo livello. MaxPressure resta la scelta più efficiente in media, Pro riduce il $wait_{\max}$ del 34-41% pur restando entro pochi punti percentuali su travel time medio.

Lo studio sul peso α quantifica questo compromesso direttamente. Al diminuire di α , il $wait_{\max}$ e $tt_{\max}$ peggiorano in modo pressoché monotono, un gradiente netto e prevedibile: più equità si chiede alla ricompensa, più il controllore la garantisce. Il travel time medio non peggiora invece in modo altrettanto sistematico. $\alpha = 0.04$ lo riduce persino sotto quello di MaxPressure in test, il primo caso in questo lavoro in cui una variante RL supera MaxPressure su questa metrica in generalizzazione.

Il caso $\alpha = 0$ va però letto con un'avvertenza, perché non è il punto finale di quel gradiente ma un caso a parte. Rimuovere del tutto la penalità, lasciando intatti stato, maschera e vincolo di visibilità, produce un blocco quasi totale di un veicolo per l'intera durata dell'episodio su quattro dei cinque seed della griglia 5×5 più densa mai vista in training, il risultato peggiore misurato in questo lavoro tra i modelli con il resto della procedura invariato. Questo non dimostra però che rimuovere il termine di equità comprometta sempre e comunque la garanzia osservata: è più probabile che qui a fallire sia lo specifico allineamento tra stato e ricompensa di questo modello. Il vettore w_i^t , il segnale di attesa per corsia, resta nello

stato osservato dalla rete anche quando $\alpha = 0$ ne azzerava il peso nell’obiettivo, per cui l’agente continua a vedere un segnale che l’addestramento non penalizza più, un disallineamento che probabilmente confonde l’ottimizzazione più di quanto farebbe la sola assenza del termine di equità. Per questo motivo, nonostante $\alpha = 0.04$ sia leggermente più efficiente in media, questo lavoro adotta $\alpha = 0.08$ come riferimento per gli studi successivi, coerentemente con la priorità dichiarata tra le metriche: è la configurazione più incisiva proprio sulle due metriche di caso peggiore che questo lavoro tiene in primo piano.

Lo studio di ablation isola un effetto complementare. Dei cinque preset confrontati, solo i due che rimuovono interamente il segnale di attesa dallo stato o dalla ricompensa (`paper` ed `environment`) degradano marcatamente verso il comportamento di MaxPressure, mentre le tre varianti che disattivano singolarmente meccanismi dell’algoritmo di apprendimento (`temporal`, `single_dqn`, `vanilla_buffer`) restano vicine al riferimento o lo superano su alcune metriche. Su 35 istanze di test, `paper` ed `environment` sono gli unici, insieme a MaxPressure, a bloccare un veicolo per quasi l’intera durata dell’episodio; nessuna delle varianti che conserva stato e ricompensa lo fa mai. La garanzia di equità di questo lavoro dipende quindi dalla combinazione di segnale di attesa nello stato e termine di ricompensa dedicato, non da un singolo dettaglio implementativo dell’algoritmo di apprendimento sottostante.

Il confronto tra meccanismi spaziali aggiunge una terza componente necessaria. A parità di stato, ricompensa e maschera, MetaSTGCN (l’unico meccanismo a peso di aggregazione fissato dalla sola topologia, non appreso per coppia di nodi) resta sistematicamente indietro su ogni metrica, in un caso peggiore persino di MaxPressure nel proprio stesso regime di addestramento. Equità ed efficienza non sono quindi solo una funzione della ricompensa, richiedono anche un meccanismo di comunicazione capace di instradare il segnale del vicino in difficoltà fino a chi può agire su di esso.

Ogni modello di questo lavoro è addestrato una sola volta su una griglia 4×4 e valutato, senza riaddestramento, su topologie più grandi (5×5 , 6×6), su una spaziatura stradale diversa e su profili di traffico differenti. Il risultato principale è positivo ma non uniforme. Sulle configurazioni mai viste in training, MetaSTGAT Pro conserva il proprio vantaggio su MaxPressure e Fixed-Time su tutte le metriche principali, e le varianti che disattivano meccanismi dell’algoritmo di apprendimento (`single_dqn`, `vanilla_buffer`) mostrano un vantaggio sul riferimento concentrato quasi esclusivamente su queste topologie più grandi. Alla scala di training, lo stesso vantaggio si riduce o si inverte a seconda della metrica, segno che quei meccanismi aiutano soprattutto la generalizzazione di scala, non le prestazioni in senso assoluto.

Un secondo asse di generalizzazione, la spaziatura stradale, rivela un meccanismo distinto. Il preset `environment`, che tra le altre cose disattiva il vincolo sul campo visivo, degrada in modo sproporzionato proprio sulle due configurazioni con spaziatura di 200 metri, mai viste in training a nessuna spaziatura diversa da 100 metri. La causa più diretta è che quel vincolo limita per costruzione lo scarto tra la distribuzione di training e quella di test; senza di esso, il modello osserva conteggi di veicoli mai incontrati durante l’addestramento. Lo stesso pattern, attenuato, si osserva nel margine di vantaggio di `single_dqn` e `vanilla_buffer` sul max wait, che si restringe quasi a scomparire sulle configurazioni a 200 metri pur restando ampio sulle griglie più grandi. Generalizzare a una scala di rete diversa e generalizzare a una spaziatura fisica diversa non sono la stessa cosa, e i meccanismi che aiutano l’una non aiutano necessariamente l’altra.

Il caso di MetaSTSONAR chiude il quadro con un risultato meno atteso: a parità di ogni altro fattore, $L = 2$ ricorrenze di propagazione superano $L = 4$ su $tt_{\max}$ sia in training sia in test, quando il vantaggio teorico di un campo recettivo più ampio suggerirebbe l’opposto. Poiché lo svantaggio di $L = 4$ è già presente alla scala di training, dove non c’è alcun cambio di topologia da spiegare, la causa non può essere una generalizzazione peggiore. È più plausibile un limite legato all’ottimizzazione, dato che il gradiente deve retropropagare attraverso il doppio dei passi dell’equazione di propagazione a parità di procedura di training, un compito via via più difficile quanto più L cresce. Questo lavoro non arriva a isolare in modo conclusivo se sia davvero questa la causa.

I risultati discussi finora aprono la strada a diverse direzioni di ricerca, quasi tutte riconducibili alle stesse semplificazioni strutturali dichiarate nella descrizione dell’ambiente di simulazione (Capitolo 3). Un primo esperimento a basso costo sarebbe aumentare il numero di episodi di addestramento oltre i cento usati in questo lavoro, per verificare se lo svantaggio di MetaSTSONAR a $L = 4$ discusso sopra sia davvero un limite di ottimizzazione o solo una convergenza più lenta che un budget di addestramento maggiore risolverebbe da sé.

Le altre direzioni riguardano la generalizzazione a reti stradali strutturalmente diverse da quelle qui testate. Le griglie di questo lavoro hanno segmenti stradali di lunghezza uniforme, un vincolo che una rete con strade di lunghezza eterogenea rimuoverebbe, mettendo alla prova un modello il cui campo visivo e la cui nozione di distanza tra nodi sono stati calibrati proprio su quella uniformità. Le intersezioni sono inoltre tutte a quattro vie e ad angolo retto, mentre una griglia irregolare, con angoli diversi da novanta gradi o incroci a tre o cinque vie, metterebbe alla prova la struttura a fasi fisse qui adottata. Le vie a senso unico cambierebbero a loro volta

la struttura dei conflitti tra movimenti alla base della stessa definizione di fase, un asse di variazione distinto da quelli già esplorati. Più in generale, ciascuna delle semplificazioni elencate tra le assunzioni del modello di simulazione, dai limiti di velocità uniformi su ogni strada alla loro relativa omogeneità, è un candidato naturale per un futuro studio di generalizzazione, così come l'estensione della valutazione a topologie stradali reali oltre alle griglie sintetiche qui utilizzate.

Vale la pena di notare come molte di queste differenze strutturali, la lunghezza di una strada, il suo limite di velocità, l'angolo di un incrocio, potrebbero essere incorporate senza modificare l'architettura di base, aggiungendole semplicemente alle meta-feature già usate dal meta-learner di questo lavoro per condizionare i pesi di ciascun nodo sulle sue caratteristiche locali. Un'ultima direzione, di natura diversa dalle altre, è l'introduzione di mezzi pubblici e pedoni accanto al solo traffico veicolare privato modellato in questo lavoro: un'estensione che richiederebbe fasi e segnali dedicati, non la sola aggiunta di una nuova meta-feature.

Nonostante i risultati ottenuti, alcuni limiti vanno considerati nell'applicazione pratica di questo lavoro. Ogni modello, RL o ablation, deriva da un singolo run di addestramento. La batteria a più seed introdotta nel protocollo sperimentale ripete la sola valutazione, non l'addestramento, per cui buona parte dei risultati più sottili discussi in questo capitolo resta compatibile, oltre che con le spiegazioni proposte, con la varianza tra run di addestramento indipendenti diversi dall'unico osservato. Il confronto tra meccanismi spaziali, come già notato, non pareggia il numero di parametri tra le tre famiglie confrontate. Una parte del divario osservato, specialmente a sfavore di MetaSTGCN, potrebbe in principio riflettere anche questa differenza, non solo la capacità espressiva del meccanismo di aggregazione in sé. La valutazione di generalizzazione resta inoltre confinata a reti sintetiche a griglia regolare, generate parametricamente, mai a una rete stradale reale, un limite che questo lavoro condivide con la maggior parte della letteratura citata nell'introduzione, e che la sola estensione a griglie più grandi non risolve. Allo stesso modo, CITYFLOW privilegia la velocità di simulazione sul realismo microscopico del singolo veicolo (§2.5). Risultati assoluti su un simulatore più dettagliato, o su dati reali, potrebbero differire pur lasciando invariate le conclusioni relative tra i controllori confrontati. Infine, il confronto tra modelli in questo lavoro non include un test di significatività statistica in senso classico. La consistenza osservata su più configurazioni di test e più seed di valutazione rende i risultati qualitativamente solidi, non un sostituto formale di un test di ipotesi.

Bibliografia

- Davide Bacciu, Federico Errica, Alessio Micheli, and Marco Podda. A gentle introduction to deep learning for graphs. *Neural Networks*, 129:203–221, 2020. doi: 10.1016/j.neunet.2020.06.006.
- Michael M. Bronstein, Joan Bruna, Taco Cohen, and Petar Veličković. Geometric deep learning: Grids, groups, graphs, geodesics, and gauges. *arXiv preprint arXiv:2104.13478*, 2021.
- Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder-decoder for statistical machine translation. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, 2014.
- Tianshu Chu, Jie Wang, Lara Codecà, and Zhaojian Li. Multi-agent deep reinforcement learning for large-scale traffic signal control. *arXiv preprint arXiv:1903.04527*, 2019.
- Andrea Cini, Ivan Marisca, Daniele Zambon, and Cesare Alippi. Graph deep learning for time series forecasting. *ACM Computing Surveys*, 57(12):321, 2025. doi: 10.1145/3742784.
- Samah El-Tantawy, Baher Abdulhai, and Hossam Abdelgawad. Multiagent reinforcement learning for integrated network of adaptive traffic signal controllers (marlin-atsc): Methodology and large-scale application on downtown toronto. *IEEE Transactions on Intelligent Transportation Systems*, 14(3):1140–1150, 2013. doi: 10.1109/TITS.2013.2255286.
- David Ha, Andrew Dai, and Quoc V. Le. Hypernetworks. *arXiv preprint arXiv:1609.09106*, 2016.
- Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.

- Steven Kapturowski, Georg Ostrovski, John Quan, Remi Munos, and Will Dabney. Recurrent experience replay in distributed reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2019.
- Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.
- Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.
- Qimai Li, Zhichao Han, and Xiao-Ming Wu. Deeper insights into graph convolutional networks for semi-supervised learning. *arXiv preprint arXiv:1801.07606*, 2018. AAAI-2018.
- Yuxi Li. Deep reinforcement learning. *arXiv preprint arXiv:1810.06339*, 2018.
- Zachary C. Lipton, John Berkowitz, and Charles Elkan. A critical review of recurrent neural networks for sequence learning. *arXiv preprint arXiv:1506.00019*, 2015.
- Pablo Alvarez Lopez, Michael Behrisch, Laura Bieker-Walz, Jakob Erdmann, Yun-Pang Flötteröd, Robert Hilbrich, Leonhard Lüken, Johannes Rummel, Peter Wagner, and Evamarie Wießner. Microscopic traffic simulation using sumo. In *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*, 2018. doi: 10.1109/ITSC.2018.8569938.
- Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- Afshin Oroojlooy, Davood Hajinezhad, Mohammadreza Nazari, and Jorge Silva. Attendlight: Universal attention-based reinforcement learning model for traffic signal control. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020.
- Ning Qian. On the momentum term in gradient descent learning algorithms. *Neural Networks*, 12(1):145–151, 1999. doi: 10.1016/S0893-6080(98)00116-6.
- David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986. doi: 10.1038/323533a0.

- Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. In *International Conference on Learning Representations (ICLR)*, 2016.
- Andalib Shams, A. M. Tahsin Emtenan, and Christopher M. Day. A taxonomy of adaptive traffic signal control. *IEEE Access*, 2018.
- Chao Sun, Yuhao Yang, Jiacheng Li, Weiyi Fang, and Peng Zhang. A multi-agent regional traffic signal control system integrating traffic flow prediction and graph attention networks. *Systems*, 14(1):47, 2026. doi: 10.3390/systems14010047.
- Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018.
- Tijmen Tieleman and Geoffrey Hinton. Lecture 6.5-rmsprop: Divide the Gradient by a Running Average of Its Recent Magnitude, 2012. COURSERA: Neural Networks for Machine Learning, lecture notes.
- Alessandro Trenta, Alessio Gravina, and Davide Bacciu. Sonar: Long-range graph propagation through information waves. In *39th Conference on Neural Information Processing Systems (NeurIPS 2025)*, 2025.
- Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2016.
- Pravin Varaiya. Max pressure control of a network of signalized intersections. *Transportation Research Part C: Emerging Technologies*, 36:177–195, 2013.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018.
- Jing Wang, Xiaopeng Wang, Yang Mo, Chao Huang, Yanan Li, Guanghe Li, and Hao Zhang. Attention-based spatio-temporal graph convolutional networks-enhanced deep reinforcement learning for adaptive traffic signal control in urban traffic networks. *Scientific Reports*, 2026. doi: 10.1038/s41598-026-61371-3.

- Min Wang, Libing Wu, Man Li, Dan Wu, Xiaochuan Shi, and Chao Ma. Meta-learning based spatial-temporal graph attention network for traffic signal control. *Knowledge-Based Systems*, 250:109166, 2022. doi: 10.1016/j.knosys.2022.109166.
- Xianfeng Wang, Zachary Jerome, Ziran Wang, Chao Zhang, Shengmin Shen, Vidhya Kumar, Fan Bai, Piotr Krajewski, Danielle Deneau, Abdullatif Jawad, Robert Jones, Greg Piotrowicz, and Henry X. Liu. Traffic light optimization with low penetration rate vehicle trajectory data. *Nature Communications*, 15:1306, 2024. doi: 10.1038/s41467-024-45427-4.
- Hua Wei, Nan Xu, Huichu Zhang, Guanjie Zheng, Xinshi Zang, Chacha Chen, Weinan Zhang, Yanmin Zhu, Kai Xu, and Zhenhui Li. Colight: Learning network-level cooperation for traffic signal control. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM)*, 2019. doi: 10.1145/3357384.3357902.
- Hua Wei, Guanjie Zheng, Vikash Gayah, and Zhenhui Li. A survey on traffic signal control methods. *arXiv preprint arXiv:1904.08117*, 2020.
- Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S. Yu. A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 32(1):4–24, 2021. doi: 10.1109/TNNLS.2020.2978386.
- Feng Xiao, Jiaming Lu, Lu Li, Wenwen Tu, and Chaojing Li. Advances in reinforcement learning for traffic signal control: a review of recent progress. *Intelligent Transportation Infrastructure*, 2025:liaf009, 2025. doi: 10.1093/iti/liaf009.
- Huichu Zhang, Siyuan Feng, Chang Liu, Yaoyao Ding, Yichen Zhu, Zihan Zhou, Weinan Zhang, Yong Yu, Haili Jin, and Zhenhui Li. Cityflow: A multi-agent reinforcement learning environment for large scale city traffic scenario. In *The World Wide Web Conference (WWW)*, 2019. doi: 10.1145/3308558.3314139.
- Huichu Zhang, Chang Liu, Weinan Zhang, Guanjie Zheng, and Yong Yu. Generalight: Improving environment generalization of traffic signal control via meta reinforcement learning. In *Proceedings of the 29th ACM International Conference on Information and Knowledge Management (CIKM)*, pages 1783–1792, 2020. doi: 10.1145/3340531.3411859.
- Liang Zhang, Qiang Wu, Jun Shen, Linyuan Lu, Bo Du, and Jianqing Wu. Expression might be enough: representing pressure and demand for reinforcement

learning based traffic signal control. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, 2022.

Yuli Zhang, Shangbo Wang, Dongyao Jia, Pengfei Fan, Ruiyuan Jiang, Hankang Gu, and Andy H. F. Chow. Toward dependency dynamics in multi-agent reinforcement learning for traffic signal control. *arXiv preprint arXiv:2502.16608*, 2025.